Building Production-Grade Data Pipelines on Google Cloud

A Developer’s Field Guide to the gcp-pipeline-framework

Joseph Aruja

2026

Contents

Building Production-Grade Data Pipelines on Google Cloud

A Developer's Field Guide to the gcp-pipeline-framework

Published by Good Shepherd Press, an imprint of Good Shepherd Software Consultancy Limited, United Kingdom.

First edition: 2026.

ISBN: `<ISBN>`

VAT registration: GB `<VAT>`

Cover and interior diagrams: Joseph Aruja, using TikZ on LaTeX.

Typeset with Pandoc and XeLaTeX.

Body type: DejaVu Serif. Headings and diagrams: DejaVu Sans. Code: DejaVu Sans Mono.

For corrections, errata, or licensing enquiries: joseph.a.aruja@gmail.com

Preface

I wrote my first line of professional Java in February 2001 — a Java EE point-of-sale system on EJB 2.0 and JSP for a chain of retail discount stores. Eight years in, by 2009, I was at Wm Morrison Supermarkets, building Message-Driven Beans and BPEL processes on Oracle SOA Suite that pushed prices and promotions out of Oracle Retail Price Management, through the Oracle Retail Integration Bus, into a mainframe-backed merchandising system. We monitored it with custom HP OpenView alerts and ran reconciliation against Oracle AQ queues.

Nobody had a word for what we were doing yet. The phrase *data pipeline* hadn't escaped Silicon Valley. We called it *integration*, and it was the kind of work that ate weekends and produced incident reports written in passive voice.

I've been doing some version of that work ever since. Twenty-five years now. Different decade, different stack, same underlying job: get business-critical data out of the system that owns it, into somewhere it can be analysed — without losing rows, leaking PII, or blowing the budget.

The customers move around. **Banking** — HSBC, First Direct, M&S Bank, Allstate Insurance. **Government** — Home Office, GOV.UK, DWP, UKBA, and NHS Spine, where I was technical lead on Release 7A and built the SAML single-sign-on framework reused across every Spine module. **Retail** — Morrison's, twice (the Evolve mainframe-integration programme in 2009, then the Ocado Direct Delivery and Store Pick projects in 2016–2017). **Transport** — Smart Ticketing on Greater Manchester Metrolink with Worldline; ITSO smart cards and contactless EMV under PCI-DSS. **Automotive** — Jaguar Land Rover, also twice; the VCS event-framework migration to GCP, and now the Subscription Control Platform. **Travel** — migrating Booking.com's booking engine to AWS EKS; Thomas Cook's iTour Connect. **Betting** — Caesars Digital and William Hill (UK and US), terminal estates and back-office. **Most recently**, a financial-services client where I'm currently Senior Lead Engineer on a mainframe-to-cloud migration.

Along the way I contributed to the Java standards as a member of the JSR 255 (JMX) specification group — a long way of saying I've spent a lot of years thinking about what makes a *good interface* hold up under production load.

The platforms keep changing. Oracle SOA Suite. JBoss EAP. WebLogic. Spring. Dropwizard. AWS EKS. GCP. Kubernetes. The hard parts never do. Audit trails. Cost tracking. Error classification. Schema drift. Reconciliation. The stuff that makes a pipeline trustworthy rather than merely functional.

Around year fifteen I noticed something embarrassing: every team I joined was rebuilding the same scaffolding. Different language, different cloud, same scaffolding. By year twenty I was tired enough of writing it from scratch that I sat down and built a proper framework. I called it `gcp-pipeline-framework`. The reference repo — the one this book is a tour of — is called `gcp-pipeline-reference`.

The design language was deliberate. I had spent a decade on Spring projects and watched what Spring Framework had done for the JVM: a small framework-agnostic core, opinionated modules clipped on around it, conventions over configuration, escape hatches when you needed them. The result was an industry. `gcp-pipeline-framework` borrows that DNA without apology. There is a Spring-shaped framework hiding underneath the dbt models and the Beam jobs, and I think it deserves to be called that.

This book is the long-form version of the conversation I've been having for the last decade with senior engineers, architects, and CFOs at banks and government departments. It's the answer to the question they always end up asking: *"How do we do this in a way we can defend to a regulator and afford to operate?"*

Who I wrote this for. Three kinds of reader:

- You're newish to GCP and you want to learn how pipelines actually work in the real world. Start at Chapter 1 and go in order. Chapters 2 and 3 give you the landscape and the basics before we touch any framework code.
- You're an experienced engineer and you just want the patterns. Skip to Chapters 4 through 12. That's where the meat is.
- You're an architect trying to decide whether to adopt this thing or build your own. Read Chapters 2, 17 (the honest code review) and Appendix C (the cost model). That'll tell you most of what you need.

What this book isn't. It's not a certification cram. It's not a GUI walkthrough. I assume you've clicked around the Google Cloud Console before and you know what BigQuery is, even if you've never written a Dataflow job.

What you'll be able to do by the end. Take an unfamiliar mainframe extract and design an ingestion pipeline for it in an afternoon. Add a new entity to a running framework deployment in less than a working day. Diagnose a failed run end-to-end using only an audit trail and a run ID. Justify, with real numbers, whether your team should be paying for Cloud Composer. Read a deployment workflow and tell what it actually does. Argue convincingly with your security team about IAM, KMS, and lifecycle policies.

A note on opinions. When I tell you that audit-trail propagation matters, or that Composer is overkill below a certain scale, that's not theory. That's two decades of post-incident reviews talking. The opinions in this book are *empirically* what survives in production, calibrated across eight industries and the regulatory regimes that go with them — FCA, PCI-DSS, GOV-grade, NHS data, ITSO transport, US gaming.

A note on honesty. Chapter 17 is a code review of my own framework. I've tried to make it useful rather than flattering. The framework has real strengths and real weaknesses, and I'd rather you understood both before adopting it. If this book helps

you build something better than `gcp-pipeline-framework`, that's a win I'll happily take.

Where I could use a story instead of a bullet list, I did. Where I could cut a paragraph, I cut it. If something's still clunky, tell me and I'll fix it in the next edition.

Right. Coffee in hand? Let's go.

Joseph Aruja, Leeds, 2026

Chapter 1 — Why This Book Exists

The problem that won't go away

Mainframes were supposed to die thirty years ago. They didn't. Right now, every time you tap your bank card, there's a decent chance the transaction ends up on a COBOL program running on a box older than most of the engineers who support it.

These machines aren't going anywhere. They work. They're paid for. They run rules the business forgot it wrote. They just happen to be terrible at analytics — which is where you come in, because someone decided the data needs to be in BigQuery by Monday.

On paper, "move data from the mainframe to BigQuery" sounds easy. In practice, it's a nightmare. The mainframe sends you fixed-width files with headers and trailers. Sometimes the files are chunked into five parts because the mainframe's network can't cope with anything bigger. Sometimes they're EBCDIC-encoded. Sometimes the schema quietly changes and the person who'd have warned you is on holiday.

A pipeline that handles all of that gracefully is a thing of beauty. A pipeline that handles it badly still runs — because the business needs the data — and you personally spend the next three years on call for it. I've been that engineer. I don't want you to be that engineer.

What I actually built

`gcp-pipeline-framework` is my attempt at making "do it right" the default. It's opinionated on purpose. It gives you:

- A **foundation library** (`gcp-pipeline-core`) that doesn't depend on Beam or Airflow. Audit trails, cost tracking, structured logging, error classification, quality scoring, safe deletion, schema types — all of that lives here. You can drop it into a Dataflow job, a Cloud Function, an Airflow DAG, or a random script on your laptop.
- An **ingestion library** (`gcp-pipeline-beam`) with ready-made Apache Beam transforms for HDR/TRL parsing, split-file reassembly, schema-driven validation, and error quarantine. All the bits you were going to write yourself, already written and tested.

- An **orchestration library** (`gcp-pipeline-orchestration`) with Airflow operators, sensors, a DAG factory, and a dependency helper that knows how to wait for the right upstream jobs.
- A **transformation library** (`gcp-pipeline-transform`) with dbt macros for audit columns and PII masking.
- A **test library** (`gcp-pipeline-tester`) with base classes, fake clients, and fixtures so writing tests feels fast.
- Three **reference pipelines** that work out of the box, plus four more that show off adjacent patterns (mainframe write-back, Spanner federation, Postgres CDC, consumable marts).
- A **Terraform module set** that provisions buckets, topics, datasets, and (optionally) Composer.
- A **CI/CD suite** that tests everything on every PR, publishes libraries to PyPI, and only deploys the units that actually changed.

Everything is versioned together. Right now we're on 1.0.29. All six packages live on public PyPI. Anyone with `pip` and a GCP project can rebuild the entire repo — docs, Terraform, CI, the lot — from packages. There's a bootstrap script called `reconstruct.py` that does it in thirty seconds.

People ask me: why publish Terraform and docs to PyPI? Because a lot of teams in big companies can't reach GitHub from inside their VPC. They can reach an internal Artifactory or Nexus. If the framework is on PyPI, it's on their mirror, and "install the framework" and "get the reference project" become the same thing.

How this book is laid out

Roughly, here's the plan:

1. **Get the landscape first.** Chapter 2 explains what's out there on GCP today and why none of it quite does the job. Chapter 3 is the "zero to hero" ramp — the basics of GCS, Pub/Sub, BigQuery, Dataflow, Composer and friends, with no prior knowledge assumed.
2. **Then the architecture.** Chapter 4 is the one-page mental model: the three-unit deployment model that organises everything else. If you only read one chapter, make it that one.
3. **Then the layers, from the bottom up.** Chapters 5, 6, 7 and 8 are the four library layers in order: the framework-agnostic core, the Beam ingestion bits, the dbt transformation layer, and the Airflow orchestration.
4. **Then the production reality.** Chapters 9 through 12 cover Terraform, observability and FinOps, testing, and CI/CD. This is the unglamorous stuff that turns a demo into something your CFO will fund.
5. **Then the tour.** Chapter 16 walks through all seven reference deployments.
6. **Then the truth.** Chapter 17 is an honest code review of my own framework. I try to be more useful than flattering. Chapter 18 is how you get started today.

By the end you'll have a mental model for building production pipelines on GCP you can actually trust — plus enough of a critical eye to tell when a framework (including mine) isn't the right answer.

A few ground rules

Before we go any further:

- I'll say **"you"** when I mean you, and **"we"** when I mean the team you and I would be on together. Feels more natural that way.
- **The code samples are real.** Where I've simplified something for clarity I'll say so. Everything in this book ships in the repo.
- **British spellings** (colour, behaviour, organisation). Sorry, not sorry.
- **I care about cost.** A lot. The single biggest mistake I see is teams building a pipeline that works and then realising it costs twelve thousand dollars a month to run. I'll flag costs whenever they show up. I'd rather you raise an eyebrow now than later.

Right. Let's go.

Key takeaways

- The hard problem of mainframe-to-cloud is not moving bytes — it is making the movement auditable, recoverable, and cheap enough to operate for years.
- Every team that has ever done this has written the same scaffolding from scratch. `gcp-pipeline-framework` is that scaffolding, published so you do not have to.
- The framework is six PyPI packages, three reference pipelines, a Terraform module set, and a CI/CD suite — all versioned together, reconstructable from `pip` alone.
- Cost is a first-class concern throughout this book. If a choice saves engineering time but doubles the monthly bill, that is not a win.

Chapter 2 — The State of the GCP Data Pipeline Landscape (and the Gap This Book Fills)

Google Cloud has the services. It does not have the answers.

If you walked into a room full of Google Cloud product managers in 2026 and asked "do you have everything I need to build a data pipeline?", they would, with complete sincerity, say yes. They would point at Cloud Storage for landing files. They would point at Pub/Sub for events. They would point at Dataflow for distributed processing. They would point at BigQuery for the warehouse, Cloud Composer for orchestration, Dataform and dbt-on-BigQuery for transformations, Datastream for change data capture, Data Fusion for low-code ETL, Dataplex for governance, and a half-dozen more.

They are not lying. The services exist. They are first-rate, they autoscale, they integrate, and they bill you per second.

What they will not tell you, because it is not their job to tell you, is that none of those services *together* form an opinionated framework. They are Lego bricks. There is no instruction sheet for "build a regulator-friendly mainframe-to-BigQuery pipeline that survives the next on-call rotation". You get to design the join between Pub/Sub and Dataflow yourself. You get to invent your own audit-trail format. You get to write the retry logic, the schema validator, the cost tracker, the data-quality scorer, the deletion workflow, the lineage propagation, the reconciliation report, the alert dispatcher, and the seventeen lines of bash that turn a Python source tree into a Dataflow Flex Template.

You will, in short, write a framework. Every team I have worked with has written some version of this framework. They have all written different versions of it. None of those versions has been good.

What the open ecosystem actually offers

It is worth taking stock of the open-source landscape, because the picture is not flattering.

There are **runtime libraries** — Apache Beam, Apache Airflow, dbt, Datastream — and they are excellent at the thing they do, in isolation. There are **vendor SDKs** — `google-cloud-bigquery`, `google-cloud-storage` — and they are competent client libraries. There are **wrapper toolkits** that bundle a few of the above with conventions: Mage, Prefect, Dagster, Kedro, ZenML. Most of these are general-purpose orchestrators with the data-engineering ecosystem in mind, not opinionated GCP frameworks.

There are **enterprise reference architectures** from Google itself — beautifully diagrammed, freely downloadable, almost completely untestable. They give you a picture, not a codebase. The picture is not enough.

There are **internal frameworks** at every large GCP-using company on the planet. None of them are open. They are written for one organisation, by people who have moved on, in idioms nobody outside that organisation recognises.

There are, in short, no production-grade, open, GCP-specific, mainframe-aware, opinionated data-pipeline frameworks worth adopting. There are gaps everywhere.

The eight gaps

If you make a list of the things every serious GCP pipeline team rebuilds from scratch, it looks something like this:

- **A mainframe-aware ingestion library.** Apache Beam does not understand HDR/TRL. Nobody's open library does either.
- **A schema-driven validator.** You either invent one or you write `if pd.isna(x):` ten thousand times.
- **A cost-tracking layer.** GCP billing tells you what last month cost. It does not tell you which entity, which run, which BigQuery query. You build that yourself or you fly blind.
- **A run-correlated audit trail.** Logs, metrics, audit events, dbt invocations, Dataflow jobs, Pub/Sub messages — they all have their own identifiers. Correlating them takes a deliberate identity (`run_id`) threaded through everything. No off-the-shelf library does this.
- **An error-classification taxonomy.** Validation, integration, and resource errors require different handling. Generic retry libraries treat them the same.
- **A data-deletion workflow.** "DELETE FROM" is illegal in most regulated industries. You need a workflow with approvers, holds, tombstones, and recoverable archives. Nobody ships this.
- **A reconciliation engine.** Did the envelope count match the ingested count match the BigQuery row count? Until you know, you are guessing.
- **A coordination layer for JOIN preconditions.** "Run the transform when both sources are loaded" sounds simple. In practice every team writes a slightly broken version.

This is the gap. It is not a missing service from Google; it is a missing *opinion* about how the existing services should be assembled.

Why nobody has filled the gap

A reasonable question: if the gap is so obvious, why has the open ecosystem not closed it?

Three reasons, in my experience.

First, **the gap is mainframe-shaped**, and mainframe migrations are not glamorous. Open-source contributors prefer streaming Kafka pipelines into Snowflake to fixed-width files with EBCDIC encoding from Z/OS. The need is enormous; the contributor base is thin.

Second, **enterprise frameworks are written under NDA**. The teams who do solve this problem solve it on company time, with company data, behind company firewalls. Their code does not get extracted, generalised, and published.

Third, **the right framework is opinionated**, and opinions are unfashionable in the cloud-native world. The dominant style is "minimal toolkit, infinite extensibility". A framework that says "thou shalt use HDR/TRL, thou shalt have a `run_id`, thou shalt route bad records to a quarantine bucket" feels prescriptive in a way modern open source tries to avoid. But pipelines need that prescription. Without it, you do not have a pipeline; you have a kit of parts.

The framework as the answer

`gcp-pipeline-framework` is an answer. Not the answer; an answer. It tries to fill the eight gaps in 2.3 with concrete, opinionated, testable code, and it does so with the conventions of the rest of the GCP ecosystem (Beam, Airflow, dbt, Terraform) rather than against them.

If you have worked in the JVM world, the closest cultural reference point is **Spring Framework**. Spring did not invent dependency injection; it took a set of patterns that were already loose in the community and gave them a name, an opinionated default, a thin core that the rest of the platform clipped onto, and a culture of *convention over configuration*. That combination — small core, plug-in modules, sensible defaults, an escape hatch when you need it — turned a kit of parts into an industry. `gcp-pipeline-framework` is built on the same DNA. `gcp-pipeline-core` is the framework-agnostic kernel; the Beam, Airflow, dbt, and tester libraries clip on around it; the conventions (`EntitySchema`, `run_id`, three-unit deployment) are the equivalent of Spring's component scan and stereotype annotations. Skip this analogy if it does not land for you. Keep it if you ever shipped Spring Boot to production and wished someone had done the same thinking for data pipelines.

It is opinionated where opinions matter:

- HDR/TRL is the assumed envelope. You can opt out, but the default is mainframe-shaped.
- `EntitySchema` is the source of truth. Validation, table creation, masking, fixtures — all read from it.
- `run_id` propagates everywhere. There is no log line, metric, audit event, dbt invocation, or cost record without one.

- Errors are classified. Validation does not retry. Integration does. Resource fails loudly.
- Cost is a metric. It lives next to throughput and latency, not in a separate billing dashboard.
- Deletion is a workflow. Nothing leaves the platform without an approval chain.
- Reconciliation is mandatory. A run is not green until envelope, valid, invalid, and BQ all agree.
- JOIN preconditions are explicit. The orchestrator waits, deterministically, for the right preconditions.

It is also intentionally unopinionated where flexibility matters: you can run on Composer or substitute Cloud Functions, you can use BigQuery on-demand or flat-rate, you can embrace OpenTelemetry or ignore it, you can publish to public PyPI or to your internal Nexus.

What you get from this book

If you are reading this book, you are most likely in one of three situations:

- You are about to build a pipeline like this and want to avoid the rake-step the rest of us all took. The next chapter exists for you: it covers the GCP fundamentals every pipeline depends on, with no prior assumption.
- You have built a pipeline like this and want to compare notes. The middle chapters of this book are for you.
- You are evaluating whether to adopt this framework rather than build your own. The honest code review in Chapter 17 and the cost model in Appendix C are for you.

Whichever you are, the next chapter takes the GCP data services from zero. If you are already familiar, skim it; if you are not, it will save you weeks.

Key takeaways

- GCP has all the bricks — Cloud Storage, Pub/Sub, Dataflow, BigQuery, Composer — but no instruction sheet for assembling them into a regulator-friendly pipeline. Every team writes that instruction sheet themselves.
- The open-source ecosystem has runtime libraries and vendor SDKs, but nothing production-grade, GCP-specific, and mainframe-aware. The gap is structural, not temporary.
- The eight things every serious pipeline team rebuilds from scratch — mainframe-aware ingestion, schema validation, cost tracking, run-correlated audit, error taxonomy, deletion workflow, reconciliation, JOIN preconditions — are exactly what the framework fills.
- Opinions are not a design smell. A framework that says nothing is a kit of parts. The right framework is prescriptive where it matters and flexible where it does not.

Chapter 3 — Zero to Hero: GCP Pipeline Fundamentals

This chapter is the on-ramp. If you have never built a data pipeline on Google Cloud, read it carefully. If you have, skim it for the conventions the rest of the book assumes.

The mental model

A data pipeline on GCP, at the most reductive level, is a four-stage object:

```
[ Source ] → [ Land ] → [ Process ] → [ Serve ]
```

- **Source** is wherever the data is born — a mainframe extract, a Postgres database, a Kafka topic, a third-party SaaS export.
- **Land** is where it first arrives in the platform — almost always Cloud Storage.
- **Process** is the work of validating, transforming, joining, and shaping it — Dataflow, dbt, Cloud Functions, Cloud Run, BigQuery scheduled queries.
- **Serve** is wherever consumers reach in — BigQuery for analysts, Looker for dashboards, Pub/Sub for downstream systems, BigTable for low-latency lookups.

Every service we will discuss occupies one of those boxes. Once you have the boxes in mind, the services stop feeling like a confusing menu and start feeling like a small set of choices per box.

The core services in one paragraph each

Cloud Storage (GCS). Object storage. Buckets contain objects; objects have keys; everything is HTTP under the hood. Storage classes (Standard, Nearline, Coldline, Archive) trade cost against retrieval latency. Lifecycle rules move objects between classes automatically. Notifications let GCS publish to Pub/Sub when an object lands. The unit of work is the object, not the file system.

Pub/Sub. Managed publish/subscribe messaging. Publishers push messages to a topic; subscribers pull (or get pushed) from a subscription. At-least-once delivery by default; exactly-once is available with a configuration flip. Acknowledgements are required, retries are automatic, dead-letter routing is built in. Pub/Sub is the GCP equivalent of Kafka for most use cases — simpler, less tunable, and almost always good enough.

Dataflow. Managed runner for Apache Beam pipelines. You write a Beam program in Python or Java; Dataflow takes it, autoscales the worker pool, and runs it as either a batch or a streaming job. Beam's programming model is reduce/map/group with side inputs and side outputs; it parallelises naturally; it handles late-arriving data with watermarks. Dataflow Flex Templates package a Beam pipeline as a Docker image you can launch with a parameter set.

BigQuery. A serverless analytics warehouse. You create datasets; datasets contain tables; tables can be partitioned and clustered. Storage is columnar and compressed. Queries scan data and bill on bytes scanned (on-demand) or slot-seconds (flat rate). BigQuery is unique among warehouses in that there is no cluster to manage; you cannot oversize or undersize it.

Cloud Composer. Managed Apache Airflow on GKE. You write DAGs in Python; Composer schedules, runs, monitors, and retries them. Cloud Composer 2 starts at about 300 USD per month before you schedule a single task. Use it when you genuinely need Airflow; use Cloud Functions or Cloud Run Jobs when you do not.

Cloud Functions. Single-purpose serverless functions. Trigger on HTTP, Pub/Sub, or storage events. Charge per invocation and per hundred milliseconds of execution. Excellent for the small glue tasks (Pub/Sub trigger handlers, FDP completion notifiers) that do not need a scheduler.

Cloud Run. Serverless containers. Stateless web services or scheduled jobs. The cheapest place on GCP to run a small dbt model on a daily schedule.

Dataform. Google's native take on dbt-style SQL transformation, integrated with BigQuery. dbt-bigquery is the more popular choice in 2026; Dataform is improving and worth watching.

Datastream. Managed change-data-capture from MySQL, Postgres, Oracle, AlloyDB, and SQL Server into Cloud Storage or BigQuery. The streaming on-ramp for "I have an OLTP database I want analytics on".

Cloud Logging. Structured log ingestion, indexing, and querying. Every other service writes here by default. Filters use a YAML-flavoured query language; sinks let you export to BigQuery for long-term retention.

Cloud Monitoring. Metrics, dashboards, alert policies. Custom metrics are first-class. Service-level objective tracking is built in.

Cloud KMS. Customer-managed encryption keys. Used for storage, Pub/Sub, BigQuery, and Composer encryption when you cannot rely on Google-managed keys.

Workload Identity Federation. Keyless authentication from external systems (GitHub Actions, AWS) to GCP service accounts. The right way to authenticate from CI in 2026.

That is, modulo a few specialised services, the entire substrate this book uses.

The simplest possible GCP pipeline, in 30 lines

To make all of the above concrete, here is the smallest pipeline that does something useful: read a CSV from GCS, count the rows, write the count to BigQuery.

```
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions

class CountToRow(beam.DoFn):
    def process(self, element):
        yield {"file_uri": "gs://example/in.csv", "row_count": element}

def run():
    options = PipelineOptions(
        runner="DataflowRunner",
        project="my-project",
        region="europe-west2",
        temp_location="gs://example/tmp",
        staging_location="gs://example/staging",
    )

    with beam.Pipeline(options=options) as p:
        (
            p
            | "Read"      >> beam.io.ReadFromText("gs://example/in.csv",
            ↪  skip_header_lines=1)
            | "ToOnes"    >> beam.Map(lambda _: 1)
            | "Sum"       >> beam.CombineGlobally(sum)
            | "ToRow"     >> beam.ParDo(CountToRow())
            | "WriteBQ"   >> beam.io.WriteToBigQuery(
                "my-project:example.row_counts",
                schema="file_uri:STRING,row_count:INTEGER",
                write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND,
                create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED,
            )
        )

if __name__ == "__main__":
    run()
```

That is a complete, runnable pipeline. It is also a useless one in production: there is no schema validation, no error handling, no audit trail, no cost tracking, no reconciliation, no `run_id`. Comparing this 30-line example to the four-line user-facing API in `gcp-pipeline-beam` should make the value of the framework visible immediately.

The simplest possible orchestration, in 20 lines

A trivially small Airflow DAG that runs the above:

```
from airflow import DAG
from airflow.providers.google.cloud.operators.dataflow import
↪  DataflowCreatePythonJobOperator
```

```
from datetime import datetime

with DAG(
    dag_id="example_count",
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
) as dag:
    DataflowCreatePythonJobOperator(
        task_id="count_rows",
        py_file="gs://example/code/count.py",
        job_name="example-count-{{ ds_nodash }}",
        location="europe-west2",
        options={"runner": "DataflowRunner"},
    )
```

Again: complete, runnable, and missing every important production property. No retries policy. No SLA. No alerting. No dependency on upstream landing. No audit. No idempotence. The framework's `data-pipeline-orchestrator` deployment provides all of those by composition.

The simplest possible transformation, in 10 lines

A minimal dbt model:

```
-- models/example/row_counts_daily.sql
{{ config(materialized='table') }}

SELECT
    DATE(load_ts) AS load_date,
    SUM(row_count) AS total_rows
FROM {{ source('example', 'row_counts') }}
GROUP BY 1
```

Useful, simple, and missing audit columns, PII masking, incremental materialisation, and tests. The framework's `bigquery-to-mapped-product` shows the production-shaped equivalent.

The simplest possible Terraform, in 20 lines

A minimal infrastructure file:

```
provider "google" {
  project = "my-project"
  region  = "europe-west2"
}

resource "google_storage_bucket" "landing" {
  name          = "example-landing"
  location      = "EUROPE-WEST2"
  storage_class = "STANDARD"
```

```
  force_destroy = false
  uniform_bucket_level_access = true
}

resource "google_pubsub_topic" "file_notifications" {
  name = "example-file-notifications"
}

resource "google_storage_notification" "trigger" {
  bucket = google_storage_bucket.landing.name
  topic  = google_pubsub_topic.file_notifications.id
  payload_format = "JSON_API_V1"
  event_types = ["OBJECT_FINALIZE"]
}
```

Production Terraform for a real pipeline is fifty times this size — which is the point of the framework's modules.

The hidden complexity of the simple version

If you ran the four snippets above in sequence, you would have a "pipeline" that:

- Has no encryption at rest.
- Has no IAM segregation.
- Has no dead-letter handling.
- Has no observability beyond stdout.
- Has no testing harness.
- Has no concept of who triggered what when.
- Costs roughly the same as the production version because it leaves Pub/Sub and Composer running.

Every line of the framework exists to close one of those gaps. As we walk through the architecture in the rest of the book, recognise that what looks like complexity is, in almost every case, *load-bearing* complexity — there because the simple version was wrong in production.

Glossary refresher

A few terms you will see throughout the book, defined once here:

- **ODP** — Original Data Product. The untransformed BigQuery layer that mirrors source extracts.
- **FDP** — Foundation Data Product. The clean, business-shaped layer.
- **CDP** — Consumable Data Product. Narrow, contracted views derived from FDP.
- **JOIN pattern** — A transformation that combines multiple ODP sources into one FDP table.
- **MAP pattern** — A transformation that maps one ODP source to one FDP table.
- **Run ID** — A unique identifier per pipeline execution, threaded through every artefact.

- **HDR/TRL** — Header/Trailer envelope on a mainframe extract file.
- **Quarantine** — The four-stage workflow (REVIEW, HOLD, DELETE, ARCHIVE) for rejects and intentional deletions.
- **Reconciliation** — The check that envelope, valid count, invalid count, and BQ row count agree.

What zero to hero looks like

By the end of this book you will be able to:

- Take an unfamiliar mainframe extract and design an ingestion pipeline for it in an afternoon.
- Add a new entity to a running framework deployment in less than a working day.
- Diagnose a failed run end-to-end using only the audit trail and the run ID.
- Justify, with numbers, the choice between Composer and Cloud Functions for orchestration.
- Read a deploy workflow and understand what it actually does.
- Argue convincingly with your security team about IAM, KMS, and lifecycle policies.

That is hero. The rest of the book is the path from here to there.

Key takeaways

- Every GCP pipeline is a four-stage object: Source → Land → Process → Serve. Every service in this book occupies one of those four boxes.
- The 30-line pipeline example is complete and runnable. It is also missing reconciliation, audit, cost tracking, PII masking, retries, and tests. Each of those gaps is load-bearing in production.
- `ODP`, `FDP`, `CDP`, `run_id`, `HDR/TRL`, and `quarantine` are framework vocabulary used throughout the book; define them once here so they do not need explaining later.
- Production Terraform for a real pipeline is fifty times the size of the minimal snippet shown here. The framework's modules are what make that manageable.

Chapter 4 — The 10,000-Foot View: The Three-Unit Deployment Model

One big pipeline is the wrong mental model

The first thing most teams do when they get the brief “move our mainframe data to BigQuery” is draw a diagram. At the left is a mainframe. At the right is BigQuery. In the middle is a single rectangle labelled `data-pipeline` with an arrow going left-to-right. Sometimes, for ambition, there is a second arrow going right-to-left.

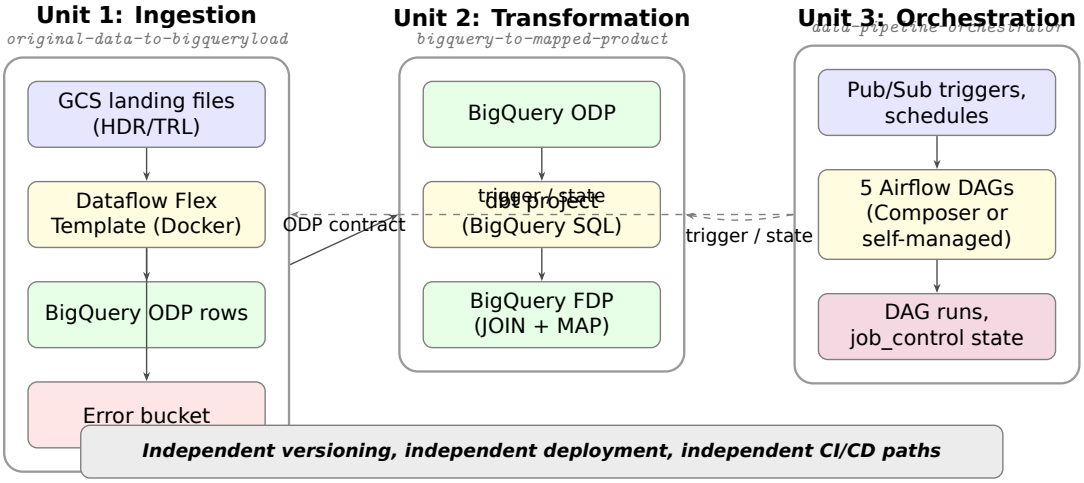

Figure 1: The three-unit deployment model.

This is the wrong picture. It is wrong because it suggests the pipeline is one thing, built and deployed as a unit. In reality, a production data pipeline has at least three distinct concerns, each with a different lifecycle, a different owner, and a different failure mode:

- **Ingestion**: getting the data into the platform, in a form the platform can reason about.
- **Transformation**: reshaping the data into something useful to the business.

- **Orchestration**: deciding when things run, in what order, and what happens when something goes wrong.

Lumping these together produces a pipeline where a typo in a SQL model causes your ingestion job to fail to deploy, where the orchestration DAG has to be updated every time a new table is added, and where nobody can tell whether a delayed dashboard is the fault of the mainframe, the parser, the transform, or the scheduler.

`gcp-pipeline-framework` calls this the **three-unit deployment model**. It is the single most important architectural choice in the codebase, and once you have seen it, you will not want to build a pipeline any other way.

Unit 1: Ingestion (`original-data-to-bigqueryload`)

Ingestion is the first unit. Its job is to read files from a GCS landing bucket, validate them, and load the records into an "Original Data Product" (ODP) dataset in BigQuery — one table per logical entity, untransformed, audit-columned, ready for reference.

The reference implementation ships an ingestion unit called `original-data-to-bigqueryload`. It is a Dataflow Flex Template — meaning the pipeline is packaged as a Docker image, registered with Dataflow, and launched on demand with a parameter set. Four entities (customers, accounts, decision, applications) are supported out of the box.

A few design principles make this unit work:

- **One entity per run.** Each Dataflow job handles one logical entity's file set. This keeps the code simple, the tests focused, and the failure domain tight. An ingestion job for `customers` cannot break `accounts`.
- **Schema-driven, not code-driven.** The structure of each entity is expressed as an `EntitySchema` — a plain Python dataclass with one `SchemaField` per column. Validation, BigQuery table creation, and PII detection all read from the same schema. Change the schema, change the behaviour.
- **HDR/TRL aware.** Mainframe files almost always come with a header record and a trailer record. The framework parses both, validates the record count, and refuses to load a file whose trailer says 500,000 rows when the body has 499,998.
- **Split-file reassembly.** If the mainframe sends `customers.part01.csv`, `customers.part02.csv`, …, the framework joins them transparently and treats them as one logical file.
- **Error quarantine.** Bad records do not fail the job. They are routed to an error bucket with enough metadata to replay them later. The Beam side output pattern makes this natural.

The result is that ingestion is a black box with a narrow contract: files in, ODP rows out, errors isolated. You could replace the implementation with something hand-rolled in Go tomorrow, and the rest of the pipeline would not notice.

Unit 2: Transformation (`bigquery-to-mapped-product`)

Transformation is the second unit. Its job is to read from the ODP and produce the "Foundation Data Product" (FDP) — the clean, business-shaped, analytics-ready layer. It is implemented as a dbt project called `bigquery-to-mapped-product`.

Two transformation patterns are demonstrated in parallel, and this is one of the most pedagogically useful decisions in the codebase:

- **The JOIN pattern.** Some FDP tables need data from multiple ODP tables before they can be built. `event_transaction_excess`, for example, joins `customers` and `accounts`. The pipeline must wait for *all* constituent ingestion jobs to finish before the transform fires.
- **The MAP pattern.** Other FDP tables map one-to-one from a single ODP table. `portfolio_account_excess` comes straight from `decision`; `portfolio_account_facility` comes straight from `applications`. These can transform the instant their source lands — no waiting required.

Most real pipelines need both patterns, which is exactly why the reference implementation uses both. The transformation unit itself is agnostic about which pattern applies; it is the orchestration layer that knows how to wait for a JOIN's preconditions and trigger a MAP immediately.

Inside the dbt project the layering is the standard staging → mart progression, with a thin analytics veneer for ad-hoc BI work:

```
models/
├── staging/        # 1:1 with ODP, light cleanup
├── fdp/            # JOIN and MAP models — the business layer
├── marts/          # Domain-specific mart views
└── analytics/      # Ad-hoc views and aggregations
```

Two framework-specific dbt macros — `generate_audit_columns()` and `apply_pii_masking()` — are applied almost everywhere. They ensure every row carries lineage metadata (`run_id`, `source_system`, `load_timestamp`) and that sensitive fields are masked at the FDP boundary rather than at report time. Getting this right once is worth more than you might think.

Unit 3: Orchestration (`data-pipeline-orchestrator`)

Orchestration is the third unit. Its job is to know *when* to run each of the first two units, in what order, and what to do when something goes wrong. The reference implementation uses Apache Airflow on Cloud Composer, and ships five DAGs out of the box:

- `pubsub_trigger` — listens for a `.ok` file on a Pub/Sub topic and emits a "run ingestion" event.
- `ingestion` — launches the Dataflow Flex Template for the appropriate entity.
- `transformation` — runs dbt models once preconditions are met.
- `error_handling` — manages the error quarantine lifecycle (reviewed, held, released, archived).

- `status` — writes pipeline state into a `job_control` dataset for downstream dashboards.

The heart of this unit is a file called `generate_dags.py` — an 87 KB DAG factory that reads a YAML configuration and generates the right DAGs for the right entities. If you have ever hand-maintained fifty DAGs and cried about it, this will feel like a revelation.

Airflow is expensive, though. A Cloud Composer 2 environment starts at around three hundred US dollars a month before you have scheduled a single task. For that reason, the reference implementation explicitly treats Composer as **opt-in**: the Terraform module is present, but the default `terraform apply` does not provision it. Smaller teams can substitute a Cloud Functions trigger for the `pubsub_trigger` DAG and run the rest as scheduled Cloud Run jobs for a fraction of the cost.

I will return to this cost-awareness theme repeatedly through the book. It is the single biggest differentiator between a reference implementation that teams actually adopt and one that teams quietly delete when the first bill comes in.

Data flow, end to end

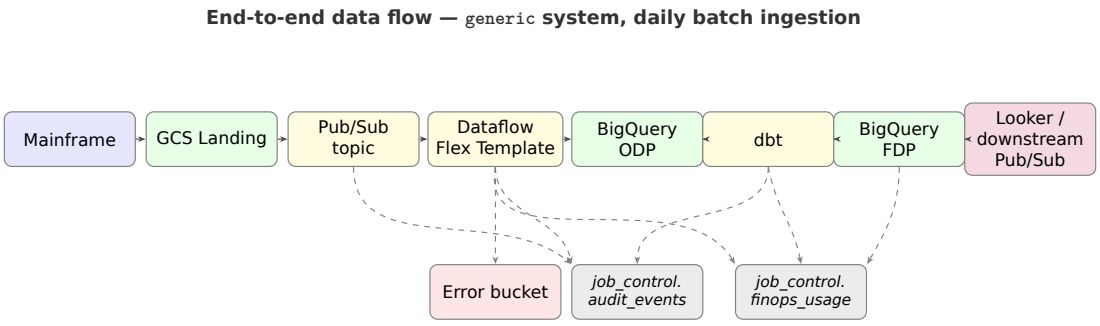

Figure 2: End-to-end data flow — `generic` system, daily batch ingestion.

Putting the three units together, a successful run looks like this:

1. The mainframe drops `customers.csv` and `customers.ok` into the GCS landing bucket. The `.ok` file is the mainframe saying "I have finished writing — you may read now".
2. GCS publishes a notification to the `generic-file-notifications` Pub/Sub topic.
3. The `pubsub_trigger` DAG, watching that topic through a pull sensor, receives the notification and kicks off an `ingestion` DAG run for `customers`.
4. The `ingestion` DAG launches a Dataflow job from the `original-data-to-bigqueryload` Flex Template with `entity=customers`.
5. Dataflow reads the CSV, parses HDR/TRL, validates records against `Customer-Schema`, and writes valid rows to `odp_generic.customers` while routing rejects to the error bucket.
6. On completion, the ingestion DAG updates a row in `job_control.pipeline_runs` with the outcome.

7. The `transformation` DAG notices the new ODP row (via `EntityDependencyChecker`). For MAP entities it fires immediately; for JOIN entities it waits for the full set.
8. dbt runs. FDP tables are rebuilt with audit columns and PII masking. Marts and analytics views are refreshed.
9. The `status` DAG writes a consolidated run record.
10. Dashboards, downstream systems, and the FDP trigger hear about the new data via Pub/Sub.

Every step is observable. Every step is cost-attributed. Every step is testable in isolation. Every step can be replayed from checkpoint. That is the whole point.

Why this model scales

The three-unit model scales for three reasons:

- **Unit isolation.** Teams can own different units without stepping on each other. An SQL analyst can modify the transformation layer without touching the Beam code. A platform engineer can upgrade the orchestration layer without risking the transformation logic.
- **Lifecycle independence.** Ingestion changes when mainframe extracts change. Transformation changes when business logic changes. Orchestration changes when operational policy changes. Each has its own release cadence and its own CI pipeline path filter.
- **Reuse by substitution.** Because the contract between units is narrow (ODP table, FDP table, Pub/Sub event), you can replace any single unit with an alternative without rewriting the other two. Dataflow not cost-effective for a tiny entity? Ship the ingestion in a Cloud Function instead; the transformation and orchestration do not care.

Chapter 5 shows you the same pipeline built with and without the framework — so the rest of the book is concrete, not abstract. Chapter 6 onwards then takes each library layer in turn. We begin at the foundation — the part of the framework that knows nothing about Beam, nothing about Airflow, and therefore everything about good design.

Key takeaways

- The pipeline is not one thing — it is three independently-versioned units: ingestion, transformation, orchestration.
- Each unit has its own lifecycle, its own CI path, its own owner. Mixing them is the source of most multi-team pain.
- The contracts between units are narrow: ODP table, FDP table, Pub/Sub event. Anything you can replace via the contract is replaceable without touching the other two units.
- `run_id` threads the three units together. Without that thread, you cannot trace a failure end-to-end.

Chapter 5 — Without the Library vs With It: A Side-by-Side Comparison

Before we dive into the library internals, let me show you what the framework is really worth. I'll build the same tiny pipeline twice. Once from scratch, the way most teams start. Once using `gcp-pipeline-framework`. Same business outcome, wildly different amount of code — and, more importantly, wildly different production-readiness.

This is the chapter I wish someone had written for me five years ago.

The scenario

A mainframe drops `customers.csv` (with `HDR`/`TRL` envelope) into a GCS landing bucket every morning. We need to:

- Parse the file.
- Validate the rows against a schema.
- Load the valid rows into `odp_generic.customers` in BigQuery.
- Quarantine invalid rows in an error bucket.
- Reconcile the envelope count to the loaded count.
- Emit an audit record.
- Track cost.
- Alert if anything goes wrong.

That's one entity. One job. Let's see what it looks like both ways.

Version A — without the library

Step 1: the Beam pipeline

```
# ingest_customers.py  — from scratch, no framework
import apache_beam as beam
import csv
import json
import logging
import uuid
from datetime import datetime, timezone
```

```
from google.cloud import bigquery, storage

RUN_ID = f"{datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%SZ')}-{uuid.uuid4().
↪  hex[:4]}"
logging.basicConfig(level=logging.INFO, format='%(asctime)s %(levelname)s
↪  %(message)s')
log = logging.getLogger("ingest")

SCHEMA = {
    "customer_id":   {"type": "STRING",    "required": True,  "regex": r"^\d{10}$"},
    "full_name":     {"type": "STRING",    "required": True,  "pii": True},
    "date_of_birth": {"type": "DATE",      "required": True,  "pii": True,
                      "formats": ["%Y-%m-%d", "%Y%m%d"]},
    "postcode":      {"type": "STRING",    "required": False, "pii": True,
                      "regex": r"^[A-Z]{1,2}[0-9R][0-9A-Z]? ?[0-9][A-Z]{2}$"},
    "opened_at":     {"type": "TIMESTAMP", "required": True},
}

class ParseHDRTRL(beam.DoFn):
    """Peel HDR and TRL off a CSV. Stash expected_count on the first pass."""
    def __init__(self):
        self.state = None  # cannot share state across workers — doesn't work in
        ↪  practice

    def process(self, line):
        if line.startswith("HDR|"):
            parts = line.split("|")
            log.info(f"HDR seen: entity={parts[1]} expected={parts[4]}")
            return                                                  # drop HDR
        if line.startswith("TRL|"):
            log.info(f"TRL seen: {line}")
            return                                                  # drop TRL
        yield line

class ParseCsv(beam.DoFn):
    def process(self, line):
        try:
            reader = csv.reader([line], delimiter="|")
            cells = next(reader)
            record = {
                "customer_id":   cells[0].strip(),
                "full_name":     cells[1].strip(),
                "date_of_birth": cells[2].strip(),
                "postcode":      cells[3].strip(),
                "opened_at":     cells[4].strip(),
            }
            yield record
        except Exception as e:
            yield beam.pvalue.TaggedOutput("invalid",
                {"reason": f"parse-failed: {e}", "raw": line})

class Validate(beam.DoFn):
    def process(self, record):
```

```
        errors = []
        for field, rules in SCHEMA.items():
            value = record.get(field)
            if rules.get("required") and not value:
                errors.append(f"{field} is required")
                continue
            if value and rules.get("regex"):
                import re
                if not re.match(rules["regex"], value):
                    errors.append(f"{field} does not match pattern")
        if errors:
            yield beam.pvalue.TaggedOutput("invalid",
                {"record": record, "errors": errors})
        else:
            yield record

def run(input_file, bq_table, error_bucket):
    options = beam.options.pipeline_options.PipelineOptions(
        runner="DataflowRunner",
        project="my-project",
        region="europe-west2",
        temp_location="gs://my-bucket/tmp",
        staging_location="gs://my-bucket/staging",
        job_name=f"ingest-customers-{RUN_ID}",
    )
    with beam.Pipeline(options=options) as p:
        valid, invalid = (
            p
            | "Read"      >> beam.io.ReadFromText(input_file)
            | "HDRTRL"    >> beam.ParDo(ParseHDRTRL())
            | "Parse"     >> beam.ParDo(ParseCsv()).with_outputs("invalid",
            ↪  main="valid")
        )
        valid2, invalid2 = (
            valid
            | "Validate"  >> beam.ParDo(Validate()).with_outputs("invalid",
            ↪  main="valid")
        )

        valid2 | "WriteBQ" >> beam.io.WriteToBigQuery(
            bq_table,
            schema={"fields": [
                {"name": k, "type": v["type"], "mode": "REQUIRED" if
↪   v.get("required") else "NULLABLE"}
                for k, v in SCHEMA.items()
            ]},
            write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND,
            create_disposition=beam.io.BigQueryDisposition.CREATE_IF_NEEDED,
        )

        all_invalid = (invalid, invalid2) | "MergeInvalid" >> beam.Flatten()
        all_invalid | "WriteInvalid" >> beam.io.WriteToText(
            f"gs://{error_bucket}/customers/{RUN_ID}/invalid",
```

```
            file_name_suffix=".jsonl",
        )

if __name__ == "__main__":
    import sys
    run(sys.argv[1], sys.argv[2], sys.argv[3])
```

Just over 100 lines. Looks complete, right? Let me count what's **still missing**:

- **Reconciliation.** Nothing asserts that `valid_count + invalid_count == TRL count`. If Dataflow silently drops 200 rows, nobody notices.
- **Audit trail.** No `run_id` is written anywhere persistent. When an auditor asks "did this run complete?" the only evidence is Dataflow's job history — which GCP retains for 30 days.
- **Cost tracking.** No record of bytes billed by BigQuery or objects read from GCS.
- **PII masking.** The schema flags `full_name` as PII but nothing masks it.
- **Structured logging.** Log lines are plain text; Cloud Logging can't index them by `run_id`.
- **Error classification.** An OOM on a Dataflow worker is treated the same as a malformed row.
- **Retries.** A transient BigQuery 500 fails the whole pipeline.
- **Worker setup optimisation.** Schema validation imports `re` per element rather than per worker.
- **Tests.** None.
- **Idempotence.** Rerunning the same file double-writes.

Step 2: the Airflow DAG (still no framework)

```
# dags/ingest_customers.py  — from scratch
from datetime import datetime, timedelta
from airflow import DAG
from airflow.providers.google.cloud.operators.dataflow import
↪  DataflowCreatePythonJobOperator

default_args = {
    "owner": "data-eng",
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
    "email_on_failure": True,
    "email": ["oncall@example.com"],
}

with DAG(
    dag_id="ingest_customers",
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
    default_args=default_args,
    tags=["ingestion", "customers"],
) as dag:
    DataflowCreatePythonJobOperator(
```

```
        task_id="run_dataflow",
        py_file="gs://my-bucket/code/ingest_customers.py",
        job_name="ingest-customers-{{ ds_nodash }}",
        location="europe-west2",
        options={
            "input_file": "gs://my-landing/customers-{{ ds_nodash }}.csv",
            "bq_table":    "my-project:odp_generic.customers",
            "error_bucket":"my-error-bucket",
        },
    )
```

Short, but:

- **No Pub/Sub trigger.** This runs at a fixed time, not when the file lands. Late file = empty run.
- **No dependency on upstream entities.** If this were a JOIN source, nothing waits for the pair.
- **No SLA alerting beyond `email_on_failure`.** A late but successful run produces nothing.
- **No structured status writing.** Downstream DAGs can't tell what happened.
- **No transformation step.** dbt is a separate DAG you haven't written yet.

Step 3: the dbt model (still no framework)

```
-- models/fdp/portfolio_account_excess.sql
{{ config(materialized='incremental', unique_key='account_id') }}

SELECT
    account_id,
    decision_type,
    decision_date,
    amount,
    CURRENT_TIMESTAMP() AS loaded_at
FROM {{ source('odp_generic', 'decision') }}
{% if is_incremental() %}
WHERE loaded_at > (SELECT MAX(loaded_at) FROM {{ this }})
{% endif %}
```

Works. Missing: run_id propagation, PII masking, data-quality tests, reconciliation hook.

Step 4: the Terraform

I'll save you the hundred lines. You'll need: landing bucket, error bucket, archive bucket, Pub/Sub topic + subscription, GCS notification, BigQuery datasets, service accounts, IAM bindings, lifecycle rules, labels. Each one is a resource you write, test, and maintain.

Step 5: the reality at month three

Here's what happens to the "from scratch" pipeline between month one and month three in production:

- Someone adds a second entity. You copy-paste `ingest_customers.py` to `ingest_accounts.py`. Duplicated logic, separate bugs to fix.
- The mainframe starts splitting files. You bolt on split-file handling. It doesn't work the first three times.
- A row is wrong, nobody can explain why. There's no audit trail, no reconciliation record. An engineer spends two days replaying jobs to investigate.
- The BigQuery bill doubles. Nobody knows which entity is responsible. FinOps asks you to find out. Another two days lost.
- An auditor asks for a list of every file processed in January. You grep Cloud Logging. Logs from before the last retention purge are gone.
- A schema change forces edits in ingest, validate, BQ table, dbt model, and Airflow config. You miss one. Bad data ships.
- Airflow 2.8 → 2.9 upgrade breaks an operator import path. You find out in prod.

Every one of those is something I have personally lived through. None of them is exotic; all of them are normal.

Version B — with `gcp-pipeline-framework`

Same scenario. Same business outcome. Here's what the code looks like.

Step 1: the schema

```
# my_schemas.py
from gcp_pipeline_core.schema import EntitySchema, SchemaField

CustomerSchema = EntitySchema(
    name="customers",
    primary_key=["customer_id"],
    fields=[
        SchemaField("customer_id",   dtype="STRING",  nullable=False, pii=False,
                    pattern=r"^\d{10}$"),
        SchemaField("full_name",     dtype="STRING",  nullable=False, pii=True),
        SchemaField("date_of_birth", dtype="DATE",    nullable=False, pii=True,
                    formats=["%Y-%m-%d", "%Y%m%d"]),
        SchemaField("postcode",      dtype="STRING",  nullable=True,  pii=True,
                    pattern=r"^[A-Z]{1,2}[0-9R][0-9A-Z]? ?[0-9][A-Z]{2}$"),
        SchemaField("opened_at",     dtype="TIMESTAMP", nullable=False),
    ],
    header_marker="HDR",
    trailer_marker="TRL",
)
```

That's the **single source of truth**. The ingestion reads it. The validator reads it. The Terraform reads it. The dbt PII macros read it. The tests read it. Change the schema once, everything follows.

Step 2: the Beam pipeline

```
# ingest_customers.py  — with the framework
from gcp_pipeline_beam.pipelines.beam import BeamPipelineBuilder
```

```
from gcp_pipeline_beam.file_management import HDRTRLParser
from gcp_pipeline_beam.transforms import RobustCsvParseDoFn,
↪  SchemaValidateRecordDoFn
import apache_beam as beam
from my_schemas import CustomerSchema

def run(options):
    builder = BeamPipelineBuilder(schema=CustomerSchema)
    with builder.pipeline(options) as p:
        raw, envelope = HDRTRLParser(file_uri=options.input_file).expand(p)
        valid, invalid = (
            raw
            | "ParseCsv" >> beam.ParDo(RobustCsvParseDoFn(schema=CustomerSchema))
            | "Validate" >>
            ↪  beam.ParDo(SchemaValidateRecordDoFn(schema=CustomerSchema))
                            .with_outputs("invalid", main="valid")
        )
        builder.write_valid(valid, table=options.bq_table)
        builder.write_invalid(invalid, bucket=options.error_bucket)
        builder.reconcile(envelope=envelope, valid=valid, invalid=invalid)
```

Fourteen lines of pipeline body. Every concern that was missing from Version A — audit trail, reconciliation, cost tracking, PII masking, structured logging, error classification, retries, worker-setup optimisation — is handled inside `BeamPipelineBuilder`. You *cannot* accidentally ship a pipeline without an audit record; the framework won't let you.

Step 3: the Airflow DAG

There isn't one. You edit four lines of YAML:

```
# system.yaml
system_id: generic
entities:
  customers:
    schedule: "@daily"
    sla_minutes: 45
    pattern: JOIN
    fdp_models: [event_transaction_excess]
```

The DAG factory (`generate_dags.py`) reads this and produces the ingestion DAG, the Pub/Sub trigger, the transformation DAG with JOIN preconditions, the error-handling DAG, and the status DAG. All of them wired to the same audit trail, all of them using the same structured logging, all of them emitting the same metrics.

Step 4: the dbt model

```
-- models/fdp/portfolio_account_excess.sql
{{ config(materialized='incremental', unique_key='account_id') }}

SELECT
    account_id, decision_type, decision_date, amount,
```

```
    {{ gcp_pipeline_transform.generate_audit_columns() }}
FROM {{ ref('stg_decision') }}
{% if is_incremental() %}
WHERE _loaded_at > (SELECT MAX(_loaded_at) FROM {{ this }})
{% endif %}
```

One macro gives you `_fdp_loaded_at`, `_fdp_run_id`, `_fdp_source_system`, `_fdp_environment` — the entire lineage. Add `apply_pii_masking(...)` on any PII field and the mask follows the schema automatically.

Step 5: the Terraform

```
module "customers_pipeline" {
  source    = "./modules/system"
  system_id = "generic"
  entities  = local.entity_config          # same system.yaml converted
  env       = "prod"
}
```

One module. Buckets, topics, datasets, IAM, labels, lifecycle rules, KMS, all provisioned per convention. You can still override anything; the defaults just work.

Step 6: the reality at month three

Same three months, same business demand:

- Add a second entity? Four lines in `system.yaml` plus a schema. Done in an hour.
- Mainframe starts splitting files? `SplitFileHandler` is already in `gcp-pipeline-beam`. It just works.
- A row is wrong? Click the `_fdp_run_id` in the row. You get the Dataflow job ID, the Airflow DAG run, the audit events, the reconciliation record, the cost, the logs, the trace. Three minutes.
- BigQuery bill doubles? Query `job_control.finops_usage` grouped by entity. You know in thirty seconds.
- Auditor asks for January? Query `job_control.audit_events` with a date filter. Done.
- Schema change? Edit `my_schemas.py`. CI reruns every test. Typed errors, not runtime surprises.
- Airflow upgrade? The framework is tested against the matrix (see next chapter). The upgrade is a dependency bump and a CI run.

Side-by-side summary

Concern	Without the library	With `gcp-pipeline-framework`
Lines of ingestion code	~120	~14
Lines of Airflow per entity	~30 hand-written	4 lines of YAML
HDR/TRL parsing	Write it; subtle bugs	`HDRTRLParser`; 359 tests
Schema drift	Five places to update	One `EntitySchema` object
`run_id` threading	Nothing	Everywhere

Concern	Without the library	With `gcp-pipeline-framework`
Reconciliation	Not done	`builder.reconcile(...)`
Cost tracking	Grep billing export	`finops_usage` table
PII masking	Hand-rolled per model	Schema-driven macro
Error classification	`except Exception`	`ErrorClassifier`
Retries with jitter	Nope	`RetryPolicy`
Structured logging	Print statements	`configure_structured_logging()`
OpenTelemetry traces	Nope	Optional, graceful degradation
Tests	None	Framework ships ~763; yours add more
Adding an entity	Copy-paste 150 lines	4 lines YAML + schema
Upgrade Airflow	Hope	Tested matrix

The punchline: **the framework is not doing anything magical**. Every single thing it does, you *could* do yourself. The cost of doing it yourself is that you are maintaining fifteen subsystems that have nothing to do with your business. The framework moves that cost off your plate.

If you take nothing else from this book: **never start a mainframe-to-BigQuery pipeline from scratch**. Either use this framework, or fork it, or find a better one. Starting over is the single most expensive mistake in data engineering.

Key takeaways

- The from-scratch version is about 120 lines of ingestion code, 30 lines of Airflow, and a hundred lines of Terraform — and it is missing reconciliation, audit, cost tracking, PII masking, error classification, retries, and tests.
- The framework version shrinks the ingestion body to 14 lines. The Airflow DAG becomes 4 lines of YAML. Every missing concern is handled inside `BeamPipelineBuilder` rather than invented again by you.
- The punchline is not magic — every single thing the framework does, you could write yourself. The cost of doing it yourself is maintaining fifteen subsystems that have nothing to do with your business.
- At month three, the ”from scratch” version accumulates copy-paste bugs, split-file failures, invisible data loss, and mystery costs. The framework version adds a new entity in an hour.

Chapter 6 — Foundations First: Building a Framework-Agnostic Core

The “core that knows nothing” principle

The most important design decision in `gcp-pipeline-framework` is negative: the foundation library, `gcp-pipeline-core`, has no runtime dependency on Apache Beam and no runtime dependency on Apache Airflow.

This sounds pedantic until you try to do it the other way. A foundation that imports Beam means your Airflow operators pull Beam in; your unit tests spin up Beam runners; your CI gets slow; your Cloud Functions cold-start lasts forty seconds. A foundation that imports Airflow means any tool that touches the core drags in half the Celery ecosystem. In both cases you have coupled your architecture to a specific execution substrate, which is exactly what a foundation library should not do.

So `gcp-pipeline-core` is careful. It depends on:

- The official GCP Python client libraries it needs (`google-cloud-bigquery`, `google-cloud-storage`, `google-cloud-pubsub`).
- `pydantic` for schema modelling.
- The standard library.
- *Optionally*, `opentelemetry-api` for tracing, but it gracefully degrades if the SDK is absent.

It does not depend on anything that would constrain where you can use it. You can import `gcp_pipeline_core.audit.AuditTrail` from a Cloud Function, a Dataflow worker, a Composer DAG, a standalone script, a Jupyter notebook, or a CI job. That portability is the point.

What the core provides

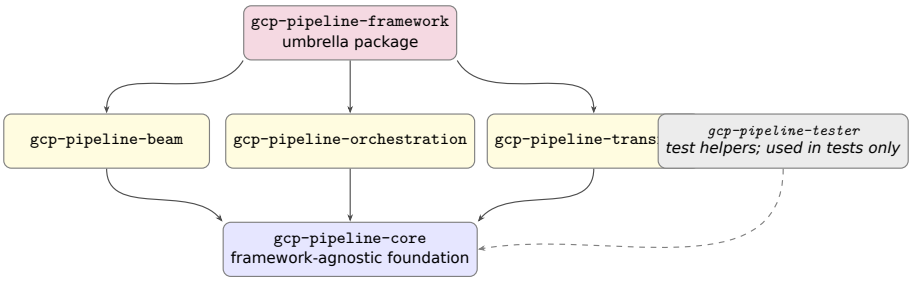

Figure 3: Library dependency tree across the six PyPI packages.

Eight subsystems live in the core:

- `audit/` — audit trail, reconciliation engine, duplicate detection.
- `monitoring/` — metrics collection, observability manager, health checks, alert manager, OTEL configuration.
- `finops/` — per-operation cost trackers for BigQuery, Cloud Storage, and Pub/Sub.
- `error_handling/` — error classifier, retry policy, backoff with jitter.
- `job_control/` — job-state repository and `PipelineJob` schema.
- `clients/` — thin wrappers over GCS, BigQuery, and Pub/Sub.
- `data_quality/` — quality checker, A-F grade calculator, anomaly detector.
- `data_deletion/` — safe deletion and four-level quarantine manager.

And one top-level module, `schema.py`, defines `EntitySchema` and `SchemaField` — the metadata types that every other layer of the framework reads from.

I will walk you through the ones that matter most.

Schema as single source of truth

The `EntitySchema` type is the beating heart of the framework. Conceptually it is just a description of a table:

```
from gcp_pipeline_core.schema import EntitySchema, SchemaField

CustomerSchema = EntitySchema(
    name="customers",
    primary_key=["customer_id"],
    fields=[
        SchemaField("customer_id",   dtype="STRING",  nullable=False, pii=False),
        SchemaField("full_name",     dtype="STRING",  nullable=False, pii=True),
        SchemaField("date_of_birth", dtype="DATE",    nullable=False, pii=True),
        SchemaField("postcode",      dtype="STRING",  nullable=True,  pii=True),
        SchemaField("opened_at",     dtype="TIMESTAMP", nullable=False, pii=False),
    ],
    header_marker="HDR",
    trailer_marker="TRL",
    expected_file_frequency="DAILY",
)
```

That single object is the source of truth for:

- **Beam ingestion.** `SchemaValidateRecordDoFn` reads the schema and enforces per-field types and nullability.
- **BigQuery table creation.** Terraform reads a JSON export of the schema to create the ODP table with the right columns and data types.
- **PII masking.** The dbt macro `apply_pii_masking` reads the same schema and emits `HASH(field)` for fields marked `pii=True`.
- **Data quality scoring.** The quality checker uses the schema to know which columns matter.
- **Test fixtures.** `gcp-pipeline-tester` can generate realistic fake rows from the schema alone.

In most codebases I have seen, these concerns live in five different places — the Beam code, the Terraform HCL, the dbt YAML, the tests, the documentation — and they drift. Within six months, someone has added a new column to the ingestion code but forgotten the masking macro. Bugs of this class are tedious to find and brand-damaging when they escape.

Making the schema the one true representation is, in my experience, the highest-leverage decision a framework author can make.

The audit trail

Every pipeline run in the framework starts by obtaining a `run_id`. Every log line, every metric, every BigQuery row, every Pub/Sub message carries that `run_id` as a field. The `audit/` subsystem is responsible for generating, propagating, and writing them.

```
from gcp_pipeline_core.audit import AuditTrail
from gcp_pipeline_core.utilities import generate_run_id

run_id = generate_run_id(system_id="generic", entity="customers")
audit = AuditTrail(run_id=run_id, system_id="generic", entity="customers")

audit.record_start(file_uri="gs://landing/customers.csv", expected_count=500_000)
# ... pipeline body ...
audit.record_end(loaded_count=499_998, rejected_count=2,
 ↪  status="SUCCESS_WITH_REJECTS")
```

Under the hood, `AuditTrail` writes to a BigQuery table called `job_control.audit_events` with a well-defined schema. Downstream, the reconciliation engine joins this table to the ingested data, the error bucket manifest, and any DLQ contents to produce a consolidated run report. When an auditor turns up in six months asking "what happened on 14 July", there is exactly one place to look.

A nice touch: the audit writer is **buffered and flushed on exit**. It does not call BigQuery synchronously on every event. Under load that would be both slow and expensive. Instead, the trail accumulates events in memory and batches them into streaming inserts on flush. On an unexpected exit, a registered `atexit` handler drains the buffer. It is a small thing that saves a lot of money.

Error classification

Not all errors are equal. A mainframe sending a row with a missing required field is a **validation error** — it is the sender's problem, and the right action is to quarantine the row and carry on. A temporary BigQuery 500 is an **integration error** — it is transient, and the right action is to back off and retry. A persistent out-of-memory is a **resource error** — it is my problem, and the right action is to fail loudly and page the on-call.

`ErrorClassifier` encodes this taxonomy:

```
from gcp_pipeline_core.error_handling import ErrorClassifier, RetryPolicy

classifier = ErrorClassifier()
policy = RetryPolicy(
    max_attempts=5,
    backoff_base_seconds=2,
    backoff_jitter_seconds=0.5,
    retry_on=classifier.integration_errors,
)

for attempt in policy.attempts():
    try:
        with attempt:
            bq.insert_rows_json(table, rows)
    except policy.RetriesExhausted as exhausted:
        audit.record_failure(kind="integration", detail=str(exhausted))
        raise
```

Two properties matter:

- The policy **does not retry validation errors**. If a row is malformed, retrying the same insert five times is just going to fail five times; the framework enforces that. This is surprisingly rare in frameworks I have reviewed.
- The backoff is **exponential with jitter**. Without jitter, large fleets of workers tend to retry in sync, which is exactly the herd behaviour that caused the failure you were retrying against.

FinOps as a first-class citizen

The `finops/` subsystem is unusual. Most pipeline frameworks treat cost as an afterthought — something your FinOps team will add dashboards for later. `gcp-pipeline-framework` treats it as a first-class concern and bakes it into the core.

Three trackers ship out of the box:

- `BigQueryCostTracker` — records bytes scanned per query, computed cost at the project's tier, and attributes it to a `run_id`.
- `CloudStorageCostTracker` — records object read/write operations and storage-class-adjusted cost.
- `PubSubCostTracker` — records publish and pull counts and the resulting cost.

Each tracker writes its records to a `finops_usage` BigQuery table with GCP labels for project, system, entity, and environment. A simple BI dashboard on top of that table answers the question most engineering managers actually want answered: "which entity is costing me the most, and is it worth it?"

The nice thing is that the trackers are optional at runtime. If a job does not import them, no cost rows are written. If it does, you get near-zero overhead observability into cost. Compare that with the alternative — mining GCP billing exports after the fact and trying to correlate them to your run IDs — and the value becomes obvious.

Data quality, grades, and anomalies

`data_quality/` implements three ideas:

- **A checker.** Given an entity and a sample, compute the proportion of rows with nulls, the proportion with invalid types (if any survived validation), the uniqueness of the primary key, and so on.
- **A grade.** Those metrics roll up into a grade from A to F. A is "this dataset is fit to ship to production". F is "hold everything, the mainframe sent us nonsense". The grade is written to the audit trail and optionally surfaced in the dashboard.
- **An anomaly detector.** A rolling baseline tracks row counts, null rates, and grade-level metrics over the last thirty days. A run whose count or distribution is more than three standard deviations outside that baseline raises an alert.

Anomaly detection is one of those things teams always intend to add and rarely do. Having it in the core, with a usable-out-of-the-box default, is one of the framework's quiet strengths.

Safe data deletion

In a regulated environment you cannot simply `DELETE FROM`. You need an approval trail, a quarantine period, and a recoverable path back if someone objects. `data_deletion/` implements a four-level workflow:

- **REVIEW.** A candidate deletion is proposed. Nothing is deleted; the row is tagged and an alert is raised.
- **HOLD.** An approver endorses the deletion. The row is moved to a hold dataset. Still reversible.
- **DELETE.** A second approver confirms. The row is removed from the primary table; a tombstone is retained.
- **ARCHIVE.** After the retention window expires, the tombstone is moved to a cold-line archive bucket. Recovery now requires a formal restore.

This is probably the single feature that most impresses auditors. It is tedious to build from scratch; having it ready-made, integrated with the audit trail, and tested is a huge head start.

Review: what works, what could be better

The core library is, in my view, the strongest part of the framework. Its insistence on being framework-agnostic, its unified schema model, and its first-class treatment of cost and deletion are all excellent choices.

The weaknesses are, to be fair, real too:

- **The `clients/` wrappers are thin.** They add value in structured logging and cost hooks, but they are not a full abstraction layer. You can still accidentally bypass them and call `google.cloud.bigquery.Client` directly, which a few places in the reference deployments do. A stricter lint rule would help.
- **Configuration coupling.** Several subsystems (audit, FinOps, deletion) read their target BigQuery dataset names from environment variables. This works but is not obvious. A single `CoreConfig` object that each subsystem accepts on construction would be cleaner.
- **OpenTelemetry degradation.** The optional OTEL dependency is handled gracefully, but the fallback is silent. A one-time warning at startup when OTEL is unavailable would save future debugging.

None of these is a deal-breaker. They are the kind of improvements you make in version 2.

With the foundation understood, we can now look upwards. Chapter 7 covers the ingestion library — where the Beam transforms that drive every pipeline live.

Key takeaways

- `gcp-pipeline-core` has no dependency on Beam or Airflow by design. That portability is what lets the same audit trail, cost tracker, and error handler work across Dataflow, Cloud Functions, and a Jupyter notebook.
- `EntitySchema` is the single source of truth: ingestion, BigQuery table creation, PII masking, test fixtures, and data quality scoring all read from the same object. Drift between those concerns is structurally impossible.
- The audit trail is buffered and flushed on exit. It does not call BigQuery on every event. That is the difference between a negligible overhead and a non-trivial line on your bill.
- The four-stage deletion workflow — REVIEW, HOLD, DELETE, ARCHIVE — is the feature that most impresses auditors, and it is the one most teams never get around to building from scratch.

Chapter 7 — Ingestion with Apache Beam: HDR/TRL and the Mainframe Problem

Why Beam, and why not just Beam

Apache Beam is not a fashionable technology in 2026. The Spark crowd thinks Beam is over-engineered. The DuckDB crowd thinks any distributed engine is overkill. The Snowflake crowd thinks ingestion is something you buy, not build. And yet, for a particular shape of problem — high-volume, schema-validated, batch-or-streaming, on Google Cloud — Beam on Dataflow is still the right answer.

The shape is roughly this: you have files on the order of gigabytes-to-terabytes, you need to apply per-record transformations that do not parallelise well in pure SQL, you need side outputs for error quarantine, and you want managed autoscaling. Dataflow gives you all of that in exchange for one Docker image and a JSON parameter file.

`gcp-pipeline-beam` is a library of opinionated Beam transforms for that use case. It provides:

- A `BasePipeline` abstract class that wires up logging, audit trails, FinOps tracking, and error handling so your pipeline body can focus on the actual work.
- A `BeamPipelineBuilder` that standardises the pattern of "read → parse → validate → split good/bad → write".
- A set of `DoFn` implementations for the things every mainframe pipeline needs: HDR/TRL parsing, robust CSV parsing, schema validation, PII detection, file archival.
- Validators for common field types: SSN, date in seven different formats, numeric with optional decimal, postcode, IBAN, and a handful of others.

Crucially, none of this is bound to Dataflow. The same library runs on the local DirectRunner for tests and on the FlinkRunner if you ever escape Google Cloud.

When to use Java instead of Python on Dataflow

If you spend any time in Dataflow pricing pages, you will eventually notice that two pipelines doing nominally the same work can have very different bills. Part of that is autoscaling configuration, or BigQuery slot reservations, or the size of side inputs.

But a meaningful slice of the difference comes down to a choice that often gets made by accident: whether the SDK is Python or Java.

The throughput gap is real. Java Beam delivers roughly 2–3× more processed records per Dataflow dollar than Python Beam, and the reason is not that Python is a slow language in general — it is two specific things. First, Dataflow Runner v2 (which is how all Python pipelines run) routes work through an SDK harness container that sits alongside the worker JVM. Every record crosses that harness-to-worker boundary; the JVM does not have an equivalent hop. Second, on the hot path, Java serialises through Kryo or AvroIO, both of which are extremely fast; Python serialises through CPython's pickle machinery, which is not. For a pipeline doing heavy per-record work — parsing, enrichment, ML inference — the CPU time on the actual business logic swamps the serialisation cost and the SDK matters less. But mainframe ingestion is *not* that kind of pipeline. The records are wide, the per-record logic is simple, and serialisation sits right in the critical path.

Volume is what makes the trade-off worth having the conversation about. Below roughly 50 million rows per day, Python Beam on Dataflow costs you tens of pounds per month. The overhead of running a polyglot toolchain — two SDKs, two CI lanes, two sets of dependency trees, doubled cognitive load for new joiners who now have to understand both — is a far bigger cost than the compute bill. The maths do not work. Above roughly 500 million rows per day, which is perfectly normal for a corporate-bank card-transaction or call-detail feed, a single high-throughput entity can be costing you £8–12K per month on Python Beam where the Java equivalent would run at £3–5K per month. At that point the saving is real money, it recurs every month, and it is worth the polyglot tax.

The cultural dimension matters too, and it is underrated. Mainframe operations teams are overwhelmingly Java-oriented. If you are writing files the mainframe will consume — segment-transform write-back, batch settlement feeds, regulatory exports — Java has the richer ecosystem: `com.legstar.*` for COBOL copybook binding, `Cp037` and `Cp1047` EBCDIC support via the standard JDK, IBM's own JZOS toolkit for direct VSAM interaction. A Python receive-side pipeline feels foreign to mainframe ops in a way that a Java one does not. That is not a reason on its own to rewrite everything, but when you are already at the volume threshold, the cultural fit is a tie-breaker that points in the same direction.

This framework is **Python-first**. We do not ship a parallel Java SDK, and we made that call deliberately. Every option we do not force a developer to choose between is a developer who ships faster. Ninety-five per cent of teams, ninety-five per cent of the time, should write Python using `gcp-pipeline-beam` and not think about any of this.

But we ship the contract. `docs/CONTRACT.md` is the language-neutral specification of the wire format: the column-level schema for `audit_events`, `finops_usage`, and `reconciliation_record`; the JSON Schema for `EntitySchema`; the `run_id` format and its generation rules; and the conformance criteria a pipeline must satisfy to be considered a valid emitter. A Java Beam pipeline that writes rows matching that contract is *interoperable* with everything else this framework produces. Audit lineage works. FinOps reporting works. Reconciliation works. The Java pipeline does not

need our Python library; it needs to honour our spec. That is the design: an open wire format, not a closed SDK.

The practical recommendation, if you have a genuine high-volume problem, is this. Build your default pipelines in Python using `gcp-pipeline-beam`. Identify the one or two entities at corporate-bank volume — the entities where the Dataflow bill is already showing up in your monthly FinOps report. Port *those specific pipelines* to Java Beam. They will write to the same BigQuery datasets, emit the same audit rows, and your downstream dashboards will be unable to tell the difference. That is the polyglot-at-the-Beam-layer pattern: Python for the 95%, Java for the 5% where the arithmetic compels you.

One thing worth being clear about: Java only helps at the Beam layer. dbt is SQL, so it does not care. Airflow DAGs are Python-only by design — there is no Java Airflow. `gcp-pipeline-core`, `gcp-pipeline-orchestration`, and `gcp-pipeline-transform` stay Python; the Beam layer is the only place a Java rewrite ever makes sense, and only when the volume justifies it.

See `docs/CONTRACT.md` for the full wire-contract spec a Java emitter would implement to.

The HDR/TRL pattern

A typical mainframe extract looks like this:

```
HDR|customers|20260417|0001|500000
0001|Alice Smith|1985-03-22|SW1A 1AA|2010-06-12T09:14:00Z
0002|Bob Jones |1979-11-04|EH8 9YL |2007-02-19T14:22:30Z
...
0500000|Zara Patel|1992-08-15|M1 1AA|2020-12-30T16:45:00Z
TRL|customers|20260417|0001|500000
```

The `HDR` row carries metadata: entity name, extract date, batch number, expected record count. The `TRL` row repeats this metadata, conventionally with the same numbers, as a self-check. The body rows are the actual data.

This pattern is older than I am, and it is everywhere. It is also subtly tricky to handle in Beam, because Beam's parallel execution model assumes records are independent, and HDR/TRL rows are not. The framework solves this with `HDRTRLParser`, a custom source that reads the file, separates the envelope from the body, and produces a stream of body records along with a single envelope object accessible through a side input.

Here is what calling it looks like in practice:

```
from gcp_pipeline_beam.file_management import HDRTRLParser
from gcp_pipeline_beam.pipelines.beam import BeamPipelineBuilder
from gcp_pipeline_beam.transforms import RobustCsvParseDoFn,
↪  SchemaValidateRecordDoFn
from my_schemas import CustomerSchema

builder = BeamPipelineBuilder(schema=CustomerSchema)
```

```
with builder.pipeline() as p:
    parser = HDRTRLParser(file_uri=options.input_file)

    raw, envelope = parser.expand(p)

    valid, invalid = (
        raw
        | "ParseCsv" >> beam.ParDo(RobustCsvParseDoFn(schema=CustomerSchema))
        | "Validate" >> beam.ParDo(SchemaValidateRecordDoFn(schema=
        ↪  CustomerSchema)).with_outputs("invalid",
        ↪  main="valid")
    )

    builder.write_valid(valid, table=options.bq_table)
    builder.write_invalid(invalid, bucket=options.error_bucket)
    builder.reconcile(envelope=envelope, valid=valid, invalid=invalid)
```

A few things to notice:

- The schema is passed once and used twice. Parsing knows the column order; validation knows the rules.
- The pipeline body fits on a screen. All the audit, cost, and error machinery from `BasePipeline` is implicit.
- `builder.reconcile()` compares the envelope's expected count to the actual valid+invalid count and writes a reconciliation row. Mismatches are surfaced as anomalies.

Robust CSV parsing

Mainframe CSVs are not the same thing as the CSVs you generate with `pandas.to_csv`. They have:

- Trailing whitespace on every field, because the mainframe pads fields to fixed widths.
- Mixed encodings (EBCDIC, latin-1, UTF-8) within a single batch.
- Embedded delimiters that may or may not be escaped.
- Empty trailing columns omitted entirely on some rows.
- Date fields where `00000000` means "null", not "1st Jan year zero".

`RobustCsvParseDoFn` handles all of this. It is a `DoFn` rather than a one-liner because each variant needs its own opinionated handling, and you want to be able to test each handler independently.

The implementation pattern is worth copying:

```
class RobustCsvParseDoFn(beam.DoFn):
    def __init__(self, schema):
        self.schema = schema

    def setup(self):
        # Heavy initialisation goes here, not in __init__
```

```
        self._encodings = ["utf-8", "latin-1", "cp037"]

    def process(self, element):
        line = self._decode(element)
        if line is None:
            yield beam.pvalue.TaggedOutput("invalid", {"reason": "decode-failed",
            ↪  "raw": repr(element)})
            return
        cells = self._split(line)
        record = self._coerce(cells)
        yield record
```

The trick is that `setup()` runs once per worker, not once per element. The framework's transforms follow this convention rigorously, which is the difference between a job that runs in eight minutes and a job that runs in eight hours.

Split-file reassembly

Real mainframes do not always send a 500,000-row file as one object. They sometimes split it into chunks, partly because of legacy network constraints and partly because their batch operators have policies about file sizes. You will see filenames like:

```
customers.20260417.001of005.csv
customers.20260417.002of005.csv
customers.20260417.003of005.csv
customers.20260417.004of005.csv
customers.20260417.005of005.csv
customers.20260417.001of005.ok
customers.20260417.002of005.ok
...
```

Each chunk has its own HDR and TRL. Each `.ok` file signals that one chunk is complete. The pipeline must wait for all five chunks to land before processing, then concatenate them in order, and finally treat them as one logical extract.

`SplitFileHandler` encapsulates this. It exposes a single method, `wait_for_complete_set(prefix)`, which polls GCS until all parts named `partXofY` have arrived along with their `.ok` markers. It returns a sorted list of URIs, which the pipeline reads as one logical input.

This kind of code is unglamorous and easy to get wrong. Having it tested, documented, and reused across deployments is a huge time saving.

Validators that compose

Field-level validation in the framework is built around small, composable validator classes:

```
from gcp_pipeline_beam.validators import (
    SSNValidator, DateValidator, NumericValidator, RegexValidator,
↪   RequiredValidator
```

```
)

postcode_validator = RegexValidator(r"^[A-Z]{1,2}[0-9R][0-9A-Z]? ?[0-9][A-Z]{2}$")
birth_date_validator = DateValidator(formats=["%Y-%m-%d", "%d/%m/%Y", "%Y%m%d"])

field_validators = {
    "customer_id": [RequiredValidator(), RegexValidator(r"^\d{10}$")],
    "postcode": [postcode_validator],
    "date_of_birth": [RequiredValidator(), birth_date_validator],
}
```

Each validator returns either `Valid` or `Invalid(reason)`. They compose: `RequiredValidator` short-circuits the rest if the field is missing, so subsequent validators do not need null guards.

`SchemaValidateRecordDoFn` collects per-field results, attaches them to the record as `_validation` metadata, and routes via Beam's tagged outputs:

```
class SchemaValidateRecordDoFn(beam.DoFn):
    def process(self, record):
        results = {field: run_validators(record, field) for field in
↪   self.schema.fields}
        if any(not r.is_valid for r in results.values()):
            yield beam.pvalue.TaggedOutput("invalid", {"record": record, "errors":
             ↪  results})
        else:
            yield record
```

This is the kind of code that looks trivial when you read it but is usually what people get wrong on their first try at building a pipeline. The framework gets it right and provides 359 unit tests that prove it.

The error bucket pattern

Bad records do not fail the job. They are written to a structured location:

```
gs://generic-error-bucket/
└── system=generic/
    └── entity=customers/
        └── extract_date=20260417/
            └── run_id=20260417T091400Z-7f3a/
                ├── invalid.jsonl
                └── manifest.json
```

`invalid.jsonl` contains one JSON object per rejected record, with the original raw input, the parsed cells, and the per-field error reasons. `manifest.json` summarises the run: how many records, how many rejects, the rejection rate, the audit `run_id`, and a link to the corresponding row in `job_control.audit_events`.

This structure is used by the `error_handling` DAG and by the quarantine review workflow. Because the layout is consistent across all entities, a single tool can scan the whole error bucket and produce a daily "rejection summary" for the data steward.

Dataflow Flex Templates

The reference ingestion deployment, `original-data-to-bigqueryload`, packages all of this as a **Dataflow Flex Template**. A Flex Template is a Docker image registered with Dataflow that can be launched on demand with a JSON parameter set. The pattern is essentially:

1. Build the image: `gcloud builds submit --config cloudbuild.yaml`.
2. Register the Flex Template: `gcloud dataflow flex-template build`.
3. Launch a job: `gcloud dataflow flex-template run <name> --template-file-gcs-location=...`.

Step 3 is the only one Airflow does in production. Steps 1 and 2 happen in CI. This separation is why the deployment is small: the actual Python code is in the image; the launcher is just a parameter file.

The reference `cloudbuild.yaml` builds the image, pushes it to Artifact Registry, runs `dataflow flex-template build`, and finally writes the template manifest URI to a Terraform-known location. It is one of the cleaner CI artefacts in the project.

Review: what works, what could be better

`gcp-pipeline-beam` is the library that does the most work for the smallest interface. The compositional structure (small validators, transforms with clear contracts, side outputs for errors) is exemplary. The HDR/TRL parser is the kind of thing you would want to adopt even if you used nothing else from the framework.

A few honest observations:

- **The `BeamPipelineBuilder` is convenient but a little magical.** It hides the audit trail, FinOps wiring, and error routing. For a beginner this is great. For a pipeline that needs an unusual topology, you sometimes want the explicit version. A documented "lower-level" API would help.
- **Streaming support is uneven.** The framework is batch-first. The `postgres-cdc-streaming` reference deployment exists but is marked as planned. Real streaming use cases require care that the abstractions do not yet fully cover.
- **Performance tuning is left to the user.** Choosing the right number of workers, the right machine type, the right shuffle service mode — these are not encapsulated. A future "deployment profile" abstraction would help.

Overall, this library is the framework's biggest selling point for engineers. Anyone who has built mainframe ingestion in raw Beam will recognise immediately how much pain it removes.

In the next chapter we leave Beam behind and look at the second unit — transformation in dbt, where the JOIN and MAP patterns come into their own.

Key takeaways

- Python Beam is the right default for 95% of teams. Java Beam only makes financial sense above roughly 500 million rows per day, where the 2–3× throughput advantage starts paying off a meaningful cost delta each month.
- `HDRTRLParser` is not merely a parser — it validates the envelope count, handles split-file reassembly, and makes the envelope available as a Beam side input for reconciliation. Getting this right once is the point.
- `setup()` in a `DoFn` runs once per worker, not once per element. Violating that convention turns an eight-minute job into an eight-hour one.
- The error bucket layout is structured by design: every reject carries its `run_id`, its validation reasons, and a link to the audit event. That structure is what makes the quarantine workflow possible downstream.

Chapter 8 — Transformation with dbt: ODP, FDP, JOIN, and MAP

Why dbt, and which bits of dbt

There are two places you can put transformation logic in a GCP pipeline: in code (Beam, SQL scripts, stored procedures) or in dbt. For most use cases, dbt wins. It gives you versioned SQL, documented lineage, automated tests, incremental models, and a deployment artefact that your security team is going to find easier to audit than a 2,000-line Python Beam job.

The reference deployment `bigquery-to-mapped-product` is a dbt project. It targets BigQuery exclusively. It uses the `dbt-bigquery` adapter and expects a service account with dataset-level permissions on the ODP, FDP, and audit datasets. It runs either through the `transformation` Airflow DAG or through `dbt run` from a local shell.

What the framework adds on top of stock dbt is three things:

- A **layering convention** (staging → FDP → marts → analytics) that every model fits into.
- A **macro library** (`gcp-pipeline-transform`) that injects audit columns and PII masking without per-model boilerplate.
- A **pattern vocabulary** — JOIN and MAP — that makes the relationship between ingestion and transformation explicit.

The layering convention

Inside the dbt project, models live in four subdirectories:

```
models/
├── staging/        # 1:1 with ODP, light cleanup
├── fdp/            # JOIN and MAP models — the business layer
├── marts/          # Domain-specific mart views
└── analytics/      # Ad-hoc views and aggregations
```

Staging models are views, one per ODP table. They deal with the dull mechanical work: renaming columns into snake_case if the source uses camelCase, casting strings to proper types where the ODP left them as strings for flexibility, and trimming padded fields. Every staging model ends in `_staging`.

```
-- models/staging/stg_customers.sql
{{ config(materialized='view') }}

SELECT
    customer_id,
    TRIM(full_name)      AS full_name,
    CAST(date_of_birth AS DATE) AS date_of_birth,
    UPPER(TRIM(postcode)) AS postcode,
    opened_at,
    _run_id,
    _loaded_at
FROM {{ source('odp_generic', 'customers') }}
```

FDP models are where the business logic lives. They are tables (not views), they are incremental where possible, and they are where the JOIN vs MAP distinction becomes important.

Mart models are domain-oriented projections of the FDP. They answer specific business questions: the "customer 360" mart, the "excess-management" mart, the "loan-origination" mart.

Analytics models are the fastest-moving layer. They exist for ad-hoc analysis and may be dropped or rebuilt without affecting anything downstream. Keeping them in a clearly labelled directory stops the FDP from getting polluted with one-off SQL.

The MAP pattern in detail

A MAP model is one-to-one: one ODP source, one FDP target. It triggers immediately after its source is loaded; it does not wait for anything else.

```
-- models/fdp/portfolio_account_excess.sql
{{ config(
    materialized='incremental',
    partition_by={'field': '_loaded_at', 'data_type': 'timestamp'},
    unique_key='account_id'
) }}

SELECT
    d.account_id,
    d.decision_type,
    d.decision_date,
    d.amount,
    -- Audit columns injected by the framework macro
    {{ gcp_pipeline_transform.generate_audit_columns() }}
FROM {{ ref('stg_decision') }} d
WHERE
    {% if is_incremental() %}
      d._loaded_at > (SELECT MAX(_loaded_at) FROM {{ this }})
    {% endif %}
```

Two things are worth noting here:

- The **incremental materialisation** means dbt only processes rows loaded since the last run. For a daily full-load table of five million rows, this turns a ten-minute transform into a ten-second one.
- The **audit macro** injects a consistent set of columns (`_fdp_loaded_at`, `_fdp_run_id`, `_fdp_source_system`) without the model author having to think about them.

MAP models are cheap, simple, and easy to reason about. If your use case fits, always prefer MAP.

The JOIN pattern in detail

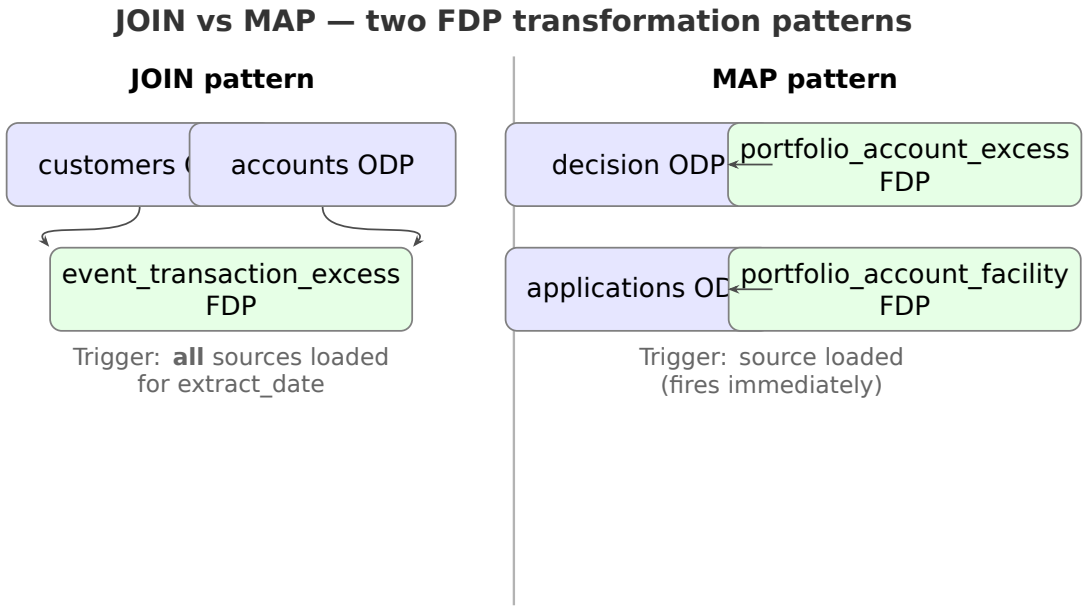

Figure 4: JOIN vs MAP — two FDP transformation patterns.

A JOIN model combines two or more ODP sources into a single FDP table. The classic example in the reference deployment is `event_transaction_excess`, which joins customers to accounts to produce a per-account event record enriched with customer metadata:

```
-- models/fdp/event_transaction_excess.sql
{{ config(
    materialized='incremental',
    partition_by={'field': 'event_date', 'data_type': 'date'},
    cluster_by=['customer_id']
) }}

SELECT
    a.account_id,
    a.customer_id,
    c.full_name,
```

```
    c.postcode,
    a.event_type,
    a.event_amount,
    a.event_date,
    {{ gcp_pipeline_transform.apply_pii_masking(schema='customers',
↪   field='full_name') }} AS full_name_masked,
    {{ gcp_pipeline_transform.generate_audit_columns() }}
FROM {{ ref('stg_accounts') }} a
LEFT JOIN {{ ref('stg_customers') }} c
    USING (customer_id)
WHERE
    {% if is_incremental() %}
      a._loaded_at > (SELECT MAX(_loaded_at) FROM {{ this }})
    {% endif %}
```

The architectural subtlety is that JOIN models **depend on multiple upstream ingestion runs finishing successfully**. Running the transformation before both sources are loaded produces a table that is subtly wrong: the join finds no matches, or finds stale matches. In the worst case nobody notices until a regulator does.

The framework solves this with `EntityDependencyChecker` in the orchestration layer — a component we will look at in detail in Chapter 9. For now it is enough to know that the transformation DAG waits for every JOIN entity's full source set before firing.

Audit columns and PII masking

Two macros in `gcp-pipeline-transform` do the heavy lifting:

`generate_audit_columns()` expands into:

```
CURRENT_TIMESTAMP() AS _fdp_loaded_at,
'{{ invocation_id }}'::STRING AS _fdp_run_id,
'{{ var("system_id") }}'::STRING AS _fdp_source_system,
'{{ var("environment") }}'::STRING AS _fdp_environment
```

Every FDP row carries the dbt invocation ID that produced it, which aligns with the Airflow DAG run ID and with the Dataflow job ID upstream. End-to-end lineage reduces to a single join across `job_control.audit_events`.

`apply_pii_masking(schema, field)` reads the framework's `EntitySchema` definitions, looks up whether the named field is marked `pii=True`, and emits `TO_HEX(SHA256(CAST(field AS STRING)))` if so, or the raw field if not. This means masking is a property of the data, not of the SQL. Add a new PII field to the schema; every FDP that references it starts masking automatically.

Marts and analytics

The `marts/` directory holds domain-specific projections. A mart is not supposed to do heavy computation — that lives in the FDP. A mart selects, filters, and shapes FDP rows for a particular audience.

```sql
-- models/marts/excess_management/excess_customer_view.sql
SELECT
    e.account_id,
    e.customer_id,
    e.full_name_masked,
    e.event_date,
    e.event_amount,
    pax.decision_type,
    pax.decision_date
FROM {{ ref('event_transaction_excess') }} e
LEFT JOIN {{ ref('portfolio_account_excess') }} pax
    USING (account_id)
WHERE e.event_date >= CURRENT_DATE() - 90
```

The `analytics/` directory is intentionally rougher. Analysts add views here without going through full code review, knowing that nothing downstream depends on them. If a view proves its worth, it graduates to `marts/`.

dbt tests that matter

Every FDP model ships with two classes of test:

- **Schema tests.** Declared in `schema.yml`: uniqueness on the primary key, `not_null` on required fields, `accepted_values` on enums, `relationships` for referential integrity.
- **Data-quality tests.** Custom tests that pipe through `gcp-pipeline-transform` macros. Example: `test_audit_columns_present` asserts that every row has a non-null `_fdp_run_id`, which in turn asserts that every row came through the framework's macros rather than through an ad-hoc `INSERT`.

```yaml
version: 2
models:
  - name: event_transaction_excess
    columns:
      - name: account_id
        tests:
          - not_null
          - unique
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('stg_customers')
              field: customer_id
    tests:
      - gcp_pipeline_transform.audit_columns_present
      - gcp_pipeline_transform.pii_fields_masked
```

dbt tests run in CI on every PR and in production as a final step of the transformation DAG. A failing test produces a structured alert into the same Alert Manager that the ingestion side uses, and the audit trail gets a corresponding `TEST_FAILED` event.

Reviewing the transformation layer

The dbt layer is the simplest part of the framework — deliberately so. Keeping it boring and opinionated is what makes the rest of the system reliable.

Strengths:

- The **audit macro is mandatory in convention and enforced by test**. Skipping it produces a failing build, not a silent data-quality regression.
- The **PII macro drives masking from the schema**, not from the SQL. Masking is therefore consistent across every model that references a PII field.
- The **mart/analytics split** keeps the FDP stable while letting analysts experiment.

Weaknesses:

- The `gcp-pipeline-transform` library **lacks explicit pytest coverage**. dbt's own test framework covers the macros at the SQL level, but unit tests for the Python helpers that generate the macro SQL are thin.
- **Incremental models on wide tables can still be expensive**. The framework does not currently offer a partition-pruning helper beyond the standard dbt `is_incremental()` block.
- **There is no standard "full refresh" playbook** for recovering a JOIN table after a historical correction. The reference implementation assumes you drop-and-recreate, which is fine at small scale but painful on a multi-terabyte FDP.

Chapter 9 picks up where this one leaves off — with orchestration, where the JOIN/MAP distinction turns from a pattern into real code.

Key takeaways

- The MAP pattern — one ODP source, one FDP table — fires immediately. The JOIN pattern — multiple ODP sources, one FDP table — must wait for every constituent. Confusing them produces subtly wrong data, not an obvious failure.
- `generate_audit_columns()` injects `_fdp_run_id`, `_fdp_source_system`, and `_fdp_loaded_at` into every FDP row. End-to-end lineage is then a single join across `job_control.audit_events`.
- `apply_pii_masking()` reads the `EntitySchema` directly. Add a new PII field to the schema; every FDP model that references it starts masking automatically, with no SQL to update.
- Incremental materialisation on the MAP pattern turns a ten-minute full-table scan into a ten-second incremental one. Always prefer MAP when the data fits.

Chapter 9 — Orchestration with Airflow: When and When Not to Use Composer

The brief for orchestration

An orchestration layer answers four questions:

- **When should a pipeline run?** On a schedule, on demand, on an event, or on a chain of events.
- **In what order?** Parallel where possible, sequential where required.
- **What if something fails?** Retry, quarantine, alert, page.
- **How do I know it worked?** State, history, dashboards, audit.

Airflow answers all four, which is why the framework uses it. The reference deployment `data-pipeline-orchestrator` runs on Cloud Composer 2 with Airflow 2.8+. It ships five DAGs:

- `pubsub_trigger` — event-driven entry point.
- `ingestion` — entity-parameterised Dataflow launcher.
- `transformation` — dbt runner with dependency gating.
- `error_handling` — quarantine lifecycle manager.
- `status` — consolidated run reporting.

Crucially, none of these DAGs is hand-written. They are all generated by a factory, `generate_dags.py`, from a YAML configuration. Hand-maintaining a DAG per entity is a well-known anti-pattern; the framework avoids it by design.

The DAG factory

`generate_dags.py` is an 87 KB Python file in the `dags/` directory of the orchestration deployment. Composer picks up any `.py` file in its DAGs bucket, so all a factory has to do is register `DAG` objects in module globals. The framework does exactly that:

```
# Simplified illustration
from gcp_pipeline_orchestration.factories import DagFactory

factory = DagFactory.from_config("system.yaml")
```

```
for entity_name, dag in factory.ingestion_dags():
    globals()[f"ingestion_{entity_name}"] = dag

for dag in factory.transformation_dags():
    globals()[dag.dag_id] = dag

globals()["pubsub_trigger"] = factory.pubsub_trigger_dag()
globals()["error_handling"] = factory.error_handling_dag()
globals()["status"] = factory.status_dag()
```

`system.yaml` is short and declarative:

```
system_id: generic
entities:
  customers:
    schedule: "@daily"
    sla_minutes: 45
    pattern: JOIN
    fdp_models: [event_transaction_excess]
  accounts:
    schedule: "@daily"
    sla_minutes: 45
    pattern: JOIN
    fdp_models: [event_transaction_excess]
  decision:
    schedule: "@daily"
    sla_minutes: 30
    pattern: MAP
    fdp_models: [portfolio_account_excess]
  applications:
    schedule: "@daily"
    sla_minutes: 30
    pattern: MAP
    fdp_models: [portfolio_account_facility]
```

Adding a new entity is a four-line YAML change plus a new schema definition. No DAG edits. No orchestration code review. Auditors love this.

The full entity configuration schema

The abbreviated YAML above covers the minimum case. In practice, the factory understands a richer configuration surface. Here is the complete schema, with defaults annotated:

```
system_id: generic                # required; maps to dataset/bucket prefixes
environment: prod                 # required; drives tfvars and labels

# Global defaults applied to every entity unless overridden
defaults:
  schedule: "@daily"
  sla_minutes: 60
  retries: 2
```

```
  retry_delay_minutes: 5
  max_active_runs: 1
  worker_machine_type: n1-standard-2
  worker_max_count: 20
  dataflow_region: europe-west2
  dataflow_network: pipeline-vpc
  dataflow_subnetwork: pipeline-subnet-euw2
  on_failure_callback: alert_manager.paging
  on_sla_miss_callback: alert_manager.warn
  labels:
    team: data-platform
    costcenter: 1024

entities:
  customers:
    schedule: "@daily"
    sla_minutes: 45
    pattern: JOIN                  # JOIN | MAP
    fdp_models:
      - event_transaction_excess
    # Optional per-entity overrides
    worker_max_count: 40
    expected_row_count_min: 450000
    expected_row_count_max: 550000
    # Source mapping — where to find the file, how to recognise it
    source:
      landing_uri_template: gs://{landing_bucket}/customers/{ds_nodash}/*.csv
      ok_file_suffix: .ok
      split_pattern: "{entity}.{ds_nodash}.{part:03d}of{total:03d}.csv"
    # Target mapping — ODP table
    target:
      project_id: my-project
      dataset: odp_generic
      table: customers
      partition_field: opened_at
      cluster_fields: [customer_id]
    # Error mapping — quarantine layout
    error:
      bucket: generic-error-bucket
      prefix: system=generic/entity=customers
    # Dependency graph (for JOIN entities)
    joins_with:
      - accounts

  accounts:
    pattern: JOIN
    fdp_models: [event_transaction_excess]
    joins_with: [customers]
    source:
      landing_uri_template: gs://{landing_bucket}/accounts/{ds_nodash}/*.csv
    target:
      dataset: odp_generic
      table: accounts
```

```yaml
  decision:
    pattern: MAP
    fdp_models: [portfolio_account_excess]
    source:
      landing_uri_template: gs://{landing_bucket}/decision/{ds_nodash}/*.csv
    target:
      dataset: odp_generic
      table: decision

  applications:
    pattern: MAP
    fdp_models: [portfolio_account_facility]
    source:
      landing_uri_template: gs://{landing_bucket}/applications/{ds_nodash}/*.csv
    target:
      dataset: odp_generic
      table: applications

# FDP models referenced above — lets the factory build correct transformation DAG
fdp_models:
  event_transaction_excess:
    pattern: JOIN
    sources: [customers, accounts]
    materialization: incremental
    schedule: post_all_sources
  portfolio_account_excess:
    pattern: MAP
    sources: [decision]
    materialization: incremental
    schedule: post_source
  portfolio_account_facility:
    pattern: MAP
    sources: [applications]
    materialization: incremental
    schedule: post_source

# Alerting mapping
alerts:
  channels:
    slack:  "#generic-pipeline-alerts"
    pagerduty_service_key: ${PD_SERVICE_KEY}
    email_oncall: oncall@example.com
  rules:
    sla_miss: slack
    reconciliation_failure: slack, pagerduty
    data_quality_below_B: slack
    ingestion_failed: slack, pagerduty, email_oncall
```

Two things to notice.

First, every piece of a pipeline's behaviour that a team would reasonably change is in this file. Not in Python. Not in a Terraform variable. Not in a

BigQuery table. Reducing the editable surface to one YAML is, in my view, the single most valuable thing this factory does. It is what makes auditors comfortable: every behavioural decision is in one version-controlled file, reviewable as a diff.

Second, the config is not just consumed by Airflow. The same `system.yaml` is read by:

- The **DAG factory** to generate the Airflow DAGs.
- The **Terraform module** to provision the right buckets, topics, and datasets.
- The **entity dependency checker** to know which JOIN sources to wait for.
- The **error-handling DAG** to know where each entity's quarantine lives.
- The **status DAG** to consolidate per-entity state.
- The **reconciliation engine** to know expected row ranges.
- The **FinOps dashboard** to label cost records per entity.

This is the heart of the framework's "configuration as first-class citizen" philosophy. The YAML is not a script input; it is the specification. Everything else derives from it.

Mapping: source → ODP → FDP → CDP

The word "mapping" appears in a few places in this codebase. It is worth separating them out because confusing them causes real bugs.

File-to-ODP mapping. The `source` and `target` blocks in `system.yaml` define where files arrive and which ODP table they populate. The framework's ingestion does not need to be told "customer file → customers table" beyond this mapping — the runtime pipeline reads the entity name, looks it up in the config, and knows the rest.

ODP-to-FDP mapping. Two flavours, as you have seen: **JOIN** mapping (many ODP sources feed one FDP target) and **MAP** mapping (one ODP source feeds one FDP target). These are declared in the `fdp_models` block and enforced by the transformation DAG.

FDP-to-CDP mapping. The `fdp-to-consumable-product` deployment (Chapter 16) takes FDP rows and materialises contracted CDP views. Mapping here is SQL-driven (dbt models), but the **contract** is declared in a separate YAML per CDP:

```yaml
# consumable_products/customer_360.yml
consumable_product: customer_360
owner: bi-team
source_fdp_models: [event_transaction_excess, portfolio_account_excess]
contract_version: 1.2.0
columns:
  - name: customer_id
    source: event_transaction_excess.customer_id
    pii: false
  - name: full_name_masked
    source: event_transaction_excess.full_name_masked
    pii: true
  - name: most_recent_decision
    source: portfolio_account_excess.decision_type
    pii: false
```

```
refresh_cadence: daily
retention_days: 365
```

The advantage: a consumer reads the YAML and knows exactly which FDP fields they are coupled to. A change to an FDP field's name triggers a CDP contract failure in CI. No silent schema drift.

Airflow dynamic task mapping. Since Airflow 2.3, the framework uses Airflow's **dynamic task mapping** to fan out work across entities without hand-writing task groups. The ingestion DAG, for example, produces as many `run_dataflow` task instances at runtime as there are entities to process in that run:

```
# Simplified illustration inside the factory
@task
def list_ready_entities(**context):
    """Return the list of entities ready to run this DAG instance."""
    return EntityRouter.ready_for_extract_date(context["ds"])

@task
def run_ingestion(entity_name: str, **context):
    """Launch the Dataflow Flex Template for this entity."""
    return DataflowLauncher(entity_name).run(context)

ready = list_ready_entities()
run_ingestion.expand(entity_name=ready)
```

This is what Airflow calls **expand/map/reduce** — the upstream task returns a list, the downstream task runs once per element, and a final `.reduce` (if you need one) consolidates results. The framework uses it anywhere you would otherwise have written a for-loop around DAG construction. Adding an entity does not add a task; the mapping adjusts itself at runtime.

When you combine config-driven DAG generation with Airflow dynamic task mapping, you get the best of both:

- **Static shape known at parse time** — Airflow's scheduler can plan the DAG with stable task-group names for SLA tracking, UI grouping, and RBAC.
- **Dynamic breadth at runtime** — the same DAG handles 4 entities today, 40 entities next quarter, without a code change.

The Pub/Sub trigger pattern

The `pubsub_trigger` DAG is small but clever. It uses a custom sensor, `BasePubSubPullSensor`, that pulls from a topic subscription, extracts the file URI, matches it against the known entity set, and triggers the appropriate `ingestion_<entity>` DAG via Airflow's `TriggerDagRunOperator`.

```
# Simplified
class IngestionPubSubSensor(BasePubSubPullSensor):
    def process_message(self, message):
        uri = message.attributes["objectId"]
        entity = self.match_entity(uri)
```

```
        if entity is None:
            return False  # Not a file we care about; ack and move on
        self.trigger_dag(
            dag_id=f"ingestion_{entity}",
            run_id=f"{entity}-{uri.split('/')[-1]}",
            conf={"input_file": f"gs://landing/{uri}"},
        )
        return True
```

A few subtleties make this production-ready:

- **Idempotent acks.** If the sensor triggers the DAG but dies before acking, Pub/Sub redelivers. The trigger operator checks for an existing run with the same `run_id` and no-ops rather than double-running.
- **Dead-letter routing.** A message that matches no entity after three attempts is routed to a dead-letter topic and the `error_handling` DAG picks it up.
- **Backpressure.** The sensor limits to ten messages per pull, preventing a backlog from overwhelming the scheduler.

The Ingestion DAG

A generated ingestion DAG is astonishingly short:

```
with DAG(
    dag_id=f"ingestion_{entity}",
    schedule=None,  # Triggered by pubsub_trigger
    default_args=default_args,
    max_active_runs=1,
    catchup=False,
    tags=["ingestion", system_id, pattern],
) as dag:

    start = record_start(entity=entity, run_id="{{ run_id }}")
    launch = DataflowFlexTemplateOperator(
        task_id="run_dataflow",
        template=template_uri,
        parameters={
            "input_file": "{{ dag_run.conf['input_file'] }}",
            "bq_table": bq_target_table(entity),
            "error_bucket": error_bucket(),
            "run_id": "{{ run_id }}",
        },
    )
    reconcile = reconcile_task(entity=entity, run_id="{{ run_id }}")
    end = record_end(entity=entity, run_id="{{ run_id }}")

    start >> launch >> reconcile >> end
```

The real power is in the helpers (`record_start`, `reconcile_task`, `record_end`) — each wraps audit, FinOps, and error-classification logic from `gcp-pipeline-core`. The DAG author never writes "if the job fails, page someone" explicitly; that behaviour is in the helper.

The Transformation DAG and `EntityDependencyChecker`

The transformation DAG is where the JOIN/MAP distinction surfaces. For MAP entities it fires immediately on their ODP load. For JOIN entities it waits.

Waiting is the job of `EntityDependencyChecker` — a small class in `gcp-pipeline-orchestration` that answers: "have all entities required for FDP model X been loaded for today's partition?" It does so by querying `job_control.pipeline_runs` for the latest successful load per entity for a given extract date.

```
from gcp_pipeline_orchestration.dependency import EntityDependencyChecker

def ready_for_event_transaction_excess(extract_date, **_):
    checker = EntityDependencyChecker(dataset="job_control")
    return checker.all_loaded(
        entities=["customers", "accounts"],
        extract_date=extract_date,
    )
```

In the DAG this becomes a `ShortCircuitOperator`: if not all entities are ready, the transform is skipped this run and reattempted on the next schedule. No `sleep` loops, no deadlocks, no "waiting for 30 minutes then giving up" anti-patterns.

The Error-Handling DAG

Error handling gets its own DAG because quarantine is a workflow, not a transaction. It has stages:

- **Daily scan.** Enumerates the error bucket for new `invalid.jsonl` files.
- **Classification.** Uses `ErrorClassifier` to decide whether each batch is "replay eligible" (validation errors likely fixed by a re-extract) or "needs intervention".
- **Replay.** For replay-eligible batches, triggers the ingestion DAG with a `replay=True` flag. The ingestion DAG in turn reads from the error bucket rather than the landing bucket.
- **Quarantine aging.** Rejects older than 30 days move to a cold-line archive bucket. Older than 365 days are deleted (subject to the safe-deletion workflow from Chapter 6).

Crucially, the error-handling DAG is **idempotent**. Running it twice has no effect beyond the first run, which means it can be scheduled frequently without risk.

The Status DAG

The `status` DAG is the smallest of the five. It does one thing: consolidates run data from `job_control.audit_events`, `job_control.pipeline_runs`, and the dbt `run_results.json` artefact into a daily rollup table that powers dashboards. Without this, every dashboard has to reimplement the rollup logic; with it, there is one authoritative source.

When Composer is overkill

Cloud Composer 2 starts at approximately 300 US dollars a month before a single task is scheduled. It is a managed Airflow environment, which means it runs a GKE cluster, a Postgres metadata database, a web server, a scheduler, and workers — all of which you pay for around the clock.

For a small pipeline, this is absurd. If you have three entities, daily loads, and no compliance requirement that demands Airflow specifically, the framework supports three cheaper alternatives out of the box:

- **Cloud Functions** for the Pub/Sub trigger. One second of execution per file, rather than a scheduler that runs forever.
- **Cloud Run Jobs** for scheduled runs. A `dbt run` can happen in a Cloud Run Job on a Cloud Scheduler trigger for approximately the cost of a coffee per month.
- **Workflows** for sequencing, if you need DAG-like semantics without Airflow.

The reference Terraform module for orchestration is therefore **explicitly opt-in**. `terraform apply` in the orchestration system does not provision Composer unless you set `deploy_composer = true` in your tfvars. This is a deliberate decision to keep the framework's default total cost of ownership under control.

The Airflow version matrix

The single most frequent question I get from teams evaluating the framework is: *"Which Airflow version do I need?"* The answer, in 2026, is slightly nuanced.

What the framework supports

`gcp-pipeline-orchestration` is tested against:

Airflow	Python	Composer	Status	Notes
2.7.x	3.10, 3.11	Composer 2.5+	Legacy, best-effort	Works but no new provider features
2.8.x	3.10, 3.11	Composer 2.6+	**Primary target**	Default CI runs here
2.9.x	3.11, 3.12	Composer 3.0	Supported	Recommended for new deployments
2.10.x	3.11, 3.12	Composer 3.1+	Supported	Deferrable operators enabled by default
3.0.x	3.12	— (not yet on Composer)	Experimental	Track on the `airflow-3` branch

Three concrete implications:

- **New projects in 2026: pick Airflow 2.10 on Composer 3.1.** That's where the scheduler performance improvements and the full deferrable operator model live.
- **Existing Composer 2 projects on Airflow 2.7: plan a jump to 2.9, then 2.10.** Two hops is cleaner than one big one. Provider changes between 2.7 and 2.10 break a handful of imports.
- **Do not jump to Airflow 3 on production yet.** It is stabilising; Composer support is not universal; provider compatibility is still settling.

What pins to watch

The framework pins providers through `constraints` files rather than hard version ranges, so a DAG that works on one environment will import cleanly on another. The three provider packages that matter most:

- `apache-airflow-providers-google` — the Dataflow, BigQuery, GCS, Pub/Sub operators. Major version jumps between Airflow 2.x series.
- `apache-airflow-providers-cncf-kubernetes` — needed if you use the KubernetesPodOperator (see the Kubernetes chapter).
- `apache-airflow-providers-slack` — for the default alert backend.

CI maintains a `constraints-airflow-2.8.txt`, `constraints-airflow-2.9.txt`, and `constraints-airflow-2.10.txt` and runs the full test suite against each. A PR that breaks any of the three is blocked.

The Airflow 3 upgrade story

Airflow 3 introduces three things the framework cares about:

- **Task execution API** — a hardened boundary between scheduler and worker. Good for security; forces a few import updates.
- **Deferrable by default** — sensors deferrable without opt-in. A few framework sensors need small changes (mostly deleting the explicit `deferrable=True` flag).
- **Dataset-driven scheduling is mature** — useful for us; the framework's Pub/Sub-trigger pattern can become a dataset-updated trigger.

When the framework migrates (likely 2026–2027), it will ship a `v2.0` of `gcp-pipeline-orchestration` with a compatibility shim that lets v1 DAGs keep running while you migrate entity-by-entity. You will not have to move all DAGs at once.

If you cannot use Composer at all

Some organisations cannot use Composer because:

- They are in GCP regions where Composer is unavailable.
- They need VPC-SC configurations Composer does not fully support.
- They need a shared cluster with non-Airflow workloads.
- They run a hybrid estate (some pipelines on-prem).

For those teams, the framework ships a Helm-chart-based self-managed Airflow deployment. We cover it in detail in the Kubernetes chapter.

Review: what works, what could be better

Strengths:

- **The DAG factory pattern is the right answer.** Configuration-driven DAGs are the only scalable way to manage a multi-entity pipeline.
- **The sensor's idempotence discipline is strong.** Double-triggers do not double-run.
- **The JOIN dependency logic is clean.** `ShortCircuitOperator` + a stateless checker beats hand-rolled sleep-and-poll every time.
- **The opt-in Composer model saves money.** Most teams do not realise they are billed for Composer on weekends too.
- **Version matrix is explicit.** CI runs against three Airflow minors simultaneously; upgrade surprises are rare.

Weaknesses:

- **The factory is a single 87 KB file.** It is readable but imposing. Splitting it by concern (ingestion, transformation, errors, status) would help future maintainers.
- **The Cloud Functions alternative is documented but not scaffolded**. A reference Cloud Functions deployment would be a useful addition.
- **SLA handling is present but conservative.** Missed SLAs are alerted, not actioned. A future version might trigger an auto-escalation workflow.
- **Airflow 3 migration path is still branch-only.** Needs a proper v2 release once Composer 4 lands.

Chapter 10 now shifts gears entirely. We leave runtime code behind and look at the infrastructure-as-code layer that provisions the resources all of the above depend on.

Key takeaways

- Composer starts at roughly $300/month before a single task is scheduled. Making it explicitly opt-in — `deploy_composer = false` by default — is the single most cost-aware decision in the framework.
- The DAG factory generates all five DAGs from a short YAML file. Adding a new entity is four lines of config; no Python, no orchestration code review, no DAG to hand-maintain.
- `EntityDependencyChecker` with a `ShortCircuitOperator` is how JOIN preconditions work. No `sleep` loops, no deadlocks, no "wait 30 minutes then give up" anti-patterns.
- The same `system.yaml` is consumed by the DAG factory, the Terraform module, the reconciliation engine, and the FinOps dashboard. One version-controlled file is the specification for the entire pipeline's behaviour.

Chapter 10 — Infrastructure as Code: Terraform at System Scale

Why Terraform, and how the framework organises it

Every resource in the pipeline — GCS bucket, Pub/Sub topic, BigQuery dataset, IAM binding, Composer environment — is declared in Terraform. The reference repository uses Terraform 1.0+ with the Google and Google-Beta providers, pinned in `.terraform.lock.hcl` at version 5.45.2 at the time of writing.

The framework's organising principle is the **system**. A system is a logical container for one coherent pipeline — in the reference, the only system is called `generic`. A production enterprise might have several: `generic`, `payments`, `fraud`, `loans`. Each system gets its own subdirectory under `infrastructure/terraform/systems/`.

```
infrastructure/terraform/
├── main.tf                 # Shared provider, backend, common locals
├── variables.tf            # Common variables
├── security.tf             # IAM and KMS defaults
├── dataflow.tf             # Dataflow worker pool configs
└── systems/
    └── generic/
        ├── main.tf         # System-level resources
        ├── variables.tf
        ├── outputs.tf
        ├── env/
        │   └── dev.tfvars  # Per-environment tfvars
        ├── ingestion/
        │   ├── main.tf     # Landing, archive, error buckets; Pub/Sub
        │   └── variables.tf
        ├── transformation/
        │   ├── main.tf     # ODP, FDP, job_control datasets
        │   └── variables.tf
        └── orchestration/
            ├── main.tf     # Composer (opt-in), service accounts
            └── variables.tf
```

Each system-level subdirectory maps to a **unit** in the deployment model. The ingestion unit's Terraform lives in `systems/generic/ingestion/`. The transformation

unit's lives in `systems/generic/transformation/`. This keeps the blast radius small: an incorrect change to the transformation module cannot accidentally delete a landing bucket.

State, workspaces, and environments

Terraform state is held in a GCS bucket, typically named `<project>-terraform-state`, with per-system prefixes. The backend block is inherited from the top-level `main.tf`:

```
terraform {
  backend "gcs" {
    bucket = "gcp-pipeline-terraform-state"
    prefix = "systems/generic"
  }
  required_providers {
    google      = { source = "hashicorp/google",      version = "~> 5.45" }
    google-beta = { source = "hashicorp/google-beta", version = "~> 5.45" }
  }
}
```

Environments are managed by tfvars files, not by Terraform workspaces. A `dev.tfvars` might declare `env = "dev"`, `project = "my-org-dev"`, and smaller machine types for Composer. A `prod.tfvars` would declare the opposite. The CI pipeline applies the right tfvars based on the branch that is being deployed from.

I have seen both tfvars-based and workspace-based approaches in production. The framework's choice of tfvars is the right one in a compliance-heavy environment: it keeps environment configuration explicit in version control rather than implicit in Terraform state.

The ingestion module

The ingestion module provisions:

- **Landing bucket** (`generic-landing`) — where mainframe files arrive.
- **Archive bucket** (`generic-archive`) — where processed files move for retention.
- **Error bucket** (`generic-error-bucket`) — where rejected records are quarantined.
- **Pub/Sub topic** (`generic-file-notifications`) — receives GCS notifications from the landing bucket.
- **Pub/Sub subscription** (`generic-file-notifications-sub`) — consumed by the pubsub_trigger DAG.
- **Dead-letter topic** and subscription — for messages that fail to trigger.

Key details worth calling out:

- **Lifecycle rules.** The landing bucket has a rule that moves objects older than 7 days to `COLDLINE`, saving money on files the pipeline has already processed. The archive bucket transitions to `COLDLINE` at 90 days and to `ARCHIVE` at 365 days.

- **Retention policy.** The error bucket has a **locked retention policy** of 90 days. This is a compliance feature: even a compromised service account cannot delete rejected records within the retention window.
- **GCS notification config.** The notification is scoped to `OBJECT_FINALIZE` only, filtered to the `.ok` suffix. This prevents spurious triggers from partial uploads.
- **Customer-managed KMS keys.** Optional. If you set `use_cmek = true`, every bucket and topic is encrypted with a key you control. The module creates the key and grants the correct GCS and Pub/Sub service agents access.

The transformation module

The transformation module provisions BigQuery datasets:

- `odp_<system>` — Original Data Product. One table per entity, owned by the ingestion service account.
- `fdp_<system>` — Foundation Data Product. Written by dbt through the transformation service account.
- `marts_<system>` and `analytics_<system>` — secondary datasets for the non-FDP models.
- `job_control` — shared across all systems; holds `pipeline_runs`, `audit_events`, `finops_usage`, and `data_quality_scores`.

Dataset-level access is tightly controlled. The ingestion service account has writer rights on `odp_*` and no rights on `fdp_*`. The transformation service account has reader rights on `odp_*` and writer rights on `fdp_*` and `marts_*`. The analyst group has reader rights on `marts_*` and `analytics_*` only.

This separation prevents the worst class of data incident: an ingestion bug that accidentally writes into the FDP and corrupts a business-facing table.

The orchestration module

The orchestration module is the most complex because it is the most optional. It provisions:

- A **Cloud Composer 2** environment, but only if `var.deploy_composer = true`.
- A **Composer service account** with narrow IAM bindings: Dataflow Developer, BigQuery Data Editor on `job_control` only, Storage Object Viewer on the landing bucket.
- A **DAGs GCS bucket** configured as the Composer environment's DAG source.
- A **sync job** that copies `dags/` from the repository to the Composer bucket on deploy.

When `deploy_composer = false`, the module provisions just the service account and the DAGs bucket, which can be re-targeted at a hand-rolled Airflow or substituted for Cloud Scheduler + Cloud Functions triggers.

The module also takes care of one often-missed detail: **the Composer worker's egress to the Pub/Sub subscription**. Private Composer environments need VPC connector configuration and a peered VPC for Pub/Sub to reach the workers. The module stitches this together automatically.

Security, KMS, and service accounts

`security.tf` at the top level defines the IAM primitives that every system inherits:

- A **per-unit service account** for ingestion, transformation, and orchestration. Unit-scoped roles; no shared accounts.
- A **dedicated KMS key ring** per environment, with separate keys for GCS and Pub/Sub.
- An **audit-log sink** that routes Cloud Audit Logs for the pipeline projects into a central logging project. Without this, compliance teams cannot correlate pipeline activity with broader access logs.

A small but valuable touch: the module attaches **resource labels** to every resource it creates, keyed on `system`, `unit`, `environment`, and `owner`. These labels flow into GCP billing exports and make the FinOps dashboards in Chapter 11 possible. Without consistent labels, cost attribution is guesswork.

Modules, composition, and reuse

The framework's Terraform is module-heavy. The ingestion module is itself composed of sub-modules for bucket creation and Pub/Sub plumbing. This allows a new system (say, `fraud`) to reuse the ingestion module with one line:

```
module "fraud_ingestion" {
  source = "../../ingestion"
  system_id = "fraud"
  landing_bucket_name = "fraud-landing"
  notification_filter = ".ok"
  use_cmek = true
}
```

The drawback is complexity: a new engineer joining the team has to navigate four levels of Terraform to understand the resource graph. The framework mitigates this with detailed `README.md` files in each module and a `docs/infrastructure.md` that walks the whole graph.

Review: what works, what could be better

Strengths:

- **Unit-aligned modules.** Terraform organisation mirrors the deployment model, which keeps mental models aligned.
- **Labels are consistent.** Every resource carries the same label vocabulary, which powers everything downstream.
- **Composer is explicitly opt-in.** A rare and valuable restraint.
- **IAM is least-privilege by default.** Each service account has only what it needs.

Weaknesses:

- **No Terraform tests.** `terraform validate` runs in CI, but there is no `terratest` suite that stands up and tears down a real environment. For a reference implementation, a smoke-test harness would be valuable.
- **Secrets in plain-text tfvars.** The reference `dev.tfvars` does not contain secrets, but there is no guardrail preventing a future engineer from adding one. A pre-commit hook that scans tfvars would help.
- **No policy-as-code.** There is no OPA or Sentinel layer enforcing constraints (e.g. "no dataset without a retention policy"). Large enterprises will add this themselves, but a stub would be a nice touch.

In Chapter 11 we turn to the runtime story that ties all of this together — observability, FinOps, and governance.

Key takeaways

- Terraform organisation mirrors the three-unit deployment model. The ingestion module cannot accidentally delete a transformation dataset because they live in separate Terraform roots.
- Labels on every resource are not cosmetic — they are what make FinOps attribution possible. Without consistent `pipeline-system`, `pipeline-unit`, and `pipeline-entity` labels, cost analysis is guesswork.
- IAM is least-privilege per unit: the ingestion service account has no rights on the FDP dataset, and the transformation account has no rights on the landing bucket. The blast radius of a compromised account is bounded by design.
- The error bucket has a locked retention policy. Even a compromised account cannot delete rejected records within the retention window. This is a compliance feature, not a convenience.

Chapter 11 — Observability, FinOps, and Data Governance

Observability is not three dashboards

A lot of teams mistake "observability" for "three Cloud Monitoring dashboards and a Slack alert". The framework takes a broader view. Observability means being able to answer, about any point in the pipeline's history: *what happened, why, how much did it cost, who was affected, and can I reproduce it?*

Answering those questions requires five things, all of which the framework ships:

- **Structured logging** with consistent context fields.
- **Metrics** with consistent labels.
- **Traces** that span service boundaries.
- **Audit events** with business-meaningful outcomes.
- **Cost records** tied to the same identifiers.

Structured logging

Every log line produced by framework code is a JSON object. At minimum it contains:

```
{
  "ts": "2026-04-17T09:14:02.381Z",
  "level": "INFO",
  "logger": "gcp_pipeline_beam.transforms.SchemaValidateRecordDoFn",
  "message": "Validated 50000 records",
  "run_id": "20260417T091400Z-7f3a",
  "system_id": "generic",
  "entity": "customers",
  "extract_date": "2026-04-17",
  "environment": "prod",
  "pid": 42,
  "trace_id": "a5b4c3d2e1f0...",
  "span_id": "1234567890abcdef"
}
```

Structured logs are cheap to produce and enormously more useful than free-text. Cloud Logging auto-indexes them; a filter like `jsonPayload.run_id="20260417T091400Z-7f3a"` recovers every event from a single run across every service it touched. A filter

like `jsonPayload.entity="customers" severity>=ERROR` gives the on-call a usable triage view.

`gcp-pipeline-core.utilities.configure_structured_logging()` is called once at service start; from then on, `logging.getLogger(__name__).info(...)` produces JSON. A `LogContext` context manager adds ambient fields so code in the middle of a pipeline does not have to pass them explicitly.

Metrics and the `MetricsCollector`

`MetricsCollector` is a thin abstraction over Cloud Monitoring custom metrics. It exposes four metric types:

- **Counter** — monotonically increasing. Records processed, bytes written.
- **Gauge** — point-in-time. Worker count, queue depth.
- **Histogram** — distributions. Record latency, field lengths.
- **Timer** — convenience over histogram for timing code blocks.

Every metric carries a standard label set: `system_id`, `entity`, `environment`, `run_id`, plus a handful of metric-specific labels. The collector buffers and flushes on a timer (default 30 s) or on process exit, whichever comes first.

```
from gcp_pipeline_core.monitoring import MetricsCollector

metrics = MetricsCollector(system_id="generic", entity="customers")
metrics.counter("records_processed").inc(len(batch))
with metrics.timer("bq_insert_latency").time():
    bq.insert_rows_json(table, batch)
```

One thing that makes this layer work: a **custom metric descriptor** is created on first use rather than declared ahead of time. New metrics appear in Cloud Monitoring automatically. The trade-off is that a typo creates a new metric rather than an error; the framework mitigates this with a lint check that compares metric names against a central allow-list in CI.

Observability Manager and health checks

`ObservabilityManager` is the composition root. It wires up a `MetricsCollector`, a `HealthChecker`, an `AlertManager`, and optionally an OTEL tracer into a single object that pipeline code can pass around or inject.

`HealthChecker` runs a small number of checks on a schedule:

- **Error rate.** Fails if the last 5 minutes of pipeline runs exceeded 2% error rate.
- **Queue depth.** Fails if the Pub/Sub subscription backlog exceeds 10,000 messages.
- **Memory headroom.** Fails if a Dataflow worker is above 85% memory.
- **Processing time.** Fails if median record latency exceeds the configured SLA.

Each check returns a `HealthResult` with a status (OK, DEGRADED, FAILED), a message, and a recovery hint. Results are exposed on an HTTP endpoint (for Cloud Monitoring to scrape) and written into `job_control.health_checks`.

AlertManager and multi-backend alerts

`AlertManager` delivers alerts to multiple backends in parallel:

- `LoggingAlertBackend` — always-on baseline; writes alerts to Cloud Logging with structured JSON. You get this even if you configure nothing else.
- `CloudMonitoringBackend` — converts alerts to Cloud Monitoring notification channels, so you can drive dashboards and uptime checks from the same signal.
- `DatadogAlertBackend` — for organisations already on Datadog.
- `SlackAlertBackend` — the most common operational channel; most teams have this active in every environment.
- `DynatraceAlertBackend` — for enterprises running Dynatrace as their APM layer.
- `ServiceNowAlertBackend` — opens an incident ticket directly from an alert, without a human in the loop.

PagerDuty support is planned for a future release; today the on-call path is Slack with a webhook into PagerDuty if your team has one configured.

Backends are plug-ins: each implements a `Backend.deliver(alert)` method. A configuration file decides which backends are active per environment. Dev typically gets `LoggingAlertBackend` and `SlackAlertBackend`; prod gets whichever subset your organisation runs.

The `Alert` object carries enough metadata to deduplicate. An alert with the same `dedupe_key` delivered twice within 10 minutes produces a single notification with a "retriggered" counter. This is the difference between a usable on-call rota and one where every engineer quits after three months.

OpenTelemetry tracing, gracefully degraded

Distributed tracing is optional. If `opentelemetry-api` and `opentelemetry-sdk` are installed, the framework:

- Creates a tracer per logger name.
- Emits spans around DoFn processing, BigQuery inserts, GCS reads, and Pub/Sub publishes.
- Propagates trace context through Pub/Sub message attributes.
- Exports to Cloud Trace via the Google Cloud exporter.

If the SDK is absent, `OTELConfig.configure()` installs a no-op tracer and logs nothing. Pipeline code that calls `tracer.start_as_current_span()` does not crash; it just does nothing useful.

This graceful-degradation pattern is one of the framework's best idioms. It lets teams adopt tracing incrementally rather than as a forced dependency.

FinOps: cost as a metric

Cost is treated like any other metric. Three trackers in `gcp-pipeline-core.finops`:

- `BigQueryCostTracker` wraps query execution, reads `query.total_bytes_billed`, multiplies by the on-demand rate (or looks up a flat-rate reservation slot cost), and writes a record to `job_control.finops_usage`.
- `CloudStorageCostTracker` observes read/write ops and aggregates them per run, using class-specific pricing (Standard, Nearline, Coldline, Archive).
- `PubSubCostTracker` records publish and pull counts per run.

The resulting `finops_usage` table supports queries like:

```
SELECT entity, SUM(cost_usd) AS cost
FROM job_control.finops_usage
WHERE run_date BETWEEN DATE_SUB(CURRENT_DATE(), INTERVAL 7 DAY) AND CURRENT_DATE()
GROUP BY entity
ORDER BY cost DESC
```

…which answers, weekly, the question "which entity is my most expensive?" Without this, cost optimisation is guesswork. With it, the team can prioritise the right transforms for partitioning, clustering, or decommissioning.

The framework also emits **GCP labels** on every resource and job: `pipeline-system=generic`, `pipeline-unit=ingestion`, `pipeline-entity=customers`, `pipeline-run-id=...`. These labels flow into the GCP billing export and make it possible to correlate internal cost records with the billed amount.

Audit trail and reconciliation engine

`AuditTrail` we met in Chapter 6. `ReconciliationEngine` is its companion: a service that, given a `run_id`, joins the audit events, the valid/invalid counts, the HDR/TRL envelope counts, and the BigQuery row count, and produces a consolidated reconciliation record.

A reconciliation fails if any of the following hold:

- HDR/TRL expected count does not match valid + invalid.
- Valid count does not match BigQuery row count for the partition.
- Any required field in the audit trail is missing.

Failed reconciliations produce an alert **and** a row in `job_control.reconciliation_failures`. The status DAG from Chapter 9 consumes this table and refuses to mark a run successful if it has an open reconciliation failure.

This closes the loop: no run is considered "green" just because Dataflow returned success. The data has to actually match the envelope.

Data governance: quality, quarantine, deletion

Three subsystems from the core come together here.

Data quality. `DataQualityChecker` runs on a sampled subset of each FDP build. It computes null rates, distinct value counts, primary-key uniqueness, and a derived grade (A to F) from a weighted score. The grade is written to

`job_control.data_quality_scores` and attached to the audit trail. A grade below B raises an alert.

Quarantine. `QuarantineManager` administers the four-level quarantine workflow — REVIEW, HOLD, DELETE, ARCHIVE — for both rejected records and intentional data deletions. Each transition requires an approver identified by email; the approver chain is configurable.

Safe deletion. `SafeDataDeletion` enforces the workflow: `DELETE` requires a prior `HOLD`, which requires a prior `REVIEW`. Attempts to delete without the chain are rejected, logged, and alerted. Tombstones are retained so the audit trail can show what was deleted, when, and by whom.

A complete lineage example

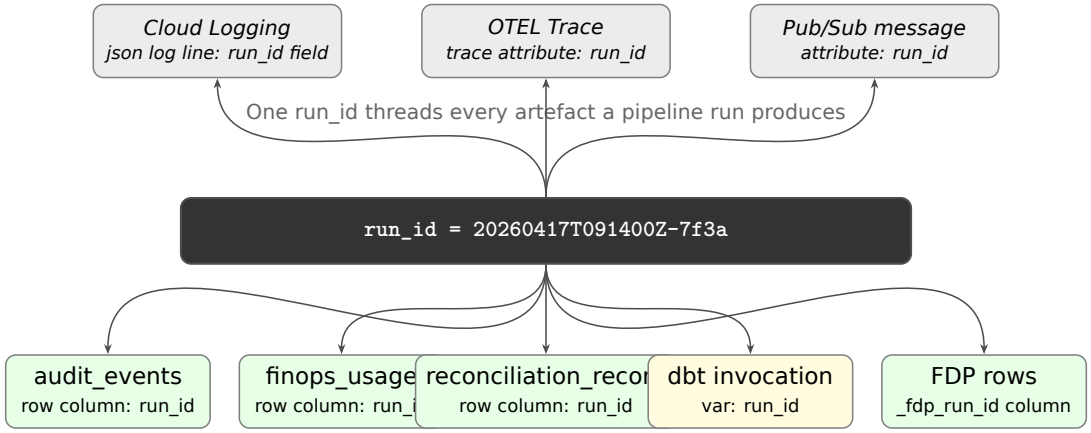

Figure 5: A single run_id threads every artefact a pipeline run produces.

Putting it together, imagine a dashboard showing a suspicious revenue figure on Thursday morning. A data steward clicks into the dashboard, finds the FDP row, and reads `_fdp_run_id`. One click into the audit UI shows:

- The dbt invocation that produced the row.
- The Airflow DAG run that triggered the dbt.
- The upstream ingestion DAG runs for both customers and accounts.
- The Dataflow job IDs for each ingestion.
- The HDR/TRL envelopes, expected vs actual counts.
- The reconciliation record.
- The data-quality grade.
- The total cost of the run.
- The logs, filtered to that `run_id`.
- The trace, with spans across every service.
- Any alerts raised.
- Any errors quarantined.

That is what "observability" means in this book. It is not three dashboards; it is a single thread of identity — `run_id` — that ties thirty separate artefacts together.

Non-functional requirements, in one table

Observability is one facet of a larger concern: the **non-functional requirements** (NFRs) the framework addresses by design. For anyone producing an architecture document, here is the whole list in one place, with concrete targets the framework ships with and the mechanisms that back them.

NFR	Target (ship default)	Mechanism
Throughput	≥ 10M rows/hour per entity on `n1-standard-2` workers	Dataflow autoscaling; `setup()` hot path
Ingestion latency	< 5 min from file landing to ODP row	Pub/Sub trigger + Dataflow Flex Template cold start
Transformation latency	< 10 min MAP; < 20 min JOIN (medium entity)	Incremental dbt; JOIN preconditions gated
Availability	99.5% monthly per unit; 99.0% end-to-end	Independent units; retries; idempotence; DLQ
Recovery time objective (RTO)	< 1 hour from a catastrophic failure	Checkpoint restart; rerun-from-audit
Recovery point objective (RPO)	Zero data loss; at-least-once	Error quarantine + replay; reconciliation enforces
Idempotence	All entry points idempotent	`run_id`-keyed outputs; dedup in DLQ consumer
Reliability/retries	5 attempts, expo backoff + jitter, 2–60 s	`RetryPolicy`; integration-only; never validation
Security — auth	No long-lived credentials in CI or runtime	Workload Identity Federation; metadata server tokens
Security — authz	Least privilege per unit	Unit-scoped service accounts; dataset-level roles
Security — encryption	At-rest and in-transit by default; CMEK optional	GCS CMEK, Pub/Sub CMEK, BQ CMEK; TLS by Google
Security — PII	Hashed at FDP boundary; never in logs	`apply_pii_masking`; structured-log redaction filter
Privacy / deletion	GDPR/CCPA aligned four-stage delete	`SafeDataDeletion` + `QuarantineManager`

NFR	Target (ship default)	Mechanism
Audit	Every row traceable to run, job, file, approver	`audit_events` + `reconciliation_record` + `run_id` everywhere
Compliance	SOX, SOC 2, HIPAA, PCI-adjacent patterns	Locked retention, KMS, immutable audit, approved deletion
Cost observability	Per-run, per-entity, per-query attribution	`finops_usage`; GCP labels; cost trackers
Cost control	Opt-in Composer; lifecycle rules; partitions	`deploy_composer=false` default; 90-day COLDLINE
Scalability	Multi-system (generic, payments, fraud)	System-scoped Terraform; no hard-coded names
Maintainability	≤ 20 mins for a new entity	DAG factory; `EntitySchema`; macros
Testability	≥ 80% line coverage; ≤ 5 min CI	Fakes; DirectRunner; path-filtered CI
Observability — logs	Structured JSON; `run_id` indexable	`configure_structured_logging`; Cloud Logging
Observability — metrics	Per-run counters/gauges/histograms	`MetricsCollector` + Cloud Monitoring
Observability — traces	Optional OTEL spans across services	`OTELConfig`; Cloud Trace export
Observability — alerts	Dedup-safe multi-backend alerts	`AlertManager` (Logging, CloudMonitoring, Datadog, Slack, Dynatrace, ServiceNow)
Data quality	A–F grade per run; anomaly detection	`DataQualityChecker` + `AnomalyDetector`

When you are writing an architecture doc or a security submission, that table is the one-page answer.

Hooking into your existing observability stack

A common mistake teams make when they adopt this framework is treating Cloud Monitoring, Cloud Logging, and Cloud Trace as if they were a finished destination. They are not. They are local stores. Most enterprises already run a centralised observability platform — Datadog, Dynatrace, Splunk, Grafana Cloud, New Relic, Sumo Logic — and the data engineers' on-call rotation is on whichever paging system the rest of the company uses. The framework's job is to *get the signal out cleanly*; the destination is a deployment choice, not a framework choice.

The pattern is the same regardless of backend. Three integration seams matter:

- **Logs.** Structured JSON arrives in Cloud Logging by default. From there you have three export options: a Pub/Sub-driven log sink (the standard pattern for Datadog, Splunk, Sumo Logic), a Cloud Storage sink with batch ingestion (cheaper, slower — fine for compliance archives), or a direct write from inside the workload using the vendor's SDK (best avoided — couples the pipeline to the vendor). Pub/Sub sinks are almost always the right answer; they keep the framework backend-agnostic, the latency is acceptable, and the cost is dominated by Cloud Logging's own ingestion fees, not the sink.
- **Metrics.** The same `MetricsCollector` that writes to Cloud Monitoring also speaks OTEL. The OTEL exporter targets any OTLP-compatible endpoint — which means Grafana Cloud, Datadog (via the OTLP receiver), New Relic, Dynatrace, and Honeycomb, with no framework change. Set `OTEL_EXPORTER_OTLP_ENDPOINT` and `OTEL_EXPORTER_OTLP_HEADERS` and the metrics flow. For Prometheus-shaped backends (Grafana on-prem, VictoriaMetrics, Mimir), the same OTEL pipeline can be terminated at an OTel collector that re-exports as Prometheus remote write.
- **Alerts.** The `AlertManager` abstraction is the framework's deliberate answer to "which paging system do you target?". It ships with backends for Cloud Logging, Cloud Monitoring, Datadog, Slack, Dynatrace, and ServiceNow; PagerDuty and Opsgenie are on the near-term roadmap and are straightforward to add — they are HTTPS-with-a-shared-secret webhooks. The point of the abstraction is that alerts are deduplicated, run-id-correlated, and severity-classified *before* they reach the backend. A Datadog event and a Slack message are produced from the same `AlertManager.fire(...)` call; the backend is configured per environment, not per call site.

Two practical patterns are worth knowing.

The Datadog pattern. A Pub/Sub-to-Datadog Cloud Function (Datadog publishes a reference implementation) consumes the log sink, parses `run_id` and `entity` as tags, and emits to Datadog Logs. The same `OTEL_EXPORTER_OTLP_ENDPOINT` points at `https://otlp.datadoghq.com/v1/metrics` with the API key in `OTEL_EXPORTER_OTLP_HEADERS`. Alerts are wired via `AlertManager`'s Datadog backend, which posts events to the Events API with `dd_source=gcp_pipeline_framework` so they group cleanly with infrastructure alerts. Dashboards in Datadog get the run-id as a high-cardinality tag — which Datadog will rate-limit on free tier, so plan to summarise per-entity before exporting if you are on a small contract.

The Grafana Cloud / Splunk pattern. Grafana Cloud takes OTLP directly for traces and metrics; logs go via Pub/Sub → an OTel collector running on Cloud Run → Grafana Loki. Splunk takes either the Pub/Sub-to-HEC pattern (Splunk Add-on for GCP) or direct HEC writes from a Cloud Function. The framework is unchanged in either case; the difference is in the sink config and in which alerts go to which Splunk index.

What you should *not* do is reach for the vendor's GCP-native log-shipping agent on every Dataflow worker and Composer node. It is technically possible and operationally a nightmare — agent version drift, IAM scope explosion, and the agent dies before the workload it was meant to instrument. The Pub/Sub-sink-to-collector pattern is boring and works.

Planning NFRs as a team, not as a checklist

The table earlier in this chapter is the *what* — the targets the framework ships with. The *how* — how you decide which NFRs you actually need for your specific data product — is a team activity, and it is the one piece of architecture work I have seen go wrong more often than any other. Engineers reach for the table and tick boxes. The result is over-engineered pipelines with availability targets nobody needed and observability budgets nobody can justify.

A better method, which I have run a dozen times: a 90-minute NFR planning session, run before any pipeline code is written, with the engineering lead, the product owner, an SRE, and someone from security. The agenda is four conversations.

First, business latency. When does the downstream business actually need this data? The answer is rarely "real time". It is usually "before the 8 a.m. morning meeting" or "before the close-of-business reconciliation". Pin that down to an hour-of-day deadline and the rest of the NFR table falls into place. A data product that needs to be available by 06:00 has a 04:00 ingestion target, which means the upstream extract has to land by 03:30, and a Dataflow cold start of ten minutes is comfortable. The same data product, if it needed sub-minute latency, would push you towards streaming and a fundamentally different cost profile. Get the latency target right and the architecture writes itself.

Second, blast radius. If this pipeline fails for a day, what stops? If the answer is "an internal dashboard becomes a day stale", availability of 99.0% monthly is fine. If the answer is "a regulator-facing report cannot be filed", you are in 99.9% territory and the cost roughly triples. The blast-radius conversation forces honesty about the consequence of downtime, which is the only honest input to an availability target.

Third, audit surface. Who needs to see what, retained for how long? Most regulators want seven-year retention on transaction-shaped data, one to three years on derived analytics, ninety days on engineering logs. These three numbers translate directly into BigQuery partition policies and Cloud Logging retention configuration. If you do not have this conversation up front, you discover the seven-year requirement during an audit and pay to backfill it from cold archives.

Fourth, on-call shape. Who is paged at 3 a.m., on what symptoms, with what runbook? The framework can produce alerts at every imaginable granularity; the question is which ones are worth waking a human for. The team should agree on at most five paging-grade alerts per pipeline: hard failure of the run, reconciliation mismatch above threshold, cost spike above threshold, schema drift, and SLA breach. Everything else is dashboard noise. Write this list before you write the alert rules; the alert rules then have to justify themselves against the list.

The output of this 90-minute session is a one-page NFR sheet for the pipeline — latency targets, availability target, retention targets, paging surface, cost cap. That sheet then anchors every subsequent decision, including which monitoring-tool integration you actually need to wire up.

A FinOps reality check

The framework ships first-class cost tracking, but cost tracking is not the same as cost control. A short, honest list of the FinOps practices I have seen actually work — and the ones that turn out to be theatre.

Things that work. Per-entity labels on every Dataflow job, BigQuery query, and Composer environment, with a query that joins `finops_usage` to billing export on label and surfaces "top 10 entities by daily spend". A weekly cost-trend review with the engineering lead and the product owner, fifteen minutes, looking at the deltas. A budget alert on each project at 50%, 75%, 90%, 100% of monthly target, with the 100% alert paging the on-call. Lifecycle rules on GCS landing buckets that move to COLDLINE after 30 days and ARCHIVE after 90; do this once at provisioning, never think about it again. A standing rule that any new Dataflow job runs with `--max-workers` set; jobs that need more get reviewed.

Things that do not work. Daily cost dashboards nobody reads. Slack channels that fire every cost alert and become noise. Cost optimisation reviews scheduled monthly that get cancelled when the team is busy (so, always). Reactive optimisation after a finance team complaint; by then the budget is already spent and the muscle memory to prevent it next time is not built.

The pattern is the same across all of these: cost discipline is a habit, not a quarterly initiative. The framework makes the data trivial to surface; the team has to choose to look at it on a cadence that catches problems while they are still small.

Review: what works, what could be better

Strengths:

- **Run-ID-driven lineage.** One identifier unifies every log, metric, audit event, and cost record.
- **First-class cost tracking.** Almost unique among open frameworks.
- **Graceful OTEL degradation.** Adoption is painless.
- **Compliance-ready.** Deletion workflow and reconciliation engine satisfy most regulators out of the box.
- **NFRs in one table.** Architects get a single artefact; no hunting across docs.

Weaknesses:

- **The metrics allow-list is maintained manually.** A typo is caught only if the lint runs. Catalog automation would help.
- **Cost data is on-demand priced by default.** Teams on flat-rate slot reservations need to override the rate. The configuration path is present but not obvious.
- **The quarantine UI is Slack-notification-driven.** There is no first-class admin UI. A small Streamlit app in a future version would be welcome.

Chapter 12 zooms in on testing — how the framework verifies its own correctness and how you can extend it.

Key takeaways

- A single `run_id` threads logs, metrics, audit events, cost records, dbt invocations, and trace spans together. Observability is not three dashboards — it is one identity that ties thirty artefacts into a single click.
- Cost tracking lives in `finops_usage` at per-run, per-entity, per-query granularity. The question ”which entity is my most expensive?” takes one SQL query, not a week of billing-export archaeology.
- The NFR table in this chapter is the one-page architecture-document answer: throughput, latency, availability, RTO/RPO, security, compliance, and cost targets, with the mechanism behind each.
- Reconciliation is mandatory: a run is not green just because Dataflow returned success. The data has to match the HDR/TRL envelope count before the status DAG marks it complete.

Chapter 12 — Testing Strategy: 763 Tests Across Six Components

The number that matters

`gcp-pipeline-framework` ships with 763 unit tests at the time of writing, distributed as follows:

- `gcp-pipeline-core`: 219 tests
- `gcp-pipeline-beam`: 359 tests
- `gcp-pipeline-orchestration`: 58 tests
- `gcp-pipeline-tester`: 101 tests
- `original-data-to-bigqueryload` reference deployment: 26 tests

That is more than most data-engineering codebases I have seen, and it is the single biggest reason to trust the framework in production. Tests do not just verify behaviour; they also document intent. A new contributor reading `tests/unit/test_hdr_trl_parser.py` learns more about what `HDRTRLParser` is supposed to do in twenty minutes than they would by reading the source for an hour.

Tooling

The test stack is, deliberately, the boring default:

- `pytest` 7.x
- `pytest-mock` for `MagicMock`-based patching
- `pytest-cov` for coverage reporting
- `freezegun` for time-sensitive tests
- `responses` for HTTP mocking (used for Datadog, Dynatrace, and ServiceNow backends)
- `apache-beam[test]` for Beam-specific test runners

There is no exotic property-based testing, no mutation testing, no Hypothesis. The framework's author made a defensible choice: keep the tooling small enough that any Python engineer can contribute on day one.

The `gcp-pipeline-tester` library

`gcp-pipeline-tester` deserves its own subsection because it is one of the framework's quiet super-powers. It exposes:

- **`PipelineTestCase`** — a base test class with set-up/tear-down for run IDs, in-memory audit trail, and a fake metrics sink.
- **Fake clients.** `FakeBigQueryClient`, `FakeGCSClient`, `FakePubSubClient`, all with the same interface as the real wrappers in `gcp-pipeline-core.clients`. They store data in memory and expose introspection methods like `inserted_rows(table)` for assertions.
- **Fixture factories.** Given an `EntitySchema`, generate N realistic-looking rows with the right types, the right value distributions, and configurable null rates.
- **Snapshot helpers.** Compare a Beam PCollection's contents to a JSONL fixture with helpful diffs.
- **`TimeTraveller`.** A wrapper over `freezegun` with conveniences like `advance_to_next_business_day()` for testing scheduled DAGs.

Test code with this library tends to look like this:

```
class TestSchemaValidation(PipelineTestCase):
    def test_valid_records_pass(self):
        rows = self.factory.rows_for(CustomerSchema, n=100, null_rate=0)
        with TestPipeline() as p:
            valid, invalid = (
                p | beam.Create(rows)
                  | beam.ParDo(SchemaValidateRecordDoFn(CustomerSchema))
                       .with_outputs("invalid", main="valid")
            )
            assert_that(valid, equal_to(rows), label="valid")
            assert_that(invalid, equal_to([]), label="invalid")
```

Notice how little ceremony there is. Set-up is in the base class. Schemas drive fixture generation. Beam's `assert_that` does the comparison. The test reads as the spec it is.

Patterns by layer

Core. Tests use the in-memory fakes. Audit, FinOps, error handling, deletion, and quality are all covered. The test suite asserts not only happy paths but also the negative space: that retries do not happen on validation errors, that cost trackers do not double-count, that deletion without a hold raises.

Beam. Tests use Beam's `DirectRunner`. Each transform is exercised against a fixture batch, and side outputs are asserted independently. Performance tests are excluded from CI but live in `tests/performance/` for ad-hoc runs.

Orchestration. Tests do not spin up Airflow in CI. They import DAG modules and assert the resulting `DAG` object's structure: task names, dependencies, `default_args`, retry policies, SLA settings. This is fast (sub-second per test) and catches the kind of regressions that "lint your DAGs" aspires to.

But fast structural tests are not enough on their own. If you change a sensor's pull logic or a task's XCom payload, you need to actually run the DAG somewhere before you push. That is where **local Airflow testing** comes in.

Local Airflow testing: the missing piece

Here is one of those truths nobody puts on the first page of the Airflow docs: **you do not need Composer to test Airflow**. You need Airflow running on your laptop. That is it.

The framework supports three local testing modes, in increasing order of investment:

Mode A — Single-task Python invocation

The fastest way to exercise a task. No Airflow server, no scheduler, no database. You just call the task's `execute()` directly with a fake context:

```
from airflow.utils.context import Context
from airflow.models import DagBag

def test_ingestion_launch_task():
    bag = DagBag(dag_folder="deployments/data-pipeline-orchestrator/dags",
                 include_examples=False)
    dag = bag.get_dag("ingestion_customers")
    task = dag.get_task("run_dataflow")

    ctx = Context(
        dag=dag, task=task,
        run_id="manual__2026-04-17T09:00:00+00:00",
        ds="2026-04-17",
        dag_run=None,
        params={"input_file": "gs://landing/customers.csv"},
    )
    task.execute(ctx)
```

This catches typos in parameter rendering, bad Jinja templates, and anything else that explodes at task render time. It runs in CI in under a second.

Mode B — `airflow dags test` on a local SQLite backend

Airflow ships with a `dags test` subcommand that runs a full DAG execution against a local SQLite metadata database, in-process, without a scheduler. No web server, no celery, no docker. You get real execution, real XCom, real logs — just single-threaded and non-scheduled.

```
# one-time
export AIRFLOW_HOME=$PWD/.airflow
pip install 'apache-airflow==2.8.*' \
    --constraint "https://raw.githubusercontent.com/apache/airflow/constraints-
    ↪  2.8.0/constraints-3.11.txt"
airflow db init
export AIRFLOW__CORE__DAGS_FOLDER=$PWD/deployments/data-pipeline-orchestrator/dags
export AIRFLOW__CORE__LOAD_EXAMPLES=False
```

```
# per run
airflow dags test ingestion_customers 2026-04-17
```

Any task that touches GCP resources would fail here — so you point them at fakes. The framework's `gcp-pipeline-tester` package ships `FakeGCSClient`, `FakeBigQueryClient`, and `FakePubSubClient`, all injectable via an environment variable. A `make test-dag DAG=ingestion_customers` target wraps this into one command.

Mode C — Docker Compose with the full Airflow stack

For DAGs where the interaction between scheduler, executor, and webserver actually matters (for example, pool contention tests, SLA tests, or concurrency tests), the framework ships a Compose file at `scripts/airflow-local/docker-compose.yml`:

```
version: "3.8"

x-airflow-common: &airflow-common
  image: apache/airflow:2.8.1-python3.11
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__CORE__LOAD_EXAMPLES: "False"
    AIRFLOW__CORE__SQL_ALCHEMY_CONN:
↪   postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CORE__DAGS_FOLDER: /opt/airflow/dags
    # Point Google clients at the emulator suite (GCS, Pub/Sub, BigQuery emulators)
    STORAGE_EMULATOR_HOST: http://gcs-fake:4443
    PUBSUB_EMULATOR_HOST: pubsub-fake:8085
    BIGQUERY_EMULATOR_HOST: http://bq-fake:9050
  volumes:
    - ../../deployments/data-pipeline-orchestrator/dags:/opt/airflow/dags:ro
    - ./plugins:/opt/airflow/plugins:ro

services:
  postgres:
    image: postgres:15
    environment: { POSTGRES_USER: airflow, POSTGRES_PASSWORD: airflow,
↪   POSTGRES_DB: airflow }

  airflow-init:
    <<: *airflow-common
    entrypoint: ["bash", "-c", "airflow db migrate && airflow users create
↪   --username admin --firstname admin --lastname admin --role Admin --email
↪   admin@example.com --password admin"]
    depends_on: [postgres]

  airflow-webserver:
    <<: *airflow-common
    command: ["webserver"]
    ports: ["8080:8080"]
    depends_on: [airflow-init]

  airflow-scheduler:
```

```
    <<: *airflow-common
    command: ["scheduler"]
    depends_on: [airflow-init]

  gcs-fake:
    image: fsouza/fake-gcs-server:latest
    command: ["-scheme", "http", "-port", "4443"]
    ports: ["4443:4443"]

  pubsub-fake:
    image: gcr.io/google.com/cloudsdktool/google-cloud-cli:latest
    command: ["gcloud", "beta", "emulators", "pubsub", "start",
↪   "--host-port=0.0.0.0:8085"]
    ports: ["8085:8085"]

  bq-fake:
    image: ghcr.io/goccy/bigquery-emulator:latest
    command: ["--project=local", "--port=9050"]
    ports: ["9050:9050"]
```

Bring it up with one command:

```
scripts/airflow-local/up.sh
# visit http://localhost:8080 (admin / admin)
# airflow-scheduler picks up dags automatically
```

Bring it down:

```
scripts/airflow-local/down.sh
```

The emulators (fake GCS, Pub/Sub, BigQuery) mean your DAGs can actually launch, produce XComs, write rows, and trigger downstream DAGs, **without touching a real GCP project and without burning a penny**. The framework's clients auto-detect the emulator environment variables and switch backends, so no DAG-side code changes are needed.

The testing pyramid for Airflow

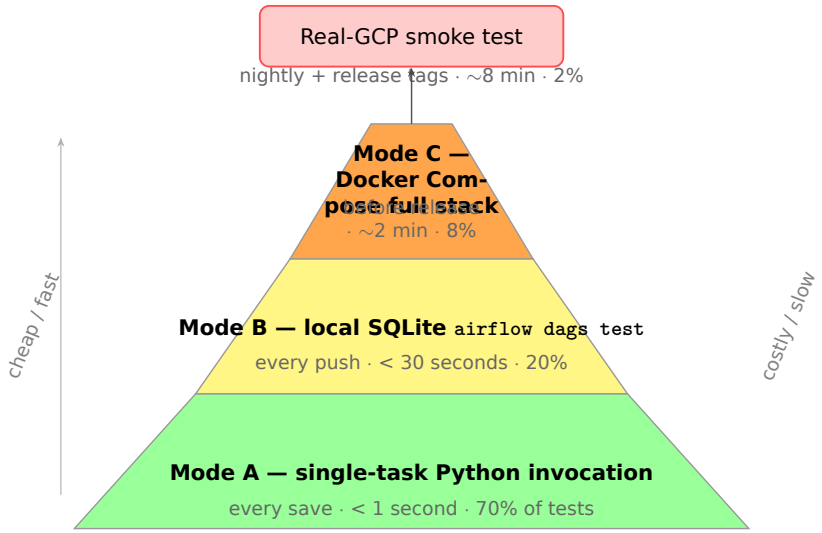

Figure 6: The framework's three-mode local Airflow testing pyramid.

To keep the trade-offs clear, the framework's recommended testing split is:

- **~70% structural tests** (DAG imports, dependency graphs, retry policies). Fast, CI-friendly, cheap.
- **~20% single-task executions** (Mode A). Catch Jinja and parameter issues.
- **~8% local full-DAG runs** (Mode B / Mode C). Catch integration issues.
- **~2% real-GCP smoke tests** (the `e2e_pipeline_test.sh` script). Catch the things only real services can break.

Running the full local stack on every PR is overkill. Running it *before* a release is not. A good rule: structural + single-task on PR, full local run on merge to `main`, real-GCP smoke on tag.

Where to put the local testing harness

The framework ships the harness under `scripts/airflow-local/`:

```
scripts/airflow-local/
├── docker-compose.yml        # the stack above
├── up.sh / down.sh           # lifecycle helpers
├── seed-pubsub.sh           # create topics / subscriptions in the emulator
├── seed-bq.sh                # create datasets and tables in the emulator
├── seed-gcs.sh               # upload fixture CSVs to the fake GCS
├── run-dag.sh <dag_id> <ds>  # wrap `airflow dags trigger` + wait
└── README.md
```

The `seed-*` scripts read from `test_data/` so fixtures are shared with the unit tests. One command (`scripts/airflow-local/seed-all.sh`) populates an empty stack with a realistic day's data.

Catching issues this harness finds that CI doesn't

Real examples, from real days:

- **Jinja template typos** that render only at task-execution time. Structural tests won't catch them.
- **XCom payload size limits.** Airflow silently truncates XComs above a threshold. Only real execution surfaces this.
- **Pool and concurrency issues.** Only the scheduler can show you a DAG that deadlocks on a pool.
- **Sensor backpressure.** A Pub/Sub pull sensor behaves differently against a topic with 10,000 queued messages than against an empty one.
- **Cross-DAG triggers.** `TriggerDagRunOperator` only really works with a scheduler running.
- **Import-time side effects.** The DAG factory pattern is vulnerable to import-time I/O. Local full-stack catches this instantly.

If you want one takeaway from this section: **wire up Mode B on day one**. It is twenty minutes of setup and it catches three classes of bugs you will otherwise only find in production.

Reference deployments. Tests cover the deployment-specific code: parameter validation, BigQuery target table naming, environment-specific behaviour. They do not exercise the framework libraries themselves; that is what the library tests are for.

The CI integration

Three GitHub Actions workflows make the test story real:

- **`test.yml`** — runs every library's test suite on every pull request. Failure blocks merge. Coverage is reported but not enforced as a percentage gate.
- **`publish-libraries.yml`** — runs tests, builds wheels, and publishes to PyPI on a tag push. A failing test prevents publication.
- **`deploy-generic.yml`** — path-filtered. A change in `gcp-pipeline-libraries/` triggers tests but not deployment. A change in `deployments/original-data-to-bigqueryload/` triggers tests, container builds, and Dataflow Flex Template publication.

Path filtering is one of those small CI niceties that pays for itself within a week. Without it, a one-line README change in a deployment kicks off a 25-minute deploy. With it, only relevant work runs.

What the framework does not test (yet)

Honesty time:

- **No automated end-to-end tests.** A real GCP environment is exercised by hand using `scripts/gcp/e2e_pipeline_test.sh`, which deploys, runs, and tears down. This is good — but it is not part of CI, and it relies on the engineer remembering to run it.

- **No load tests.** The Beam pipelines have not been benchmarked against a 100-million-row file in CI. Anecdotal numbers exist; there is no scheduled run.
- **No chaos tests.** The retry logic is exercised against deterministic mock failures. It has not been exercised against real flaky GCP behaviour.
- **dbt tests are dbt tests.** They run, and they are useful, but the macro Python helpers themselves have thinner coverage than the rest of the framework.

If I were prioritising the next quarter of test work, I would automate the e2e harness on a scheduled CI run against a dedicated test project, and I would add a small chaos suite that pokes the retry logic with simulated 503s.

Review

Strengths:

- **Volume and discipline.** 763 tests is a strong baseline.
- **Tester library reduces test friction.** Writing tests is fast.
- **CI path filtering is right.** Only relevant work runs.
- **Local Airflow harness.** Three modes (single-task, SQLite DAG test, Docker Compose full stack) cover every realistic DAG test need.

Weaknesses:

- **No automated e2e in CI.** Manual scripts only.
- **Coverage is reported, not gated.** A 0% test on a new module would still merge.
- **No chaos or load tests.** Production reliability is partly an article of faith.
- **Local stack is single-node.** Does not exercise Celery/Kubernetes executor behaviour; that still needs real Composer.

Chapter 13 covers end-to-end testing, distributed tracing, team test planning, and the practical story of testing locally against real GCP. Chapter 14 is the self-managed Kubernetes deployment guide for teams who cannot or should not use Composer. Chapter 15 then covers CI/CD and PyPI publishing.

Key takeaways

- 763 tests is a strong baseline, but 90% of them are fast unit tests. The tester library — fake clients, schema-driven fixtures, `PipelineTestCase` — is what makes writing those tests fast rather than tedious.
- Structural DAG tests (assert the DAG object's shape) are the cheapest and most important Airflow tests. They run in under a second and catch the regressions that "lint your DAGs" aspires to.
- There are three local Airflow testing modes: single-task Python invocation, `airflow dags test` on SQLite, and Docker Compose with emulators. Wire up Mode B on day one — it is twenty minutes of setup and catches three classes of bug.

- The framework does not gate on coverage percentage by design. A coverage gate produces tests that pad the number rather than tests that catch bugs.

Chapter 13 — End-to-End Testing, Tracing, and the Developer Workflow

Chapter 12 covered the **structural** side of testing — unit tests, the tester library, DAG-shape assertions, and the three local Airflow modes. That gets you about 90% of the way.

The last 10% is what this chapter is about. It is the hardest 10%:

- How do we test a pipeline **end-to-end**, in a real GCP project, without burning money?
- How do we **trace a single run** across every service it touched?
- How does a **team plan** its testing so nothing falls through the cracks?
- How does an individual developer, **at their desk**, test changes against real GCP without breaking production?

Each of those deserves a section.

Why end-to-end testing cannot be skipped

Unit tests prove each function behaves correctly in isolation. Local Airflow runs prove the DAG wires up. But neither proves that a file dropped into the real landing bucket produces the right FDP row, with the right audit trail, at the right cost, under the right IAM. Only an **end-to-end** test proves that.

This matters for three reasons:

- **IAM and network.** 90% of production failures I have seen come from permissions, not logic. A local harness cannot exercise real IAM.
- **Service behaviour.** Emulators approximate GCP; they do not reproduce every edge case (Pub/Sub ordering, Dataflow shuffle, BigQuery quotas). Some bugs only appear on the real platform.
- **Cost visibility.** The first real-GCP run is when you find out how much a pipeline actually costs. Better to know that on a 100-row fixture than on a 10-million-row production file.

The framework ships a scripted harness — `scripts/gcp/e2e_pipeline_test.sh` — that runs the full round-trip against a real GCP project in roughly 8 minutes for the `generic` system.

The E2E harness in detail

The harness does seven things, in order:

1. **Bootstrap** a fresh, dedicated project if it does not exist (`gcp-pipeline-e2e-<short-sha>`).
2. **Apply Terraform** for the system under test with a `test` environment tfvars.
3. **Seed** landing buckets with fixture files from `test_data/` (four entities, ~1,000 rows each, including deliberately malformed rows).
4. **Launch** the ingestion Dataflow templates via the factory-generated Airflow DAG (or directly via the CLI for a lighter variant).
5. **Trigger** the transformation DAG and wait for all FDP models to materialise.
6. **Verify** post-conditions:
   - Row counts match HDR/TRL envelopes.
   - Reconciliation records exist and are green.
   - Invalid rows landed in the error bucket.
   - Audit events for every step exist with the right `run_id`.
   - Data-quality grade is B or better.
   - FinOps usage records exist for BigQuery, GCS, and Pub/Sub.
7. **Tear down** — destroy the project unless `--keep` was set.

A passing run writes a one-line success record to `job_control.e2e_history` in a central observability project. A failing run writes the same record plus a structured failure report.

Running it

```
# One-shot: fresh project, full run, teardown
scripts/gcp/e2e_pipeline_test.sh generic

# Keep the project for investigation
scripts/gcp/e2e_pipeline_test.sh generic --keep

# Run against an existing test project (skips bootstrap)
scripts/gcp/e2e_pipeline_test.sh generic \
  --project=my-existing-e2e --skip-bootstrap

# Run only the verify phase (against a project already seeded)
scripts/gcp/e2e_pipeline_test.sh generic --phase=verify
```

How often to run it

My recommendation:

- On every merge to `main` — nightly at minimum, hourly if possible. Takes about $2 of GCP spend per run.
- On every release tag, across **all** supported systems (not just `generic`).

- On every major upgrade: Airflow minor bump, Composer upgrade, Beam upgrade.
- Ad hoc from a developer's laptop during local investigation of a flaky bug.

The cost of running the harness nightly for a year is roughly $700. The cost of one production incident it would have caught is usually higher than that.

Distributed tracing, in practice

`run_id` is the framework's primary identity. OpenTelemetry traces are its distributed nervous system.

What the framework instruments

If `opentelemetry-api` and `opentelemetry-sdk` are present, the framework emits spans for:

- **Beam DoFn** — one span per element for parse/validate/write, batched to keep volume sane.
- **BigQuery** — one span per query, with `bytes_billed` as a span attribute.
- **GCS** — one span per read/write, with byte count.
- **Pub/Sub** — one span per publish, one per pull, with message attributes.
- **Airflow operator** — one span per task execution, carrying DAG run and task IDs.
- **dbt** — one span per model, with rows-affected and cost.

Trace context propagates through **Pub/Sub message attributes** (`x-trace-id`, `x-span-id`) and through Airflow XCom, so a trace that begins when a file lands can continue through ingestion into transformation.

What a trace looks like

A typical trace, from file land to FDP refresh, has about 30 spans over 4 to 8 minutes. Opened in Cloud Trace, it draws a waterfall like this:

```
[file-landed]━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8 min
  [pubsub-publish]▌
  [pubsub-pull]▌
  [airflow-trigger-ingestion]▌
    [dataflow-submit]▌
      [dataflow-job]━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━  4 min
        [hdrtrl-parse]━━
        [csv-parse]━━━━━━━━━━━━━━━━
        [schema-validate]━━━━━━━━━━━━━━━━
        [bq-write]━━━━━━━━━━
        [gcs-write-errors]━
        [reconcile]━━
    [audit-record-end]▌
  [airflow-trigger-transformation]▌
    [dbt-run]━━━━━━━━━━━━━━━━━━━━
      [stg-customers]━━
```

```
      [stg-accounts]──
      [event-transaction-excess]──────
      [tests]─────
  [status-write]▌
  [fdp-trigger]▌
```

The point is not the picture; the point is that **one click on `run_id` gets you this view**. Not five dashboards; one trace.

Sampling and volume

Full tracing of every row-level DoFn span would produce absurd volumes. The framework does two things:

- **Per-element spans are sampled** at a configurable rate (default 1 in 10,000), with the first N and last N always kept so the start and tail of a batch are visible.
- **Critical spans** (submit, write, reconcile, trigger) are **always kept** regardless of sampling.

Trace cost lands at a few cents per million rows under the defaults.

Planning testing as a team

Individual developers running local tests is necessary but not sufficient. A team needs a **plan** — a written agreement about what is tested, by whom, with what tool, and when. The framework ships a reference plan in `docs/TEAM_TESTING_PLAN.md`. It looks roughly like this:

Level	Who	Tool	Frequency	Failure mode
Static analysis	Author	`ruff`, `mypy`, `qodana`	Every save	IDE hint
Unit tests	Author	`pytest`	Every save	Local fail
Structural DAG tests	Author	`pytest` + DagBag	Every push	PR blocked
Single-task tests	Author	`airflow tasks test`	Every push	PR blocked
Local SQLite DAG test	Author	`airflow dags test`	Every push (fast DAGs)	PR blocked
Local full-stack	Author or reviewer	Docker Compose	Before review	Review held
Real-GCP smoke (one entity)	Reviewer	`scripts/gcp/06_first_pipeline.sh`	Before merge	Merge blocked
E2E full system	CI	`scripts/gcp/e2e_nightly.sh`	Nightly + release tags	Release blocked
Load / chaos	SRE	custom + Locust	Quarterly	Capacity plan update
Security review	AppSec	Qodana + manual	Each release	Release blocked

Level	Who	Tool	Frequency	Failure mode
Cost regression	FinOps	`finops_usage` queries	Weekly	Alert
Data-quality regression	Data steward	`data_quality_scores`	Daily	Alert

Three things make this plan work in practice:

First, it is written. A testing plan that lives in someone's head is not a testing plan; it is a risk. The document is part of the repo, version-controlled, and updated whenever the plan changes.

Second, each level names an owner. "The author" is a role, not a team. When a test fails, the plan tells you who is expected to act.

Third, each level has a mechanical failure mode. Blocked PR, blocked merge, blocked release, alert. No level is "just notice". A level without an enforcement action tends to drift until it is ignored.

Local developer workflow against real GCP

Here is the part most frameworks skim over. A developer on a laptop, with changes they want to try, needs a safe and cheap way to run against real GCP — not the production one. The framework's convention:

The sandbox project pattern

Each developer gets a personal GCP project named `pipeline-sandbox-<username>`. The Terraform for the sandbox is the same module as production, with smaller resources and no Composer. A `scripts/dev/setup_sandbox.sh` script provisions it once per developer.

Authenticating natively

No service-account JSON keys. Developers authenticate as themselves via `gcloud auth application-default login` and `gcloud auth login`. The framework's clients pick up application default credentials automatically.

```
gcloud config set project pipeline-sandbox-jaruja
gcloud auth application-default login
gcloud auth login
export GCP_PROJECT_ID=pipeline-sandbox-jaruja
export ENVIRONMENT=sandbox
```

For tasks that require act-as-a-service-account (like running a Dataflow job under the same identity production would), the developer impersonates the sandbox SA rather than downloading a key:

```
gcloud config set auth/impersonate_service_account \
  pipeline-deployer@pipeline-sandbox-jaruja.iam.gserviceaccount.com
```

Running ingestion against the sandbox

Three increasingly realistic ways to run your changes:

1. DirectRunner against fixture data (no GCP network):

```
python deployments/original-data-to-bigqueryload/src/pipeline.py \
  --runner=DirectRunner \
  --input_file=test_data/customers.csv \
  --bq_table=odp_generic.customers_local \
  --error_bucket=file:///tmp/errors
```

2. DirectRunner against real GCS + real BigQuery (your credentials):

```
python deployments/original-data-to-bigqueryload/src/pipeline.py \
  --runner=DirectRunner \
  --input_file=gs://pipeline-sandbox-jaruja-landing/customers.csv \
  --bq_table=pipeline-sandbox-jaruja:odp_generic.customers
```

3. DataflowRunner against the sandbox project (fully production-shaped):

```
python deployments/original-data-to-bigqueryload/src/pipeline.py \
  --runner=DataflowRunner \
  --project=pipeline-sandbox-jaruja \
  --region=europe-west2 \
  --temp_location=gs://pipeline-sandbox-jaruja-temp \
  --input_file=gs://pipeline-sandbox-jaruja-landing/customers.csv \
  --bq_table=pipeline-sandbox-jaruja:odp_generic.customers
```

Option 3 is the most realistic and costs a few cents per run. That is the price of knowing, before you merge, that your change actually works in the real cloud.

Running the Airflow layer against the sandbox

Two patterns work well:

- **Local Airflow (Mode B) pointing at sandbox GCP.** Run `airflow dags test` locally, but with `GCP_PROJECT_ID=pipeline-sandbox-jaruja` and your ADC. The DAG tasks launch real Dataflow jobs into the sandbox.
- **Sync your DAGs to a sandbox Composer environment.** If the team can justify a shared sandbox Composer, everyone syncs their `dags/` to it via `scripts/dev/sync_sandbox_dags.sh`. DAGs are namespaced by developer.

I prefer the first pattern. Developers who need Composer-specific behaviour (KubernetesPodOperator, for example) use the second sparingly.

Running dbt against the sandbox

```
cd deployments/bigquery-to-mapped-product
dbt deps
dbt run --profiles-dir=./profiles --target=sandbox
dbt test --profiles-dir=./profiles --target=sandbox
```

The sandbox profile uses the developer's ADC and a sandbox dataset prefix (`fdp_generic_sandbox_jaruja`). Datasets are isolated per developer, preventing merge conflicts on shared tables.

Safety rails

A few conventions keep sandbox usage safe:

- **Budget alerts.** Every sandbox project has a $100/month budget with alert thresholds at 50% and 90%. Runaway jobs get caught early.
- **Lifecycle rules.** Sandbox buckets delete after 30 days of no access. Orphaned resources do not accumulate forever.
- **TTL on datasets.** Sandbox BigQuery datasets have default table expiration of 7 days. Forgotten tables disappear on their own.
- **Network separation.** Sandbox projects sit in a separate VPC with no peering to production networks.

What this unlocks

With this workflow, a developer can:

- Modify a validator, run it against real mainframe-shaped fixtures in their sandbox in under 90 seconds.
- Change a dbt model, rebuild the FDP in their own dataset without affecting anyone else.
- Exercise a new Airflow sensor against a live Pub/Sub topic, verify behaviour, and delete the topic afterwards.
- Reproduce a production bug safely — copy a sample of production data (masked via the same macros production uses) into their sandbox, then iterate until fixed.

Compare that to the usual "make changes, push, wait for the nightly build, read the logs, make a guess, repeat" cycle. The sandbox workflow turns a two-day investigation into a two-hour one.

Review

Strengths:

- **The E2E harness is scripted, cheap, and repeatable.** Nightly runs cost less than a takeaway coffee.
- **Tracing is end-to-end** via OTEL + Pub/Sub attribute propagation.
- **The team test plan is documented** in the repo; not folklore.
- **The sandbox pattern is mature** — per-developer isolation, native auth, budget rails.

Weaknesses:

- **The E2E harness runs serial**, not parallel. Runs take 8 minutes; could be 3 with a fan-out pattern.
- **Tracing with heavy sampling** still drops some useful mid-pipeline spans. A recent-failure-retention sampler would be better.

- **Sandbox provisioning** is a script, not a self-service UI. Onboarding a new developer takes 10 minutes, not 30 seconds.
- **Chaos testing is quarterly, not continuous.** Ideal would be weekly.

Chapter 14 takes the last alternative seriously — what to do when Composer is not an option and you need to run the framework on your own Kubernetes cluster.

Key takeaways

- The E2E harness runs the full round-trip — Terraform, file seed, ingestion, transformation, verify, teardown — against a real GCP project in about 8 minutes for roughly $2 of compute. Running it nightly for a year costs less than one production incident it would have caught.
- Emulators (fake GCS, Pub/Sub, BigQuery) let Docker Compose DAG tests execute real Airflow tasks, produce real XComs, and trigger downstream DAGs, without touching a real GCP project or burning a penny.
- Each developer gets a personal sandbox GCP project, authenticates as themselves via application default credentials, and runs changes against real services before pushing. The sandbox-to-production investigation cycle drops from days to hours.
- A testing plan is only real if it is written, assigns an owner per level, and has a mechanical enforcement action. A level without enforcement drifts to ignored within a quarter.

Chapter 14 — Running on Your Own Kubernetes Cluster

Cloud Composer is the recommended default for almost all teams. It is managed, it is well-integrated with GCP, and it removes the operational burden of running Kubernetes, Airflow, and Postgres yourself. If you can use Composer, use Composer. This chapter is for the cases where you cannot.

The framework ships a complete self-managed Kubernetes deployment that runs all three units — ingestion, transformation, orchestration — on your own GKE, on-prem OpenShift, AKS, EKS, or any CNCF-conformant Kubernetes. That capability exists for teams with genuine constraints. It is not an equal alternative to Composer; it is an escape hatch.

This chapter is that story.

Why self-managed Kubernetes

When not to do this

Stop here first. Do not self-manage Kubernetes for a pipeline if:

- You have fewer than 15 entities. Composer is cheaper and simpler.
- You do not have an existing Kubernetes operations team. The framework's charts are well-documented, but running production Kubernetes is a skill.
- Your data-sovereignty rules can be met by Composer's region selection. Verify before you over-engineer.

If none of those constraints apply, use Composer. Most teams reading this chapter should close it and go back to Chapter 9.

The constraint conditions that justify it

Three legitimate reasons to proceed:

- **Regulatory / data-sovereignty.** Your pipeline must run entirely inside a specific VPC or data centre that Composer does not support.
- **Hybrid estate.** You already run a Kubernetes fleet for microservices; adding Airflow and Beam to it is cheaper than spinning up Composer alongside.
- **Cost at extreme scale.** Above about 40 concurrent DAG runs and 30 entities, a properly-sized self-managed Airflow cluster is cheaper than Composer. Below

that threshold, you are paying a premium in operational complexity without recovering it in cost savings.

The deployment artefacts

The framework ships three component Helm charts plus an umbrella under `infrastructure/k8s/charts/`:

- *Airflow assets* — under `infrastructure/k8s/airflow/` (Dockerfile + values files). Not packaged as a standalone Helm chart; consumed by the umbrella via a `file://` dependency.
- `pipeline-dbt-runner` — a Cloud Run-ish dbt runner deployed as a CronJob + Job pattern. For teams not using Composer's DbtRunOperator.
- `pipeline-beam-runner` — if you cannot use Dataflow, a Flink cluster managed by the Apache Flink Kubernetes Operator, with the framework's Beam pipelines submitted against it.
- `pipeline-observability` — an OTEL collector, Prometheus scrape config, and Grafana dashboards tailored to the framework's metric names.

Plus a thin umbrella chart, `pipeline-system`, that composes them into a single `helm install`.

The cluster shape

Minimal working layout:

- **Control plane node pool:** 2 nodes, 4 vCPU, 16 GB RAM. Runs Airflow webserver, scheduler, triggerer, metadata Postgres (via CloudSQL proxy for data, or in-cluster for air-gapped).
- **Worker node pool:** autoscaling 0 to N, 4 vCPU, 16 GB RAM per node. Runs KubernetesExecutor pods, dbt runner Jobs, and Beam-on-Flink task managers.
- **Observability node pool:** 1 node, 2 vCPU, 8 GB RAM. Runs OTEL collector, Prometheus, Grafana.

A production cluster in a medium-sized bank I worked with ran 12 nodes total, 48 vCPU, 192 GB RAM — comfortable for ~40 entities with daily loads and a handful of streaming pipelines.

Installing the umbrella chart

```
# From the reference repo
helm dependency update infrastructure/k8s/charts/pipeline-system
helm install pipeline \
  infrastructure/k8s/charts/pipeline-system \
  --namespace pipeline \
  --create-namespace \
  --values infrastructure/k8s/values/prod.yaml
```

A representative `values/prod.yaml`:

```yaml
global:
  system_id: generic
  environment: prod
  image_pull_secrets:
    - name: registry-creds
  cloud_sql_proxy:
    enabled: true
    instance: my-project:europe-west2:pipeline-metadata

airflow:                                  # consumed from infrastructure/k8s/airflow/
  executor: KubernetesExecutor
  image:
    repository: my-registry/pipeline-airflow-image
      tag: "2.10.0-1.0.29"
    config:
      core:
        dags_folder: /opt/airflow/dags
        parallelism: 128
        dag_concurrency: 32
    workers:
      resources:
        requests: { cpu: 1000m, memory: 2Gi }
        limits:   { cpu: 4000m, memory: 8Gi }

pipeline-beam-runner:
  enabled: true
  flinkOperator:
    version: "1.19.0"
  defaultTaskManagerProfile:
    replicas: 4
    resources:
      requests: { cpu: 2000m, memory: 4Gi }
      limits:   { cpu: 4000m, memory: 8Gi }

pipeline-dbt-runner:
  image:
    repository: my-registry/pipeline-dbt-runner
    tag: "1.7.14-1.0.29"
  schedule:
    transformation: "0 3 * * *"

pipeline-observability:
  otelCollector:
    exporters: [gcp-monitoring, jaeger]
  prometheus:
    retention: "15d"
  grafana:
    admin_password_secret: grafana-admin
```

Replacing Composer with self-managed Airflow

The key substitutions:

- **Composer's DAG bucket** is replaced by a Git-sync sidecar that pulls `dags/` from a configured repository on a schedule.
- **Composer's metadata Postgres** is replaced by a managed Cloud SQL (or an in-cluster Postgres for air-gapped clusters).
- **Composer's networking** (VPC peering, private IP, IAP) is replaced by your own cluster's ingress (Istio gateway, Anthos Service Mesh, or a plain Ingress controller).
- **Composer's provided providers** are installed via our `Dockerfile` at build time, pinned to the matrix from Chapter 9.

The DAGs themselves do not change. The factory and all generated DAG code are cluster-agnostic.

Replacing Dataflow with Beam-on-Flink

This is the bigger lift. Dataflow is a managed runner; Flink is not. You get:

- **The same Beam SDK.** Pipelines written for Dataflow run on Flink with almost no changes.
- **Autoscaling** via the Flink Kubernetes Operator's `FlinkSessionJob` resource.
- **Checkpointing** to a GCS or S3 bucket.
- **Cost control** via node-pool autoscaling.

You lose:

- **Autotuning.** You set worker counts and resources; Flink does not rebalance on skew the way Dataflow's shuffle service does.
- **UI polish.** Dataflow's job graph is far nicer than the Flink dashboard.
- **Some hotfixes.** Google ships patches to Dataflow in days; Flink patches arrive in minor-version releases.

For teams who cannot use Dataflow, this is an acceptable trade. For teams who can, Dataflow is still the pragmatic choice.

dbt-runner on Kubernetes

`pipeline-dbt-runner` is a small image that packages dbt, the framework's macros, and the BigQuery adapter. It ships two ways to be invoked:

- **As a CronJob** for scheduled daily runs.
- **As a Job triggered by Airflow** via the KubernetesPodOperator — this is how the transformation DAG launches dbt in a Kubernetes-only deployment.

The runner writes logs to stdout (captured by Cloud Logging), emits OTEL spans to the cluster's collector, and writes `run_results.json` to a shared GCS bucket for the status DAG to read.

Observability on a Kubernetes deployment

Three components:

- **OTEL collector** — receives traces and metrics from pipeline code, batches, and forwards to Cloud Trace + Cloud Monitoring (or Jaeger + Prometheus if you are fully air-gapped).
- **Prometheus** — scrapes pipeline pods for health checks, resource usage, and JVM metrics (for Flink).
- **Grafana** — pre-built dashboards for: per-entity throughput, per-entity error rate, per-run cost, reconciliation health, data-quality grade trends, Airflow scheduler health, Flink job manager health.

The Grafana dashboards are shipped as ConfigMaps in the observability chart. They import automatically on `helm install`.

Cost profile

A realistic cost picture for a mid-sized self-managed deployment running the `generic` system daily, in EU:

Component	Cost
GKE control plane	$75/month
12-node worker pool (mix of 4 vCPU / 16 GB)	$900/month
Cloud SQL Postgres (db-custom-2-8192 HA)	$180/month
GCS (DAG sync, temp, checkpoint)	$20/month
Cloud Logging + Monitoring	$50/month
Networking (VPC, NAT, peering)	$60/month
Total	**~$1,285/month**

Compare that to a medium Composer 2 environment at roughly $700/month plus workers. At three entities, Composer wins on cost by a wide margin. At forty entities and extreme DAG concurrency, self-managed wins. The break-even is around 25 entities or 100 concurrent DAG runs. If your scale or constraints do not force you off Composer, you are over-engineering: you are trading $585/month in compute savings for a full-time Kubernetes operations burden.

Review

Strengths:

- **Full Helm umbrella.** One `helm install` brings the stack up.
- **DAG code is portable.** No DAG-side changes to move from Composer to self-managed.
- **Flink-backed Beam** works, and the Beam SDK portability story is better than most teams expect.
- **Observability is baked in**, not bolted on.

Weaknesses:

- **Flink is not Dataflow.** Skew handling and autotuning are weaker.

- **The umbrella chart has many knobs.** Teams without Helm experience will find the values file daunting.
- **No GitOps recipe shipped.** ArgoCD integration is documented but not scaffolded. A future version would do this.
- **Air-gapped story is partial.** Cloud SQL is required by default; in-cluster Postgres is documented but not fully validated.

Key takeaways

- Self-managed Kubernetes is an escape hatch, not an equal alternative to Composer. Use it only under three conditions: regulatory data-sovereignty constraints Composer cannot meet, an existing Kubernetes fleet that makes the operational burden negligible, or extreme scale above roughly 40 concurrent DAG runs.
- The break-even between Composer and self-managed is around 25 entities or 100 concurrent DAG runs — below that, Composer wins on total cost of ownership even though the compute bill is higher.
- The DAG code does not change between Composer and self-managed Airflow. The factory, the generated DAGs, and the entity config are cluster-agnostic.
- Beam-on-Flink works, but you lose Dataflow’s autotune and shuffle-service skew handling. If you can use Dataflow, use Dataflow.

Chapter 15 — CI/CD and Publishing to PyPI

The eleven workflows

`.github/workflows/` contains eleven files:

- **`test.yml`** — runs library and deployment unit tests on pull requests.
- **`publish-libraries.yml`** — builds and publishes libraries to PyPI on tags.
- **`publish-deployments.yml`** — builds and publishes reference deployments to PyPI on tags.
- **`deploy-generic.yml`** — deploys the `generic` system to a GCP project.
- **`deploy-orchestration.yml`** — deploys the orchestration unit (DAGs sync) on its own cadence.
- **`qodana_code_quality.yml`** — runs the JetBrains Qodana scanner on each PR.
- **`ci-automation.yml`** — utility workflow for cron-driven tasks (dependency updates, link checks).
- **`release.yml`** — drafts release notes from PR labels and tags.
- **`ci.yml`** — general CI checks on every PR (lint, format, light unit tests across all libraries).
- **`deploy-gke.yml`** — deploys to a self-managed GKE target when the K8s charts change.
- **`deploy-segment-transform.yml`** — deploys the `mainframe-segment-transform` Beam template separately from the main `deploy-generic` workflow.

Workflow authorship in the framework follows the principle that **publishing and deploying are different things**. A library publish creates a versioned artefact in PyPI; a deployment runs Terraform and pushes a Dataflow template. Conflating them — say, a "release" workflow that does both — invites race conditions where a deployment uses a published version that has not yet propagated through PyPI's CDN.

PyPI as the artefact registry

Six packages live on PyPI:

Package	Role
`gcp-pipeline-core`	Foundation — audit, monitoring, FinOps, errors, schema
`gcp-pipeline-beam`	Beam transforms and pipeline builder
`gcp-pipeline-orchestration`	Airflow operators, sensors, factories
`gcp-pipeline-transform`	dbt macros
`gcp-pipeline-tester`	Test base classes, mocks, fixtures
`gcp-pipeline-framework`	Umbrella; pulls in all of the above plus reference deployments

All six are versioned together. The single source of truth is the top-level `VERSION` file. CI reads this file when packaging and refuses to publish a tag whose version does not match.

The umbrella package, `gcp-pipeline-framework`, is the recommended install. A new team runs `pip install gcp-pipeline-framework` and gets the entire framework plus reference deployments in one step.

The reconstruct.py trick

This is the cleverest piece of CI in the repository, and the one that makes the framework genuinely portable.

`reconstruct.py` is a small Python script that, given an optional version and an optional PyPI index URL, does the following:

- Creates a temporary virtual environment.
- Runs `pip install gcp-pipeline-framework==<version>` against the chosen index.
- Walks the installed package tree.
- Copies the embedded `docs/`, `infrastructure/`, `scripts/`, `templates/`, and `dags/` folders out of the wheel into a destination directory.
- Writes a `README.md` and a `VERSION` file at the destination root.

The result is a fresh, ready-to-use copy of the entire reference repository, **regenerated from PyPI**. No GitHub access required. No source repository required. Just `pip` and a Python interpreter.

This matters for two reasons:

- **Internal corporate networks.** Many large enterprises restrict outbound access to GitHub but allow PyPI mirrors (Nexus, Artifactory). `reconstruct.py --index-url=https://nexus.internal/...` works on those networks.
- **Versioned environments.** `reconstruct.py --version 1.0.27` produces exactly the layout that was current at version 1.0.27, regardless of subsequent changes. This is invaluable for incident replay and for reproducing customer environments.

The mechanism that makes this possible is `pyproject.toml`'s `package_data` configuration:

```
[tool.setuptools.package_data]
"gcp_pipeline_framework" = [
    "embedded/**/*",
]
```

...and a CI step that copies the relevant directories into `embedded/` before the wheel build:

```
- name: Embed project assets
  run: |
    mkdir -p src/gcp_pipeline_framework/embedded
    rsync -a docs/ src/gcp_pipeline_framework/embedded/docs/
    rsync -a infrastructure/ src/gcp_pipeline_framework/embedded/infrastructure/
    rsync -a scripts/ src/gcp_pipeline_framework/embedded/scripts/
    rsync -a templates/ src/gcp_pipeline_framework/embedded/templates/
```

The wheel is therefore self-contained. `reconstruct.py` knows where to look because the layout is fixed by convention.

The deploy workflow

`deploy-generic.yml` is the heart of the deployment story. Its structure is:

```
on:
  push:
    branches: [main]
    paths:
      - 'deployments/original-data-to-bigqueryload/**'
      - 'deployments/bigquery-to-mapped-product/**'
      - 'deployments/data-pipeline-orchestrator/**'
      - 'infrastructure/terraform/systems/generic/**'

jobs:
  detect-changes:
    # determine which units changed
  apply-terraform:
    # only if infrastructure/terraform changed
  build-and-push-dataflow:
    # only if ingestion changed
  deploy-dbt:
    # only if transformation changed
  sync-dags:
    # only if orchestration changed
  smoke-test:
    needs: [...above]
    # run a tiny e2e against a pre-seeded landing file
```

The `paths` filter at the top means the workflow does not run for documentation changes. The `detect-changes` job sets job outputs that the downstream jobs gate on, so even within a triggered run, only the relevant units actually deploy.

A small but valuable touch: the smoke test runs only after all deploy jobs complete, and its failure rolls the deploy backwards via a Terraform `plan` against the previous

tag. This is not full transactional rollback (Terraform cannot easily undo a Dataflow template registration), but it is enough to catch the most common class of bad deploy.

Secrets, identities, and Workload Identity Federation

Authentication to GCP from GitHub Actions uses **Workload Identity Federation**, not service-account JSON keys. The setup script `scripts/gcp/setup_github_actions.sh` provisions:

- A workload identity pool.
- A provider trusted to GitHub's OIDC issuer.
- Per-environment service accounts with the necessary IAM bindings.

In each workflow:

```
- uses: google-github-actions/auth@v2
  with:
    workload_identity_provider:
↪   projects/.../locations/global/workloadIdentityPools/github-
↪   pool/providers/github
    service_account: pipeline-deployer@<project>.iam.gserviceaccount.com
```

This avoids long-lived JSON keys, which are the leading cause of accidental credential leakage in pipeline projects.

Versioning, tags, and releases

The release flow is:

1. Bump `VERSION`.
2. Update `CHANGELOG.md` (often via the `release.yml` draft).
3. Open a PR; pass CI; merge.
4. Tag the merge commit with `v<version>`.
5. The publish workflows fire on the tag and push to PyPI.

There is no semantic-version automation. The framework's author decided that version numbers are an editorial decision, not a mechanical one. I think this is the right call for a framework whose changes' breaking-ness is contextual.

Code quality gates

Two automated gates back the test suite:

- **Qodana** runs on every PR. The `qodana.yaml` configuration uses the JetBrains Python profile with framework-specific exclusions. The CI uploads the report to Qodana Cloud and posts a summary comment on the PR.
- **Pre-commit hooks** run `black`, `ruff`, and `mypy` locally and (via a separate workflow) in CI on PRs.

Coverage is calculated but not gated. The framework's author deliberately avoided a "must be 90%" rule, on the grounds that a coverage gate produces incentive to write

tests that pad the number rather than tests that catch bugs. I have a lot of sympathy for this view.

Review

Strengths:

- **Path-filtered deployments.** Only relevant units redeploy.
- **PyPI as portable artefact registry.** The reconstruct.py trick is, frankly, brilliant.
- **WIF, not JSON keys.** Modern, secure, easier to rotate.
- **Publish vs deploy separation.** Avoids race conditions.

Weaknesses:

- **Smoke test is single-entity.** The full e2e is still manual.
- **No canary deploy pattern.** A bad ingestion image goes straight to prod.
- **Release notes are PR-label driven.** Drift is possible if labels are forgotten.

Chapter 16 takes the camera back to the seven reference deployments and walks through what each demonstrates.

Key takeaways

- Publishing and deploying are different things. A publish creates a versioned PyPI artefact; a deploy runs Terraform and pushes a Dataflow template. Conflating them creates race conditions.
- `reconstruct.py` regenerates the entire reference project from a single `pip install`. On corporate networks that cannot reach GitHub but can reach an internal PyPI mirror, this is the difference between ”adoptable” and ”not adoptable”.
- Path-filtered CI means a one-line README change does not redeploy production. Only the units that actually changed redeploy — the others do not even run.
- Workload Identity Federation, not JSON keys. Long-lived service-account JSON files are the leading cause of accidental credential leakage in pipeline projects.

Chapter 16 — A Tour of the Reference Deployments

The framework ships seven reference deployments. Three are "active" — they are exercised by CI, deployed by the standard workflow, and exercised by the e2e test. Four are "specialised" — they exist to demonstrate adjacent patterns and may not be deployable in every environment without modification. One additional component, `fdp-trigger`, provides downstream notification.

This chapter is the catalogue.

`original-data-to-bigqueryload` — the JOIN+MAP ingestion

We have met this already. It is a Dataflow Flex Template, packaged as a Docker image, parameterised per entity. It supports four entities (customers, accounts, decision, applications) and proves both the JOIN sources (customers, accounts) and the MAP sources (decision, applications) in the same deployment.

Its Cloud Build YAML is the canonical pattern for any future ingestion deployment in the framework. Twenty-six unit tests exercise its parameter validation, its target-table naming, and its environment-specific behaviour.

`bigquery-to-mapped-product` — the FDP transformation

This is the dbt project we explored in Chapter 8. Nineteen models across staging, FDP, marts, and analytics layers. JOIN model: `event_transaction_excess`. MAP models: `portfolio_account_excess`, `portfolio_account_facility`. Custom macros (`generate_audit_columns`, `apply_pii_masking`, `data_quality_check`).

Two non-obvious details worth knowing:

- The dbt project uses `generate_schema_name` to redirect models into per-environment datasets (`fdp_generic_dev` vs `fdp_generic_prod`) without changing model SQL.
- The `dbt.yml` declares a `vars:` block that the Airflow `transformation` DAG populates at runtime: `system_id`, `extract_date`, `run_id`. Models reference these variables in audit macros.

`data-pipeline-orchestrator` — the five-DAG orchestrator

Covered in Chapter 9. Five DAGs (pubsub_trigger, ingestion, transformation, error_handling, status). Generated by the 87 KB `generate_dags.py` factory. Reads `system.yaml` for entity definitions.

Worth noting: the orchestrator deployment has its own `pyproject.toml` and is published to PyPI as `gcp-pipeline-data-pipeline-orchestrator`. A team can `pip install` the orchestrator and drop the resulting `dags/` folder into their own Composer environment without taking the rest of the framework.

`fdp-to-consumable-product` — the Consumable Data Product layer

This is a fourth, specialised dbt project. It reads from the FDP and produces a "Consumable Data Product" (CDP) — narrow, documented, contract-bound tables for downstream consumers (data scientists, BI teams, machine-learning pipelines).

Why is this separate from `bigquery-to-mapped-product`? Because the lifecycle is different. The FDP changes when the source mainframe changes. The CDP changes when downstream consumers' contracts change. Coupling them in one dbt project means a downstream contract change forces an FDP redeploy, which is a regulator-visible event and therefore expensive.

`fdp-to-consumable-product` is code-complete in the reference repository but not deployed by the standard workflow. Teams adopt it when they have downstream consumers needing contracted views.

`mainframe-segment-transform` — round-tripping back to the mainframe

A surprising number of mainframe migrations are bidirectional. The mainframe sends data to BigQuery for analytics; BigQuery sends derived data (scores, segments, decisions) back to the mainframe for operational use.

`mainframe-segment-transform` is a Beam pipeline that reads from a CDP table and writes a fixed-width segment file in a format the mainframe can read. It handles:

- Field padding (left vs right, space vs zero).
- HDR/TRL on output.
- Encoding (UTF-8 → cp037 EBCDIC where required).
- Split-file output for files larger than the mainframe's network policy.

It is a useful demonstration that the framework's HDR/TRL idiom is bidirectional, not just a parser. Code-complete in the repository; deployment is environment-specific.

`spanner-to-bigquery-load` — federated query patterns

Some entities live in Cloud Spanner, not on a mainframe. For these, batch ingestion through GCS is wasteful: the data is already in a relational database. A federated query is cheaper and faster.

`spanner-to-bigquery-load` is a dbt-only deployment (no Beam, no Dataflow) that uses BigQuery's `EXTERNAL_QUERY` against a Spanner connection to materialise FDP tables directly from Spanner sources. It is tagged "reference" because the connection setup is environment-specific and requires an additional IAM pattern not provisioned by the standard Terraform.

The pattern it demonstrates is general: when your "source system" is already a queryable database, prefer federation to extract-load. It saves a layer.

`postgres-cdc-streaming` — the streaming reference

This deployment is tagged "reference/planned". It exists as a skeleton for streaming change-data-capture from a Postgres source to a BigQuery streaming insert, using:

- Datastream as the CDC source.
- A Dataflow streaming pipeline to consume the Datastream output.
- Streaming inserts into a BigQuery FDP table.
- Watermark-aware joins for the streaming JOIN equivalent.

The skeleton is honest about its state. It is not production-ready. It is published as a starting point for teams who need streaming and want a framework-aligned baseline.

`fdp-trigger` — downstream notification

`fdp-trigger` is not a pipeline. It is a small Cloud Function (or, optionally, Cloud Run service) that listens for FDP completion events on a Pub/Sub topic and:

- Verifies that the run was successful end-to-end (audit, reconciliation, quality grade).
- Publishes a downstream "data ready" event to a tenant-specific Pub/Sub topic.
- Updates a small `fdp_state` table for ad-hoc consumers that prefer polling.

It exists to formalise the contract between the pipeline and its consumers. Without it, every consumer has to figure out for itself when "the data is fresh"; with it, there is a single, auditable signal.

Picking a deployment style

For a new team adopting the framework, my advice is:

- Start with `original-data-to-bigqueryload` and `bigquery-to-mapped-product`.
- Add `data-pipeline-orchestrator` only if you genuinely need Airflow.
- Add `fdp-trigger` as soon as you have your first downstream consumer.
- Adopt `fdp-to-consumable-product` when you have second-tier consumers with contracts.

- Treat `mainframe-segment-transform`, `spanner-to-bigquery-load`, and `postgres-cdc-streaming` as templates to crib from when their patterns apply, not as drop-in deployments.

Key takeaways

- The three active deployments (`original-data-to-bigqueryload`, `bigquery-to-mapped-product`, `data-pipeline-orchestrator`) prove both JOIN and MAP patterns in a single system. That is intentional — a reference that only showed one pattern would not be representative.
- `mainframe-segment-transform` proves the HDR/TRL idiom is bidirectional. The same envelope pattern that receives mainframe data can produce mainframe-readable fixed-width output with EBCDIC encoding and split-file output.
- `fdp-trigger` formalises the contract between the pipeline and its downstream consumers. Without it, every consumer reinvents ”how do I know the data is fresh?” With it, there is one auditable signal.
- Adopt the specialised deployments when the pattern fits, not by default. Starting with the three active ones is almost always the right first move.

Chapter 17 — An Honest Code Review

Throughout the book I have woven in a "what works, what could be better" subsection at the end of each chapter. This chapter pulls those observations together and adds the cross-cutting concerns that did not fit anywhere else.

I want to be clear: this is a strong codebase. The fact that there is anything to criticise is not damning; it would be more worrying if a system this large had no rough edges. Treat what follows as a punch list for the next major version.

What the framework gets very right

- **Framework-agnostic core.** No Beam, no Airflow imports in `gcp-pipeline-core`. This is the design decision that makes everything else possible.
- **Schema as single source of truth.** `EntitySchema` drives ingestion, validation, table creation, masking, and tests. The reduction in drift is enormous.
- **JOIN/MAP vocabulary.** Naming the two transformation patterns explicitly turns architectural folklore into engineering process.
- **Run-ID-driven lineage.** One identifier ties logs, metrics, audit events, dbt invocations, and cost records together.
- **First-class FinOps.** Cost tracking is a built-in subsystem, not a downstream dashboard. The label discipline makes attribution actually work.
- **Three-unit deployment model.** Ingestion, transformation, and orchestration are independently versioned, deployed, and owned. Nothing else scales as well across teams.
- **Composer is opt-in.** The single most cost-aware choice in the framework. Default deploys do not bury teams in $300/month bills.
- **PyPI publishing of everything.** `reconstruct.py` plus embedded assets means a team can fully rebuild the project from packages alone.
- **Path-filtered CI.** A docs change does not redeploy production. A library change runs tests but not deploys.
- **Workload Identity Federation.** No long-lived JSON keys.
- **Quarantine and safe deletion.** A four-stage workflow with audit trail satisfies most regulators out of the box.

What I would change first

- **Automate the e2e test.** A scheduled CI run against a dedicated `pipeline-e2e` GCP project, exercising the full landing-to-FDP path, would convert "we have a script for that" into "we know it works at 06:00 today".
- **A canary deploy pattern.** A new Dataflow image goes straight to prod. A canary that runs the new image against a small subset of an entity's files for a few hours would catch regressions that unit tests cannot.
- **A cost gate.** A simple "if any entity costs more than $X this week, alert" rule. The data is in `finops_usage` already; the alert wiring is not.
- **Stricter client wrappers.** It is too easy to bypass `gcp-pipeline-core.clients` and call `google.cloud.bigquery` directly. A lint rule (and a documented escape hatch) would tighten the discipline.
- **A central `CoreConfig` object.** Audit, FinOps, and deletion all read environment variables independently. A single, injected config object would be more testable and more discoverable.

What I would change next

- **A `streaming` deployment that actually works.** The Postgres CDC reference is a skeleton. A working streaming deployment with end-to-end exactly-once semantics would round out the framework.
- **A small admin UI.** Quarantine review and FDP failure investigation are currently CLI-and-Slack flows. A small Streamlit app would dramatically improve operability.
- **Policy-as-code.** OPA or Sentinel hooks in the Terraform CI would catch the next "someone created a public bucket" incident before it ships.
- **Property-based tests for the parsers.** The HDR/TRL parser handles a lot of edge cases. Hypothesis would find more.
- **A documented deployment-profile abstraction.** Choosing the right Dataflow worker count, machine type, and shuffle mode is currently ad-hoc. A small set of named profiles ("small daily", "large weekly", "streaming") would help.

What I would not change

A few things I noticed that are tempting to "improve" but I think are correct as they are:

- **The single VERSION file** for all six libraries. Synchronised versioning is unfashionable. It is also enormously easier to reason about than independent semver per package.
- **No coverage percentage gate**. Coverage targets push tests towards the easy wins, not the important ones.
- **Tfvars rather than workspaces** for environments. More explicit, more diff-friendly, easier to audit.
- **The 87 KB DAG factory file**. It is large, but it is one cohesive concept. Splitting it would just create cross-file coupling.

Reading recommendations for new contributors

If you are joining a team using this framework, the path I would recommend through the source is:

1. `README.md` and `docs/TECHNICAL_ARCHITECTURE.md`
2. `gcp-pipeline-libraries/gcp-pipeline-core/src/gcp_pipeline_core/schema.py`
3. `gcp-pipeline-libraries/gcp-pipeline-core/src/gcp_pipeline_core/audit/`
4. `gcp-pipeline-libraries/gcp-pipeline-core/src/gcp_pipeline_core/finops/`
5. `gcp-pipeline-libraries/gcp-pipeline-beam/src/gcp_pipeline_beam/file_management/hdr`
6. `gcp-pipeline-libraries/gcp-pipeline-beam/src/gcp_pipeline_beam/transforms/`
7. `gcp-pipeline-libraries/gcp-pipeline-orchestration/src/gcp_pipeline_orchestration/fa`
8. `deployments/data-pipeline-orchestrator/dags/generate_dags.py`
9. `infrastructure/terraform/systems/generic/`
10. `.github/workflows/deploy-generic.yml`

In that order, you will see the framework grow from data structures upwards. By the end you will know enough to extend it.

Key takeaways

- The framework's genuine strengths — framework-agnostic core, schema as single source of truth, `run_id`-driven lineage, first-class FinOps, opt-in Composer — are all architectural decisions, not implementation details. They hold up under production scrutiny.
- The most important gap is the missing automated e2e test in CI. "We have a script for that" is not the same as "we know it works at 06:00 today".
- Things that look like candidates for "improvement" but are actually correct: the single VERSION file, no coverage gate, tfvars over workspaces, the monolithic 87 KB DAG factory. The temptation to split them is real; the reason not to is coherence.
- A canary deploy pattern and a cost-budget alert are the two changes with the highest production-incident prevention value for the smallest engineering effort.

Chapter 18 — Governance, Masking, and the Data Product Lifecycle

What "governance" actually means once a product is live

In Chapter 6 I made a fuss about `EntitySchema` being the single source of truth, and in Chapter 8 I showed off the `apply_pii_masking()` macro that reads it. Both are true and both still hold. But I want to be honest about something the framework, on its own, does not quite solve.

The framework's view of governance is *schema-driven*. You declare which columns are PII, the macro hashes them at the FDP boundary, and `_fdp_run_id` lets you trace any row back to the run that produced it. That is more than most teams ship in production, and on a small platform with one or two systems it is genuinely enough.

Once you cross a certain threshold, though — half a dozen systems, three regulatory regimes, an internal data marketplace where analysts request access to columns they have never seen — the schema-driven story develops gaps. The ones I keep tripping over are these:

- **Mask variety.** The macro only hashes. Hashing is irreversible, which is excellent for analytics safety and useless if your fraud team legitimately needs to re-identify a customer.
- **Column-level access control.** Hashing protects the value once it is in BigQuery, but nothing stops a clever analyst from joining `customer_id` (unmasked) against another table they happen to have read on, and reconstructing the population. Real governance needs per-column visibility, not just per-value redaction.
- **Lineage at scale.** Our `run_id`-stitched lineage is excellent within a pipeline. It does not, by itself, tell a data steward "this Looker dashboard ultimately depends on six tables across three projects".
- **Catalogue and discovery.** When an analyst asks "do we already have a table with customer postcodes in it?", I do not want them grepping the dbt project. I want them searching a catalogue.
- **The post-creation lifecycle.** A data product is not done when it ships. Schema evolution, deprecation, ownership rotation, access reviews, audit-trail retention — none of that lives in our framework today.

This chapter is about the layer on top: what to use for masking beyond hashing, how column-level access works, what Cloud DLP and Dataplex actually give you in 2026, and what governance work is still your job once the product is in production.

Four masking techniques, and when each applies

There are four masking techniques worth knowing. Our framework ships one of them. I will say so plainly and then say what I would do if I needed the others.

Hashing — irreversible, deterministic, no key management. The framework's `apply_pii_masking()` macro emits `TO_HEX(SHA256(CAST(field AS STRING)))`. Two identical input values produce identical hashes, which means analysts can still `GROUP BY` and `COUNT(DISTINCT)` over the masked column, which is usually what they actually need. There is nothing to rotate, nothing to leak. The cost is that you cannot go back. If a fraud analyst rings up and says "we need the real email for this customer", you cannot help them from the FDP — you would have to go upstream to the ODP, which most of your users do not have access to. That is usually a feature, not a bug.

Tokenisation — reversible via a secure vault. The plaintext is replaced with an opaque token, and a separate, tightly controlled service can map the token back to the plaintext when authorised. The vault becomes the single point of risk and the single point of compliance. Format-preserving tokenisation (FPE) is a useful variant: a sixteen-digit card number stays sixteen digits, so downstream systems that expect that shape do not need to change. Tokenisation is the right answer when you genuinely need re-identification — fraud investigations, customer support, regulator subject access requests — and when you can defend the operational cost of running and auditing the vault. Cloud KMS plus a small internal service is the usual recipe on GCP; Cloud DLP also offers de-identification with format-preserving cryptographic tokens, which is worth a look.

If I needed to add tokenisation to the framework, I would not change the macro signature. I would add a second macro, `apply_pii_tokenisation(schema, field)`, that emits a `CALL` into a BigQuery remote function backed by a Cloud Run service that talks to KMS. The choice between hashing and tokenisation would then live on the `SchemaField` itself — `pii_treatment="hash"` vs `pii_treatment="token"` — alongside the existing `pii=True` flag. That keeps the schema as single source of truth and keeps the FDP author from having to decide which scheme applies.

Nullification — the simplest of the four. Replace the value with `NULL` (or a constant like `"[REDACTED]"`). Useful for audit copies, public exports, and any case where the column simply should not exist in the consumer's view. It loses all utility — you cannot join, group, count distinct — and that is the point. I tend to use nullification for shipping anonymised samples to a vendor or for export buckets read by a third party with no need-to-know.

Partial masking — show last four of a SSN, first letter of a name, first half of a postcode. This is the masking technique analysts secretly want. It preserves enough utility that fraud teams can still pattern-match without anyone seeing the full value. The downside is that "partial" requires judgement: how much of a UK postcode is OK to expose? (Probably the outward code; never the full unit.) Partial masking

belongs in a macro that takes a strategy argument — `apply_pii_masking(schema, field, strategy="last4")` — so the choice is auditable and consistent rather than copy-pasted across models.

The framework, to repeat, only ships hashing. I do not think that is a bug for the use cases it was built for — regulated mainframe-to-BigQuery pipelines with strict need-to-know — but if your organisation has a fraud investigation team, a customer-services population that needs to look up real records, or a public-data programme, you will need at least tokenisation and partial masking, and you should be honest with yourself that those are not free.

BigQuery policy tags: who sees the unmasked column

Masking decides *what value* a column holds. Policy tags decide *who can read which column at all*. The two are complementary; you want both.

A policy tag in BigQuery is a label attached to a column that points at a taxonomy in the (now-merged-into-Dataplex) catalogue. The taxonomy has a hierarchy — `PII > High`, `PII > Medium`, `PII > Public` — and each node has IAM bindings. If the user querying does not have `bigquerydatapolicy.dataPolicyUser` on the tag, they get a permission error on that specific column, even if they otherwise have `bigquery.dataViewer` on the table.

In practice the DDL looks like this:

```
-- Once, per project: create the taxonomy
CREATE SCHEMA `analytics_eu.taxonomies` OPTIONS(location='EU');

-- Tag the columns that need protection
ALTER TABLE `fdp.event_transaction_excess`
ALTER COLUMN full_name
SET OPTIONS (
  policy_tags = (
    'projects/my-proj/locations/eu/taxonomies/123/policyTags/pii_high'
  )
);

ALTER TABLE `fdp.event_transaction_excess`
ALTER COLUMN postcode
SET OPTIONS (
  policy_tags = (
    'projects/my-proj/locations/eu/taxonomies/123/policyTags/pii_medium'
  )
);
```

A junior analyst with `bigquery.dataViewer` on the `fdp` dataset can `SELECT account_id, event_amount FROM event_transaction_excess` and it works. The moment they do `SELECT *` or explicitly `SELECT full_name`, they get a `Access Denied: Policy tag projects/.../pii_high requires permission ...` error. The fraud team, who *do* have that role on `pii_high`, see the column without issue.

The interaction with our masking macro is the bit worth thinking through. The macro produces a column called, say, `full_name_masked`. The unmasked `full_name` is upstream in `stg_customers`. You can tag *both* — `full_name` in staging with `pii_high`, `full_name_masked` in the FDP with `pii_low` — and your access policy then expresses, in IAM-bindings rather than SQL, exactly who sees what. The mask is the floor, the policy tag is the ceiling.

Two honest caveats. First, policy tags require the taxonomy to exist in what is, in 2026, Dataplex Universal Catalog (formerly Data Catalog) — so you cannot adopt them in isolation; you are at least adopting the catalogue. Second, policy tags are stricter than most analysts expect. `SELECT *` against a table with any tagged column fails for unprivileged users, which surprises people who came up on Postgres. Train your users early.

Cloud DLP / Sensitive Data Protection

Cloud DLP — rebranded by Google as **Sensitive Data Protection** somewhere around 2024 and still called both interchangeably — is the GCP service that auto-discovers PII in arbitrary data. It ships a library of *infoType detectors* (email addresses, credit card numbers, NHS numbers, US SSNs, UK NI numbers, IBANs, dozens more) and lets you scan a BigQuery table, a GCS bucket, or a Datastore kind, and ask "what PII is in here?".

It is genuinely useful for a specific shape of problem. The canonical case is: **you have inherited a bucket full of CSVs, exported by some legacy system, and you have no idea what is in them.** DLP will scan, classify, and report. It can also de-identify in-place using a configurable template (redact, mask, tokenise with FPE, hash with HMAC), which is handy as a one-shot sanitisation step before loading into BigQuery.

It is much less useful for the case our framework was built for. In our world, your engineers already know the schema. You declared which fields are PII when you wrote the `EntitySchema`. You do not need a service to *discover* what you already declared. Running DLP across an FDP that you know to be hash-masked is paying ten cents per GB scanned to confirm what your schema told you for free.

So my rule of thumb is:

- **Use DLP** when you do not yet have a schema (initial discovery, migration projects, third-party-supplied data of unknown shape).
- **Skip DLP** when your `EntitySchema` already declares the PII fields and the framework already masks them.
- **Use DLP** when you need format-preserving cryptographic tokenisation and you do not want to build a vault — DLP's de-identification templates are a reasonable shortcut.
- **Skip DLP** as a periodic compliance scan over the FDP; you are paying to relearn what you already know, and the false-positive rate (an integer column that happens to look like a credit-card prefix) generates noise that nobody triages.

A practical mid-point: schedule a DLP *profile* (not a full scan, just the lightweight statistical profile) over the ODP datasets monthly. It is cheap, it catches the case

where a mainframe quietly started sending a new column you didn't model, and the findings flow into Dataplex automatically — which gets us to the next section.

Dataplex Universal Catalog

By 2026 Google has finished the merger they had been signposting for years: **Data Catalog and Dataplex are one product called Dataplex Universal Catalog**. The old Data Catalog APIs still work for backwards compatibility; new development is on the unified surface. There are four sub-products that matter for the kinds of pipelines this book is about.

Catalogue and discovery

The catalogue layer indexes BigQuery datasets, Cloud Storage buckets, Pub/Sub topics, Spanner, Bigtable, and a few others, and gives you full-text search across all of them. You can attach *tag templates* — typed metadata structures — to any asset. A reasonable starter tag template for our pipelines might be:

```
# tag_template: data_product
display_name: "Data Product"
fields:
  - id: owner
    type: STRING
    required: true
  - id: classification
    type: ENUM
    values: [public, internal, restricted, secret]
    required: true
  - id: domain
    type: STRING
  - id: source_system
    type: STRING
  - id: review_cycle_days
    type: DOUBLE
    required: true
```

Attach that template to every FDP table and every CDP mart. Now an auditor can ask "show me every restricted data product that has not been reviewed in 180 days" and get a single Dataplex search rather than a heroic spreadsheet exercise. This is the most clearly-worth-it part of Dataplex for any organisation past about ten data products.

Automated data quality (AutoDQ)

AutoDQ is Dataplex's answer to "scan my BigQuery tables and tell me if they look healthy". Rules are defined in YAML against a scan target:

```
# autodq_scan: event_transaction_excess_quality
data:
  resource: //big-
↪  query.googleapis.com/projects/proj/datasets/fdp/tables/event_transaction_excess
rules:
```

```
  - column: account_id
    dimension: COMPLETENESS
    nonNullExpectation: {}
  - column: account_id
    dimension: UNIQUENESS
    uniquenessExpectation: {}
  - column: event_amount
    dimension: VALIDITY
    rangeExpectation:
      minValue: "0"
      maxValue: "1000000"
  - column: event_date
    dimension: FRESHNESS
    rowConditionExpectation:
      sqlExpression: "event_date >= CURRENT_DATE() - 7"
postScanActions:
  bigqueryExport:
    resultsTable:
↪   //bigquery.googleapis.com/projects/proj/datasets/dq/tables/autodq_results
```

It is genuinely good. The findings land in a BigQuery table you can join with anything else, and Dataplex will fire a Cloud Monitoring alert on failure.

The honest comparison with what we already have: AutoDQ overlaps significantly with dbt's `schema.yml` tests and our `DataQualityChecker` macro. Both check completeness, uniqueness, and validity. Both can fire alerts. The differences worth weighing:

- **dbt tests run in the transformation DAG**, gate downstream steps, and produce a `TEST_FAILED` event in our audit trail. AutoDQ runs on a Dataplex schedule, independent of the pipeline, and gates nothing by default.
- **dbt tests are versioned with the model SQL in git**. AutoDQ rules are managed in Dataplex (or via Terraform, which is what I would actually do); the source of truth is less obvious to a dbt-native team.
- **AutoDQ produces a visible quality score on the Dataplex catalogue entry**, which non-engineers — data stewards, product managers, auditors — actually look at. dbt test results live in CI logs that nobody reads.

My recommendation is to run both, not one instead of the other. dbt tests for in-pipeline gating; AutoDQ for the catalogue-level health badge that the rest of the organisation sees. Yes, that is some duplication. The duplication buys you a real audience for the quality signal.

Data profiling

Dataplex profiling computes column statistics — null rate, distinct count, min/max, top-N values, length distribution — and runs the DLP infoType detectors over a sample. The result is attached to the catalogue entry and visible to anyone with read access. Two things make this worth turning on:

- It picks up the **silent schema drift** case I mentioned in the DLP section. If a mainframe quietly starts sending email addresses in a column you modelled as

a free-text note, the profile catches it. Your ingestion pipeline does not, because the value is still a string.
- It gives analysts a **light-touch data dictionary** with no engineering effort. "What does the `decision_type` column actually contain?" — open the profile, see the top-10 values and their frequencies, problem solved.

The cost is modest if you sample (Dataplex defaults to a sensible sample size) and prohibitive if you scan full tables. Sample.

Managed data lineage

Dataplex's lineage tracker observes BigQuery jobs, Dataflow jobs, and dbt runs (via an integration) and builds a directed graph of table-to-table dependencies. The graph is visible in the Dataplex UI and queryable via API.

This is the bit that confused me when I first looked at it, because we already have lineage in the framework — every row has `_fdp_run_id`, the audit trail joins it to upstream jobs, you can answer "what made this row?" with a single SQL query. So what does Dataplex lineage add?

The honest answer is **they are different lineages and you want both**.

- **Our run-id lineage is *run-to-artefact*.** Given a specific row, what specific pipeline run, dbt invocation, and Dataflow job produced it? That is operational lineage. It answers "why is this row wrong today?".
- **Dataplex lineage is *table-to-table*.** Given a table, what tables flow into it and what tables flow out of it? That is structural lineage. It answers "if I deprecate this column, who breaks?".

For a data steward planning a schema change, structural lineage is the right tool. For an on-call engineer debugging Thursday morning's numbers, run-id lineage is the right tool. They are complements, not alternatives.

A pragmatic detail: turning on Dataplex lineage for BigQuery is essentially free and a single Terraform line. There is no reason not to do it the moment you have a project worth governing.

Lakes and zones

Dataplex has a concept above the catalogue and the scans: **lakes and zones**. A *lake* is a logical grouping of data assets — typically per business domain or per regulatory boundary. Inside a lake, *zones* express maturity:

- A **raw zone** holds untransformed, source-of-truth data — typically GCS buckets and ODP-style BigQuery datasets. Schema may be loose; access is engineering-only.
- A **curated zone** holds modelled, masked, business-ready data — the FDP and the mart layer. Schema is strict; access is broader, gated by policy tags.

If you squint, that maps cleanly onto our ODP / FDP / CDP layering. Raw zone ≈ ODP. Curated zone ≈ FDP + mart. The framework's layering convention and Dataplex's

zone model are not in tension; they are saying the same thing at different levels of abstraction.

The question is whether the extra structure is worth adopting. My view:

- **For a small team running one or two systems in one GCP project**, lakes and zones are overkill. You already know which dataset is which; the zone model adds Terraform you don't need and a UI nobody opens. Skip it.
- **For an organisation with a dozen systems, multiple projects, and a real data stewardship function**, lakes and zones are worth the complexity. They give the steward a single console to govern across projects, attach domain-wide tags and policies, and report on health. The cost of adoption pays back within a year.
- **For anyone in between**, adopt the catalogue and AutoDQ first, watch how your stewards actually use them, and only add lakes and zones once the seams start to show.

A small concrete tip: when you do create lakes, make them **domain-aligned, not technology-aligned**. A "customer" lake spanning ODP and FDP datasets and the customer-360 GCS bucket is more useful than a "BigQuery" lake and a "GCS" lake. The whole point of the model is to elevate domain concerns above storage concerns.

The data product lifecycle, after creation

Most of this book has been about *building* data products. What the framework helps less with — and what governance, properly understood, is mostly about — is what happens after the product is live. A data product has a lifecycle, and the work does not stop at deployment. Six things keep happening:

Schema evolution and reverse compatibility. The day after a CDP goes live, someone will need a new column. Adding columns is easy; removing or renaming them is where teams hurt themselves. My rule: never remove a column in less than two release cycles. Mark it `deprecated` in the `EntitySchema` metadata, raise a warning in dbt when it is selected, give consumers a quarter to migrate, and only then drop. A `deprecated_after` date on the field is the simplest way to encode this.

Deprecation paths. The same principle applies to whole tables. When a mart is being retired, the catalogue tag should change from `active` to `deprecated`, every consumer query should produce a warning in logs (BigQuery's row-access policies can be coaxed into this), and a sunset date should be visible to anyone looking at the catalogue entry. The framework's `data_deletion` workflow — REVIEW, HOLD, DELETE, ARCHIVE — applies just as well to whole tables as to rows. Use it.

Ownership rotation. Owners leave. A data product whose `owner` tag still points at someone who departed two years ago is one of the most reliable signs of an organisation that has stopped caring. A simple Dataplex search — "owner in (list of departed employees)" — surfaces these, and a quarterly rotation review forces them to be reassigned to someone who will actually answer the pager.

Access reviews. IAM bindings on FDP datasets, policy tags, and tokenisation vaults all drift. Someone joins a project, gets `dataViewer`, moves to a different project, and

nobody removes the binding. SOX and SOC 2 both expect periodic access reviews; you should expect them too. A quarterly cron that exports IAM bindings to a BigQuery table, joins them against your HR system, and reports orphans is twenty lines of code and saves a great deal of audit-time pain.

Audit-trail retention. The framework writes audit events liberally. Without a retention policy, `job_control.audit_events` becomes the most expensive table in your warehouse within eighteen months. Decide explicitly: hot-tier retention (typically 90 days) for everyday queries, then partitioning-out to a long-term archive (Cloud Storage Coldline or BigQuery's long-term storage tier) for the regulator-mandated period — often seven years for financial data. The reconciliation engine should read from the union view; analysts should read from the hot tier only.

Quality-score SLOs. The `DataQualityChecker` produces an A–F grade per run. Once you trust that grade, codify it as an SLO: "the customers entity will maintain a grade of B or better on 95% of daily runs over a rolling 28-day window." Now the quality grade is not just a number on a dashboard — it is a contract with consumers, and a breach is a real incident with a real post-mortem.

None of this is glamorous, and none of it is a single Terraform module away. It is process, and process is mostly the part of governance that organisations under-invest in. If your team can do four of those six well, you are ahead of nearly every data platform I have worked with.

An honest recommendation

If you have read this far you might reasonably ask: should I adopt all of Dataplex tomorrow?

No. Here is what I would actually do, in order, for a team running the framework in production today.

Week one — catalogue and lineage. Turn on Dataplex catalogue and BigQuery lineage. Both are essentially free, both immediately useful. Tag every FDP and CDP table with owner, classification, and review cycle. Spend an afternoon, ship it, do not over-engineer the tag template.

Month one — policy tags on PII. Create a small taxonomy — `pii_high`, `pii_medium`, `pii_low`, `public` — and tag the columns the schema already declares as PII. Grant the appropriate IAM bindings. Train your analysts on what `SELECT *` is now going to do. This is the single highest-leverage governance move I know of on GCP.

Quarter one — AutoDQ alongside dbt tests, not instead. Pick the half-dozen most business-critical FDP tables, write AutoDQ scans for them, surface the results in the catalogue. Keep your dbt tests exactly as they are; they gate the pipeline, AutoDQ gates the conversation with stewards.

Year one — lifecycle process. Stand up the access-review cron, the deprecation playbook, the quality-score SLO. None of this needs new technology. It needs calendar invites and a `governance.md` in the repo.

Only if it pays back — lakes and zones, DLP at scale. Adopt lakes and zones when you have more than one project worth governing. Run DLP when you have data of unknown provenance. Both are powerful and both are easy to over-buy.

And finally — and this is the bit that matters most — **do not throw away the framework's masking macro because Cloud DLP exists, or your dbt tests because AutoDQ exists, or your `run_id` lineage because Dataplex lineage exists.** They are layers of the same picture. The framework's mechanisms are precise, cheap, and live inside the pipeline; the GCP services are broad, more expensive, and live above it. The best governance posture I have seen is honest about which questions each layer answers, and uses each only for the questions it is good at.

Key takeaways

- The framework ships hashing only. Tokenisation, nullification, and partial masking are real needs; if you have a fraud team or a public data programme, plan to add them, ideally as a `pii_treatment` field on `SchemaField` so the schema stays the source of truth.
- Masking and policy tags are complements, not alternatives. The mask defines the value; the policy tag defines who sees the unmasked column. You want both.
- Cloud DLP is the right tool for data of unknown provenance; it is the wrong tool for an FDP whose `EntitySchema` already declares the PII. Use it as a discovery service, not a periodic compliance scan.
- Dataplex lineage (table-to-table) and our `run_id` lineage (run-to-artefact) answer different questions. Adopt both; do not let one displace the other.
- Lakes and zones map cleanly onto ODP / FDP / CDP, but they are overkill for a small team. Adopt the catalogue and AutoDQ first; add lakes only when you have multiple projects worth governing.
- Governance after a product ships is mostly process — schema deprecation, ownership rotation, access reviews, audit retention, quality SLOs. Four out of six done well puts you ahead of nearly every data platform I have worked with.

Chapter 19 — Streaming and Batch: Choosing the Right Mode

The mistake almost everybody makes

I have lost count of the number of teams I have watched reach for streaming because it sounded modern, then sit in a meeting six months later wondering why their Dataflow bill is the size of a junior engineer's salary and their on-call rota is on fire. I have made the mistake myself, more than once. The first time was a "real-time customer dashboard" that, when we finally checked, was being looked at by precisely four people, none of whom ever loaded it more than once a day. The second time was a "real-time" fraud feed that the downstream team batched up and re-aggregated hourly anyway, because their model could not tolerate per-event scoring. We had built a sub-minute pipeline to feed a system that was, by design, hourly. The wasted money was embarrassing. The wasted operational complexity was worse.

Streaming is not the modern way to do data. It is one of two modes — streaming and batch — both of which are perfectly modern, both of which have their place, and one of which is dramatically cheaper to build, run, and reason about than the other. The question "should this be streaming?" is genuinely interesting, has a correct answer for any given problem, and is almost never asked. Instead the conversation tends to start with someone in a meeting saying "well obviously this needs to be real-time" and from that moment onwards the technical choice has been made for cultural reasons, not engineering ones.

The rest of this chapter is an attempt to put the conversation back where it belongs. There are four decision factors. They are not all of equal weight, but you should walk through all four before committing.

Data arrival pattern. Do events arrive continuously, or do they arrive in clumps? A Kafka topic with steady traffic is genuinely continuous. A nightly file from a mainframe is not, no matter how fast you process it once it lands. A "real-time API" that emits twelve events between 09:00 and 17:00 and is silent the rest of the day is closer to batch than to streaming, regardless of what the marketing material says.

Downstream latency SLA. What is the worst-case freshness anyone downstream actually needs? Not what the product manager says they want, but what the business will actually notice. A fraud system catching a stolen card needs sub-second. A daily regulatory report needs sub-day, and you have eight hours of overnight window to do

it in. A "Tuesday's executive dashboard" needs to be right by Tuesday morning, not at 03:47 on Tuesday morning. The honest SLA is almost always less stringent than the requested one.

Ordering and deduplication. Do records have a meaningful per-key order? If you have CDC events for a customer record, they must be applied in source order or the final state is wrong. If you have independent click events that aggregate to a count, you can process them in any order and the result is the same. Deduplication has a similar shape: some pipelines tolerate doubles, some break catastrophically on them. The cost of "exactly-once" is high, and you should not pay it where "at-least-once" plus an idempotent sink will do.

Operational cost. Streaming pipelines are 24/7 infrastructure. Dataflow Streaming workers run all the time. BigQuery streaming inserts have a per-row fee. Pub/Sub subscriptions accumulate cost even when idle. Cold infrastructure — a Dataflow batch job that runs for twenty minutes and disappears — is fundamentally cheaper to operate than warm infrastructure that runs forever. The bill is one part of that, but the on-call burden is the bigger part. A streaming pipeline that goes wrong at 02:00 pages someone; a batch pipeline that goes wrong at 02:00 retries at 02:15 and quietly succeeds.

Hold those four factors in your head and we will keep coming back to them.

A decision matrix that is not glib

Here is the matrix I actually use in practice. I have tried to be specific about the ballparks rather than waving my hands.

Dimension	Batch (nightly or hourly)	Micro-batch (every 5–15 min)	Streaming (sub-minute)
Typical use case	Regulatory reports, EOD reconciliation, mainframe ingestion, dbt FDP builds	Operational dashboards, near-real-time analytics, CDC into BigQuery	Fraud detection, real-time personalisation, IoT telemetry, alerting on click streams
GCP services	Cloud Storage, Dataflow batch jobs, BigQuery batch load, Cloud Run jobs, Composer / Cloud Scheduler	Dataflow batch on a tight schedule, BigQuery scheduled queries, dbt incrementals, Pub/Sub with batched pulls	Pub/Sub, Dataflow Streaming, BigQuery streaming inserts, Datastream, Bigtable

Dimension	Batch (nightly or hourly)	Micro-batch (every 5–15 min)	Streaming (sub-minute)
Cost profile (small system)	£50–£300/month for compute; pay only when running	£200–£800/month; longer-lived but not 24/7	£1,500–£6,000/month minimum; Dataflow Streaming worker alone is £250–£400/month before any work
Cost profile (mid system)	£500–£2,000/month	£1,500–£4,000/month	£8,000–£25,000/month, with reservations being the cheaper end
Ops burden	Low. Failures retry. On-call hears about it only after repeat failures.	Medium. Backlogs accumulate; freshness alerts need tuning.	High. Pipelines page. Cold restart is 10+ minutes. Updates are non-trivial.
Exactly-once story	Trivial. Idempotent sinks (BigQuery batch load with `WRITE_TRUNCATE` or merge by key) handle replays cleanly.	Workable. Incremental dbt with a `unique_key` deduplicates; some care needed across the window boundary.	Hard. Requires careful Pub/Sub message-id dedup, Dataflow's exactly-once mode, and BigQuery streaming insert dedup window — all working together.
Recovery story	Re-run the job. Same inputs, same outputs. Backfill is free.	Re-run the affected windows. Slight care needed if downstream has consumed already.	Drain, snapshot, restore, replay from Pub/Sub retained-ack window (default 7 days). Backfill is awkward and often requires a separate batch pipeline.

The numbers above are real ballparks from real GCP bills I have seen, not synthetic. They are sensitive to region, slot reservations, and whether your team has done any optimisation. The point is the relative shape, not the precise digit. Streaming is

roughly an order of magnitude more expensive than batch for an equivalent workload, and roughly twice as expensive again as micro-batch.

The most important row in that table is the one labelled “recovery story”. Streaming pipelines do not gracefully recover from arbitrary failures. They recover from the failures their designers anticipated. A streaming pipeline that has been wrong for six hours and now needs the last six hours reprocessed is not “just rewind the offset” — it is “spin up a parallel batch pipeline that reads the historical Pub/Sub messages, deduplicates against what already landed, and merges back into the same target tables”. Most teams who set out to build streaming-only end up writing that batch pipeline at 03:00 on a Saturday, and at that point they have a Lambda architecture by accident.

Pub/Sub to Dataflow streaming, the canonical pattern

Despite all the caveats, there are real problems for which streaming is the right answer. When it is, the pattern on GCP is well-trodden enough that you should not deviate from it without a strong reason.

The shape is: a publisher writes messages to a Pub/Sub topic; a Dataflow streaming pipeline reads from a Pub/Sub subscription, windows the messages, applies aggregations or enrichments, and writes to a sink (BigQuery streaming insert, Bigtable, or another Pub/Sub topic). The pattern is well-supported, well-documented, and well-instrumented.

A few concepts you must understand before you write a single line of streaming Beam code.

Windowing. A streaming pipeline that aggregates anything needs windows, because “aggregate this unbounded stream” is a question with no answer. Beam supports three window shapes:

- **Fixed windows** divide the stream into non-overlapping intervals of equal size: every minute, every five minutes, every hour. The natural choice for “count events per minute” or “sum amounts per hour”.
- **Sliding windows** overlap: a five-minute window that advances every minute gives you five overlapping windows at any moment, each one containing the last five minutes of data. Useful for rolling averages and “trending” calculations.
- **Session windows** are not fixed in size at all. They group events by a per-key gap of inactivity: “all events for user X that arrived within thirty minutes of each other”. Excellent for things like user-session reconstruction, terrible for things that need fixed reporting boundaries.

Watermarks. A watermark is Beam’s notion of “we believe we have seen all events with timestamps up to this point”. It is heuristic — based on the rate of incoming messages and their event-time skew — and it can be wrong. A watermark advances; events arriving after the watermark for their window are called late.

Triggers. A trigger decides when a window’s contents are emitted downstream. The default (“at watermark”) emits each window once, when the watermark passes its end. Other triggers fire early (so you see partial counts before the window closes), late (so

you re-emit when late data arrives), or accumulating (so each emission re-states the full count rather than incrementing).

Here is a small example doing windowed aggregation with allowed lateness and accumulating mode:

```
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions, StandardOptions
from apache_beam.transforms import window, trigger

options = PipelineOptions()
options.view_as(StandardOptions).streaming = True

with beam.Pipeline(options=options) as p:
    (
        p
        | "ReadFromPubSub" >> beam.io.ReadFromPubSub(
            subscription="projects/my-project/subscriptions/transactions-sub",
            with_attributes=True,
            id_label="message_id",  # Used for dedup
        )
        | "ParseJson" >> beam.Map(lambda m: json.loads(m.data))
        | "KeyByCustomer" >> beam.Map(lambda r: (r["customer_id"], r["amount"]))
        | "FiveMinuteWindow" >> beam.WindowInto(
            window.FixedWindows(5 * 60),
            trigger=trigger.AfterWatermark(
                early=trigger.AfterProcessingTime(60),  # Speculative every minute
               late=trigger.AfterCount(1),             # Re-fire on each late event
            ),
            accumulation_mode=trigger.AccumulationMode.ACCUMULATING,
            allowed_lateness=window.Duration(seconds=15 * 60),
        )
        | "SumAmounts" >> beam.CombinePerKey(sum)
        | "WriteToBigQuery" >> beam.io.WriteToBigQuery(
            table="my-project:analytics.customer_spend_5min",
            schema="customer_id:STRING,amount:FLOAT,window_start:TIMESTAMP",
            write_disposition=beam.io.BigQueryDisposition.WRITE_APPEND,
            method="STREAMING_INSERTS",
        )
    )
```

A handful of things in that snippet are worth being clear about.

The `id_label="message_id"` parameter is what tells Dataflow to deduplicate Pub/Sub messages by a message attribute. Pub/Sub delivers at-least-once; without this, a republished message becomes a double-counted event. The label corresponds to whatever the publisher set as a per-message unique id. If you control the publisher, you set this. If you do not, you are at the mercy of whoever does.

The early trigger emits speculative results every minute so downstream sees the window's current value before it closes. The late trigger re-fires on each late event, so the count gets corrected when stragglers arrive. Accumulating mode means each emission re-states the full count for the window, so the consumer can take the latest

value rather than try to sum increments. Discarding mode emits only the delta since the last firing, which is what you want if your sink is naturally additive.

Allowed lateness is fifteen minutes here. After that, late events for the window are dropped. There is no fully correct value for this — too short and you lose stragglers, too long and watermark progress lags and downstream freshness suffers. Fifteen minutes is a defensible default for many workloads; revise it based on observed lateness in your real data.

Then there is the exactly-once question, which deserves to be said plainly rather than implied through the jargon. Pub/Sub delivers at-least-once. Dataflow has an exactly-once mode that, in combination with `id_label`, will deduplicate by the message id within a Dataflow window. BigQuery streaming inserts have their own dedup window of approximately one minute, keyed on `insertId`. None of these are free; each one adds latency, cost, or complexity. End-to-end exactly-once on this stack is achievable but requires all three layers to cooperate, and it is the responsibility of the pipeline author to get it right, not the platform's responsibility to handle it for you. Anyone who tells you "Dataflow gives you exactly-once" is selling something.

Change Data Capture

Change Data Capture — CDC — is the pattern by which an operational database streams its changes to an analytical store. Instead of running a nightly extract that copies the full table, CDC tails the database's transaction log and emits one event per row mutation: insert, update, or delete. The result is a continuously up-to-date analytical replica with sub-minute lag.

On GCP, the Postgres-to-BigQuery pattern has two well-known shapes.

Datastream is the managed answer. You configure a Datastream stream pointing at your Postgres instance, give it a target (Cloud Storage or BigQuery), and it does the rest: log mining, schema discovery, event emission, and target hydration. Setup is half a day; running cost is modest. The catch is that Datastream is a black box. You do not get to inject transformations between source and target; the BigQuery output schema is what Datastream chooses; reconciliation is what Datastream tells you it is. For most teams, that is fine. For teams who have strong opinions about audit, PII masking, or downstream contract shape, it is too rigid.

Debezium plus Pub/Sub is the DIY answer. A Debezium connector — running in Kafka Connect or, increasingly, as a standalone container on Cloud Run — tails the Postgres logical replication slot, emits change events to a Pub/Sub topic, and a Dataflow streaming pipeline consumes that topic to populate BigQuery. Setup is a fortnight rather than half a day; running cost is comparable for small-to-medium volumes and higher for large ones; flexibility is unlimited. You can transform, mask, enrich, filter, route by tenant, and apply your own schema rules between the source and the target.

The honest position on the framework's own `postgres-cdc-streaming` reference is the one I have already taken in Chapter 17: it is a skeleton. The patterns it sketches — Datastream as the source, a Dataflow streaming consumer, streaming inserts into

a target FDP table, watermark-aware joins — are the right patterns. The code is not yet a deployment you can stand up and trust. A working version is on the roadmap, and I will not pretend otherwise.

Two specific problems CDC always faces, both of which a "real" implementation must solve and which the skeleton does not yet fully solve.

The first is the **ordering problem**. CDC events must be applied in source order per primary key. If you receive `update t=09:00 set status=ACTIVE`, then `update t=09:01 set status=CLOSED`, and you process them in reverse, the row in BigQuery ends up `ACTIVE` even though the source says `CLOSED`. Dataflow's parallelism makes this non-trivial — you cannot serialise the whole stream — so the canonical solution is to key by primary key and use per-key ordering within a window. Pub/Sub's `ordering_key` attribute and Dataflow's keyed state together make this workable, but you have to wire it up deliberately.

The second is the **deletion problem**. When a row is deleted in the source, what do you do in the analytical replica? Two answers. **Soft delete** marks the BigQuery row with a `_deleted_at` timestamp and leaves the row in place. This preserves history for analytics and is reversible. **Hard delete** issues a `MERGE ... WHEN MATCHED THEN DELETE` against BigQuery, which removes the row entirely. Hard delete is what you want for GDPR-style right-to-be-forgotten requests; soft delete is what you want for everything else. The right default is soft delete, with a documented hard-delete path triggered by a separate workflow. CDC events that include `BEFORE` images — the row's state before the change — make this easier because you have the prior values to compare against; events with only `AFTER` images force you to maintain that history yourself.

The Lambda / Kappa debate, briefly

There has been a long-running architectural argument about whether the right way to build an analytical system is **Lambda** — a batch path and a streaming path running in parallel, with reconciliation between them — or **Kappa** — a single streaming path that can be replayed from a retained log for backfills and corrections. The Kappa pitch is simpler: one codebase, one mental model, one set of operational concerns. The Lambda pitch is that batch is genuinely better at some things and streaming is genuinely better at others, so use both.

Having watched a fair number of teams try one or the other, my honest observation is that most real pipelines end up Lambda-ish whether they planned to or not. The reason is backfills. A streaming pipeline that has been running for six months and now needs to reprocess all six months — because a transformation rule changed, or a downstream consumer found a bug, or a regulator asked for a recomputation — cannot do that from the streaming path alone in any reasonable time. Pub/Sub's retained-ack window is seven days by default and a maximum of thirty-one. After that, the events are gone, and your "replay from the log" answer evaporates. So you write a batch pipeline that reads the historical data from cold storage and reconciles into the same target tables, and at that moment you have Lambda whether you call it that or not.

The framework is, accordingly, Lambda-friendly by design. The same FDP table is written by a batch ingestion path and (where a streaming reference exists) a streaming CDC path. Reconciliation between them is one of the open problems and is a meaningful piece of work; today it relies on operator discipline and audit-trail comparison rather than an automated reconciler. I will say more about that in the next section.

The framework's streaming surface

Let me be precise about what does and does not exist today.

`fdp-trigger` is a Cloud Function (optionally a Cloud Run service) that subscribes to FDP completion events on a Pub/Sub topic and publishes a downstream "data ready" event. It is the framework's only production-ready streaming-shaped component, and it does its job. Latency from FDP completion to downstream notification is on the order of seconds. The wiring is simple, the audit trail is honest, and the failure mode (the subscription accumulates messages, the Cloud Function retries) is the standard Pub/Sub pattern.

`BeamPipelineBuilder` in `gcp-pipeline-beam` supports streaming modes — you can pass `streaming=True` to its options, and the underlying read transforms are happy to come from `ReadFromPubSub` rather than `ReadFromText`. The builder's audit-trail wiring is mode-agnostic; the `run_id` discipline that propagates through batch jobs also propagates through streaming ones. So in principle a streaming-shaped Beam pipeline using `BeamPipelineBuilder` is buildable today. In practice, the patterns are not fully fleshed out — the `write_valid` / `write_invalid` / `reconcile` triad assumes a finite envelope, and that assumption breaks for an unbounded stream.

What is missing. Three specific gaps I would call out:

- A **streaming-shaped Composer DAG pattern**. The five-DAG factory is fundamentally batch-oriented: trigger, ingestion, transformation, error-handling, status. A streaming pipeline does not fit that mould — there is no "today's run", there is just "the job is running, here is its watermark, here is its lag". Adapting the orchestrator to manage long-lived streaming jobs (monitor, drain, update, restart) is a non-trivial piece of design work.
- A **streaming reconciliation engine**. The batch reconciliation pattern compares envelope counts (from HDR/TRL) against the rows actually landed in BigQuery. Streaming has no envelope; "expected count" is meaningless for an unbounded stream. The closest analogue is windowed reconciliation: in window W, the source emitted N events and we landed M; if $M \neq N$ within some tolerance, alert. The framework does not yet provide a clean API for that.
- A **streaming dbt story**. dbt is fundamentally batch. The standard advice for streaming-feeding-dbt is micro-batch: have streaming insert into a "raw" landing table at sub-minute latency, then run dbt incrementally every five to fifteen minutes to populate the FDP. That works, it is well-understood, and it is the answer I would give today. Pure streaming dbt does not exist and probably should not.

The honest summary is that the framework today is a credible batch system with a small, well-defined streaming surface (the trigger Cloud Function and the streaming-capable Beam builder). It is not a streaming system that happens to support batch. That gap is not a defect — it is a reflection of where the real demand is — but anyone evaluating the framework for a streaming-first use case should know it.

Operational realities

A few things you only learn after running streaming pipelines in production, often the hard way.

Cold starts are slow. A Dataflow Streaming job that has just been launched typically takes ten to fifteen minutes before it is fully processing at steady state. Workers come up, the SDK harness initialises, the source connection is established, the watermark stabilises. If you are draining and restarting frequently, that ten minutes adds up. Plan for it.

Reservation slots matter for sustained streaming. On-demand Dataflow Streaming pricing is roughly the cost of running the underlying Compute Engine instances plus a Dataflow surcharge. For a pipeline that runs 24/7, BigQuery slot reservations and Dataflow committed-use discounts will save you 20–40% on the compute portion of the bill. The break-even point is around two to three months of continuous operation; below that, on-demand is cheaper because reservations are billed whether you use them or not.

Drain versus cancel. When you stop a streaming pipeline, you can `drain` it — which lets in-flight elements complete and watermarks advance to infinity, so downstream sees clean end-of-window emissions — or you can `cancel` it, which kills the pipeline immediately and leaves any in-flight work unfinished. Drain is what you want for graceful updates. Cancel is what you want when the pipeline is broken and you do not care about the in-flight work. Mixing them up loses data.

The `--update` flag. Dataflow Streaming supports rolling updates via `gcloud dataflow jobs update` (or the equivalent Beam pipeline option). You launch the new pipeline, it adopts the old pipeline's state and watermarks, and the cutover is seamless. The catch is that not every pipeline change is update-compatible. Adding a new source, changing a key type, removing a `PTransform`, changing a window's size — these all break the update path and force a full drain-and-restart. The compatibility rules are documented but easy to fall foul of. A useful discipline is to test every code change with a simulated update against a dev pipeline before pushing to production.

Backpressure has a specific shape in metrics. When a Dataflow Streaming pipeline cannot keep up with its input, Pub/Sub's subscription backlog grows, the `oldest_unacked_message_age` metric climbs, and the Dataflow worker pool either autoscales up (if you have headroom) or stalls (if you have hit your max workers). The first sign in dashboards is the subscription age. The second is downstream lag. By the time downstream consumers notice their data is stale, the backlog is usually hours old. Alerting on `oldest_unacked_message_age > 5 minutes` catches this before customers do.

On-call burden. This is the one most teams underestimate. A batch pipeline that fails at 02:00 fails silently — its retry succeeds at 02:15 — and the on-call engineer hears about it only on their morning Slack catch-up. A streaming pipeline that fails at 02:00 pages someone, because backlogs do not retry themselves and freshness alerts trigger immediately. Over a year, a single streaming pipeline can be the difference between a quiet on-call rota and a noisy one. Two or three streaming pipelines, run by a small team, will dominate the rota. This is the real cost of streaming that does not appear on any GCP invoice.

A recommendation framework

The decision tree I actually walk through, for anyone asking me whether their next pipeline should be streaming, is short.

```
Is your real latency SLA > 1 hour AND
does data arrive in files (not as events)?
    → Batch. Composer schedule, Dataflow Flex Template,
      dbt nightly. This is the cheapest, most reliable option.

Is your real latency SLA < 1 minute AND
does data arrive as a continuous event stream
(Pub/Sub, Kafka, CDC log, IoT telemetry)?
    → Streaming. Pub/Sub → Dataflow Streaming →
      BigQuery streaming insert + sink-specific dedup.

Anything else (the vast middle ground)?
    → Micro-batch. Dataflow batch on a 5–15 minute
      schedule, dbt incrementals after each load.
      Cheapest by a wide margin for the same freshness
      perception. The right default for "near-real-time"
      analytics.
```

If you ignore everything else in this chapter, take that tree. The middle case — micro-batch — is the one almost everybody underuses. It gives you most of the freshness of streaming at a fraction of the cost, an order of magnitude less operational pain, and an inventory of GCP services that is shorter and cheaper to run. It is rarely the answer people reach for first, because “micro-batch” sounds less impressive than “real-time”, but if you measure on engineering cost per business outcome it is usually the winner.

The chapter that follows looks at the same trade-off from a different angle — how to evolve a pipeline that started as batch and now needs to be more frequent. That is the path the majority of real systems actually walk, in my experience, and it has its own set of mistakes to avoid.

Key takeaways

- Streaming is roughly an order of magnitude more expensive than batch for an equivalent workload, and roughly twice as expensive again as micro-

batch. The break-even point where streaming is the right answer is narrower than most teams assume — sub-minute SLA on continuous event streams, not "we want it to feel responsive".

- End-to-end exactly-once on the Pub/Sub → Dataflow → BigQuery stack is achievable but requires three layers to cooperate: `id_label` on the Pub/Sub read, Dataflow's exactly-once mode, and BigQuery streaming-insert dedup. None of it is free, none of it is automatic, and "the platform handles it" is not true.
- Micro-batch is the underused middle ground. Dataflow batch on a 5–15 minute schedule plus dbt incrementals gives you most of the freshness of streaming at a fraction of the cost and operational burden. It is the right default for anything that does not have a genuine sub-minute SLA.
- CDC is two specific problems disguised as one: per-key ordering (which Pub/Sub's `ordering_key` and Dataflow keyed state can solve) and deletions (where soft-delete with a documented hard-delete path is almost always the right policy). The framework's `postgres-cdc-streaming` reference sketches the right patterns but is not yet a deployment you can trust — a working version is on the roadmap.
- Most real pipelines end up Lambda-ish whether they planned to or not, because backfills past Pub/Sub's retained-ack window force a batch path to exist anyway. Plan for both from the start rather than discovering it at 03:00 on a Saturday.
- The real cost of streaming is on-call burden, not the GCP bill. Batch pipelines retry; streaming pipelines page. Two or three streaming pipelines will dominate a small team's rota. Choose streaming knowing that, not despite it.

Chapter 20 — Setting Up Your GCP Environment: Projects, Networks, Kubernetes, and the Engineer Skill Set

Why this chapter exists

The book up to this point has told you what to build. It has told you why HDR/TRL envelopes matter, how to wire a DAG factory, how to argue with Composer on cost grounds, and how to put the whole thing on your own Kubernetes cluster if you must. It has not told you any of the things you need to know to actually *do* that on a Monday morning, with an empty GCP organisation and a vague mandate from a director to "get the data platform off the mainframe by Q4".

The gap is real. I have lost count of the times I have walked an architect through the framework, watched their eyes light up at the audit trail and the cost-tracking, and then watched them go quiet thirty seconds later when I said the words "shared VPC host project". They are not stupid. They are senior. They simply have not had to do the GCP plumbing themselves, because in their last role someone else had already done it.

This chapter is that someone-else's runbook, compressed. It covers the project topology you almost certainly want, the network you will probably regret if you do not lay it out properly the first time, the Composer switches you actually need to pass, the Kubernetes namespace model the framework uses, and a candid skills primer for Docker, Kubernetes, Helm, and kustomize. It is not exhaustive. It is the on-ramp.

If you are already running a production GCP estate with a shared VPC, Terraform-managed projects, and a working GKE fleet, skip to Chapter 22. This chapter is for the engineer who has done the eight previous chapters of pipeline work in a sandbox project and is now staring down the question *"how do I do this for real?"*.

GCP project topology

The shape

Almost every successful GCP data estate I have seen converges on the same shape: one project per environment, per system, with separate host projects for shared networking and shared artefacts.

For the `acme` corporation running this framework's `generic` system, that gives you:

```
acme/  (organisation)
└── data-platform/  (folder)
    ├── acme-net-prod/             (host project: shared VPC, DNS, NAT)
    ├── acme-net-nonprod/          (host project: shared VPC for dev + stg)
    ├── acme-artifacts-prod/       (Artifact Registry, container images)
    ├── acme-dataplatform-dev/     (service project)
    ├── acme-dataplatform-stg/     (service project)
    ├── acme-dataplatform-prod/    (service project)
    └── acme-warehouse-prod/       (BigQuery datasets, dbt target)
```

The folder gives you a place to apply organisation policies — "no public IPs on Compute", "Cloud Storage requires CMEK" — once, and have them inherited by every project beneath. Each project has its own billing account or sub-account, its own IAM, and its own quota.

The split between *host* and *service* projects is the Shared VPC pattern. A **host project** owns the VPC; one or more **service projects** are attached to it and run workloads that consume that VPC. You do this because networking is a shared concern — DNS zones, firewall rules, peering to on-prem, NAT egress — and you want it managed by one team (typically platform engineering) while data engineers, ML engineers and application teams own their service projects independently.

The warehouse lives in its own project, separate from the pipeline that loads it. This is for blast-radius reasons: a runaway Dataflow job in `acme-dataplatform-dev` cannot accidentally drop a BigQuery table in `acme-warehouse-prod` because the service account it uses has no permission there. The dbt-runner reaches into the warehouse project with a cross-project grant; nothing else does.

Artefacts — container images, Helm charts, Python wheels — sit in `acme-artifacts-prod`. Every environment pulls from the same registry. You do not want to be in the position of explaining why the staging image is different from the production image because they were built in different projects with different toolchains.

Two practical ways to create the projects

The first time you do this, `gcloud projects create` is fine:

```
gcloud projects create acme-dataplatform-dev \
  --folder=123456789012 \
  --name="Acme Data Platform (dev)"

gcloud beta billing projects link acme-dataplatform-dev \
  --billing-account=01ABCD-EF2345-6789AB
```

By the third project, you will be tired of typing. The conventional answer is the `terraform-google-modules/project-factory` module, which lets you declare projects, billing links, API enablement, and default IAM bindings in a single Terraform stanza:

```
module "dataplatform_prod" {
  source  = "terraform-google-modules/project-factory/google"
  version = "~> 16.0"

  name                = "acme-dataplatform-prod"
  org_id              = var.org_id
  folder_id           = google_folder.data_platform.id
  billing_account     = var.billing_account
  shared_vpc          = "acme-net-prod"
  shared_vpc_subnets  = [
     "projects/acme-net-prod/regions/europe-west2/subnetworks/dataplatform-
prod",
  ]
  activate_apis = [
    "compute.googleapis.com",
    "container.googleapis.com",
    "composer.googleapis.com",
    "dataflow.googleapis.com",
    "bigquery.googleapis.com",
    "storage.googleapis.com",
    "secretmanager.googleapis.com",
    "artifactregistry.googleapis.com",
    "iam.googleapis.com",
  ]
}
```

That single module creates the project, links billing, attaches it as a Shared VPC service project, and enables every API you need. Run it three times with different `name` and `folder_id` values and you have dev, stg, prod ready to receive workloads.

A small piece of advice from twenty-five years of doing this: name your projects after their *role*, not their *contents*. `acme-dataplatform-prod` survives a re-org; `acme-customer-360-prod` becomes a lie the day the Customer 360 programme is renamed. Roles are stable; product names are not.

Networking: VPCs, subnets, private services

This is the section people skip, then regret six weeks later when they cannot make Composer talk to Cloud SQL because the PSA range collides with on-prem.

Why a custom VPC, not `default`

Every GCP project starts with a network called `default`. It has a subnet in every region. It has permissive firewall rules. It cannot be attached as a Shared VPC. It

auto-creates new subnets when new regions launch. None of those properties are what you want in a production data platform.

The first thing you do in a host project is delete the default network and create a custom one:

```
resource "google_compute_network" "vpc" {
  name                    = "acme-data-vpc"
  auto_create_subnetworks = false
  routing_mode            = "REGIONAL"
}
```

`auto_create_subnetworks = false` means you choose every subnet by hand. `routing_mode = "REGIONAL"` means traffic stays in a region unless you explicitly route it otherwise — which is what you want for cost and latency.

Choosing CIDRs that will not bite you

Subnet CIDR planning is one of those things that feels pedantic until it stops you deploying. The constraint is that every range — primary subnet, GKE pod range, GKE service range, Composer's tenant range, Cloud SQL's PSA range — must not overlap with anything else, including your on-prem networks if you ever peer them.

A workable allocation for a data platform in `europe-west2`:

Range	CIDR	Used for
`10.20.0.0/22`	1,024 IPs	Primary subnet (GKE nodes, Composer nodes, GCE)
`10.30.0.0/14`	262,144 IPs	GKE pod secondary range (`/14` per cluster minimum for production)
`10.34.0.0/20`	4,096 IPs	GKE service secondary range
`10.40.0.0/22`	1,024 IPs	Composer tenant range (Composer needs `/22` minimum)
`10.50.0.0/16`	65,536 IPs	Private Services Access (Cloud SQL, Memorystore)
`10.60.0.0/28`	16 IPs	GKE master authorised range (private cluster control plane)

Two non-obvious points. First, GKE pods consume IPs aggressively because each node gets a `/24` slice of the pod range by default — a 100-node cluster eats 25,600 pod IPs without breaking sweat. If you give it a `/16`, you cap the cluster at about 250 nodes; a `/14` lets it grow. Second, Composer 2 will refuse to create with anything

tighter than a `/22` for its tenant range; I have watched a senior engineer spend two hours debugging `INSUFFICIENT_IP_SPACE` errors because someone hard-coded `/24`.

Cloud Router and Cloud NAT

Your pipeline workers — Dataflow VMs, GKE pods, Composer workers — should not have public IPs. They should reach Google APIs over a private route and reach the public internet (PyPI, package registries, third-party APIs) through a Cloud NAT.

```
resource "google_compute_subnetwork" "primary" {
  name                     = "dataplatform-prod"
  ip_cidr_range            = "10.20.0.0/22"
  region                   = "europe-west2"
  network                  = google_compute_network.vpc.id
  private_ip_google_access = true

  secondary_ip_range {
    range_name    = "gke-pods"
    ip_cidr_range = "10.30.0.0/14"
  }
  secondary_ip_range {
    range_name    = "gke-services"
    ip_cidr_range = "10.34.0.0/20"
  }
}

resource "google_compute_router" "router" {
  name    = "data-router"
  region  = "europe-west2"
  network = google_compute_network.vpc.id
}

resource "google_compute_router_nat" "nat" {
  name                               = "data-nat"
  router                             = google_compute_router.router.name
  region                             = "europe-west2"
  nat_ip_allocate_option             = "AUTO_ONLY"
  source_subnetwork_ip_ranges_to_nat = "ALL_SUBNETWORKS_ALL_IP_RANGES"
}
```

`private_ip_google_access = true` is the toggle that lets a worker without a public IP still reach `bigquery.googleapis.com` over Google's internal backbone. It costs nothing, it is on the same router, and it is the difference between "I can deploy this" and "every Dataflow job hangs at startup because it cannot reach the API".

Private Services Access

Composer's metadata database, Cloud SQL, and Memorystore all run in tenant projects owned by Google. They reach into your VPC over **Private Services Access** (PSA) — a VPC peering with a CIDR range you allocate up front:

```
resource "google_compute_global_address" "psa_range" {
  name          = "google-managed-services"
  purpose       = "VPC_PEERING"
  address_type  = "INTERNAL"
  prefix_length = 16
  address       = "10.50.0.0"
  network       = google_compute_network.vpc.id
}

resource "google_service_networking_connection" "psa" {
  network                 = google_compute_network.vpc.id
  service                 = "servicenetworking.googleapis.com"
  reserved_peering_ranges = [google_compute_global_address.psa_range.name]
}
```

Allocate this *once*, sized generously — a `/16` is fine — and never touch it again. Shrinking a PSA range later is a multi-day migration; growing it is fiddly. Pay the IP-space cost now.

Composer environment creation

If you have decided you actually need Composer — and Chapter 9 has the long argument about when you do not — this is what the create call looks like in practice:

```
gcloud composer environments create acme-orchestrator-prod \
  --project=acme-dataplatform-prod \
  --location=europe-west2 \
  --image-version=composer-2.9.7-airflow-2.10.0 \
  --environment-size=small \
  --node-count=3 \
  --machine-type=n1-standard-2 \
  --network=projects/acme-net-prod/global/networks/acme-data-vpc \
  --subnetwork=projects/acme-net-prod/regions/europe-
  ↪  west2/subnetworks/dataplatform-prod
  ↪  \
  --cluster-secondary-range-name=gke-pods \
  --services-secondary-range-name=gke-services \
  --service-account=composer-runner@acme-dataplatform-prod.iam.gserviceaccount.com
  ↪  \
  --env-variables=PIPELINE_ENV=prod,SYSTEM_ID=generic \
  --enable-private-environment \
  --enable-private-endpoint=false
```

That is most of what matters. `--image-version` pins both the Composer release and the Airflow version. `--environment-size=small` plus three `n1-standard-2` workers is the bottom of the range — somewhere between £300 and £450 a month, depending on region and reserved-capacity discounts, *before you schedule a single DAG*. Larger sizes (medium, large) climb to a thousand or two thousand a month quickly.

`--enable-private-environment` keeps the workers off the public internet (they egress through your Cloud NAT). `--enable-private-endpoint=false` keeps the GKE control

plane reachable on a public IP for `kubectl` access from your developer machines — flip it to `true` if your security team requires control-plane access only from the VPC, and be ready to use an IAP tunnel or jump box.

This is also where the framework's "opt-in Composer" default matters. The reference Terraform in `infrastructure/terraform/systems/generic/variables.tf` defaults `enable_composer = false`. You have to consciously flip it. The framework's authors have done this on purpose: too many teams have woken up to a £500 Composer bill on a sandbox project nobody is using. If you do not need Airflow on day one, do not turn it on.

Kubernetes namespace and access model

Whether you are running Composer (which gives you a GKE cluster underneath, hidden but real) or your own GKE cluster for the umbrella chart on Chapter 14, you will care about namespaces.

Namespaces in this framework

The framework's umbrella chart `pipeline-system` deploys into a single namespace, conventionally called `pipeline-system`, with the standard Kubernetes namespaces (`default`, `kube-system`, `kube-public`) left alone. The pattern is:

- `kube-system` — cluster operators, CNI, DNS. Do not deploy workloads here.
- `default` — empty, by convention. If you find workloads here, somebody forgot a `--namespace` flag.
- `pipeline-system` — the framework's components: Airflow, dbt-runner, beam-runner, observability stack.
- `pipeline-data` — optional sidecar namespace for shared infrastructure (Cloud-SQL proxy, secrets manager CSI driver) that the framework consumes but does not own.

For multi-tenant deployments, the framework's chart values let you suffix the namespace per tenant (`pipeline-system-acme`, `pipeline-system-globex`) and deploy multiple copies into the same cluster. That pattern is less common than namespace-per-environment, but it is supported.

Workload Identity, the only sensible auth

The right way to give a pod access to GCP is **Workload Identity**: bind a Kubernetes service account (KSA) to a Google service account (GSA) such that the pod can call `bigquery.googleapis.com` as the GSA without ever holding a JSON key. The framework's charts do this for you, but you need to know the shape:

```
# Bind the KSA to the GSA
gcloud iam service-accounts add-iam-policy-binding \
  composer-runner@acme-dataplatform-prod.iam.gserviceaccount.com \
  --role=roles/iam.workloadIdentityUser \
  --member="serviceAccount:acme-dataplatform-prod.svc.id.goog[pipeline-
  ↪  system/airflow-worker]"
```

```
# Annotate the KSA
kubectl annotate serviceaccount airflow-worker \
  --namespace pipeline-system \
  iam.gke.io/gcp-service-account=composer-runner@acme-dataplatform-
  ↪  prod.iam.gserviceaccount.com
```

The KSA `airflow-worker` in namespace `pipeline-system` now impersonates the GSA `composer-runner@…` when it calls Google APIs. No JSON keys, no secrets in the cluster, no rotation problem. This is the model. Use it.

RBAC: what you actually need

Kubernetes RBAC is overwhelming if you treat the documentation as a checklist. In practice, for a data team running this framework, you need three roles:

- **`view`** — read-only. Every team member gets this by default in non-production clusters. Lets them `kubectl get pods` and `kubectl logs`, nothing more.
- **`edit`** — create and modify workloads, but not RBAC. The role for developers in dev and stg clusters.
- **A custom `namespace-admin` role** — `edit` plus permission to manage `RoleBinding` inside one namespace, but no cluster-wide rights. This is what an on-call lead gets in production.

Cluster-admin should be held by humans only via emergency break-glass and by the Terraform deploy service account. Anyone who tells you their dev team should all be cluster-admin in production is going to have an interesting weekend at some point.

The skills primer

This is the candid bit. If you have not done much GCP infrastructure work, the previous twenty chapters of this book have used terms that you have probably learned to pattern-match on without quite understanding. Here is the minimum mental model for each, in the order you will hit them.

Docker

What it is. A tool for building, distributing, and running container images. A container image is a tarball of a root filesystem plus a manifest telling the kernel how to run it.

The mental model that clicks. *An image is an immutable artefact, like a JAR or a wheel.* You build it once, you tag it, you push it to a registry, and from that point on it is a fixed reference. A running container is a *process* with that image as its filesystem. The image does not change; the container is ephemeral. If your mental model is "Docker is like a virtual machine", you will keep getting surprised. If it is "Docker is a way to package a process and its dependencies as a hash", you will not.

The three commands you use 95% of the time:

```
docker build -t
 ↪  europe-west2-docker.pkg.dev/acme-artifacts/pipelines/dbt-runner:1.0.29 .
docker run --rm -it
 ↪  europe-west2-docker.pkg.dev/acme-artifacts/pipelines/dbt-runner:1.0.29 dbt run
docker push europe-west2-docker.pkg.dev/acme-artifacts/pipelines/dbt-runner:1.0.29
```

Where the framework uses it. Two main places: the Dataflow Flex Template images under `templates/` (one per ingestion pipeline) and the dbt-runner image under `infrastructure/k8s/dbt-runner/Dockerfile`. Both are built in CI by `deploy-generic.yml` and pushed to Artifact Registry. Chapter 14 also references the custom Airflow image under `infrastructure/k8s/airflow/Dockerfile` when you self-manage.

Kubernetes

What it is. A system for running containers across a cluster of machines, with strong opinions about declarative state and self-healing.

The mental model that clicks. *You describe desired state in YAML; controllers reconcile reality to match.* You do not "start a pod"; you submit a Deployment manifest that says "I want three replicas of this image with this much CPU", and a controller notices reality has zero replicas, so it creates three. If one dies, the controller notices the gap and replaces it. *Imperative thinking — "run this, then run that" — is the wrong model for Kubernetes and will hurt you. Declarative thinking — "this should always be true" — is the right one.*

The three commands you use 95% of the time:

```
kubectl apply -f manifest.yaml          # make this YAML true
kubectl get pods -n pipeline-system     # what is running
kubectl logs -n pipeline-system airflow-worker-abc123 -c worker --tail=200
```

Where the framework uses it. Chapter 14 is the full story. The umbrella chart `pipeline-system` deploys Airflow, dbt-runner, beam-on-Flink, and the observability stack into a GKE (or other CNCF-conformant) cluster. Even when you use Composer, Composer is running Kubernetes under the bonnet — `kubectl get pods -A` on the underlying cluster shows you Airflow's scheduler, triggerer, and worker pods.

Helm

What it is. A package manager and templating engine for Kubernetes manifests. A Helm chart is a directory of templated YAML plus a `values.yaml` that fills in the blanks.

The mental model that clicks. *Helm is parameterised templates for Kubernetes.* You write the manifest *once*, with `{{ .Values.replicas }}` in place of the literal number, and you ship a base `values.yaml` with `replicas: 3`. To deploy to staging with two replicas, you supply a `values-staging.yaml` that overrides just that line. The chart is the shape; the values are the configuration.

The three commands you use 95% of the time:

```
helm template pipeline-system charts/pipeline-system --values values-prod.yaml >
 ↪  out.yaml
helm install pipeline-system charts/pipeline-system --values values-prod.yaml -n
 ↪  pipeline-system
helm upgrade pipeline-system charts/pipeline-system --values values-prod.yaml -n
 ↪  pipeline-system
```

`helm template` is the one most people skip and shouldn't. It renders the chart to plain YAML without applying it, so you can read what Helm *will* do before you let it do anything. I run it every single time before an upgrade in production. It has saved me from at least four bad releases.

Where the framework uses it. The umbrella chart `pipeline-system` under `infrastructure/k8s/charts/pipeline-system/` composes three sub-charts: `pipeline-dbt-runner`, `pipeline-beam-runner`, `pipeline-observability`. Each sub-chart has its own `values.yaml` with sensible defaults; the umbrella's `values.yaml` exposes the knobs you actually flip. There is a base `values.yaml` and per-environment overrides — `values/dev.yaml`, `values/stg.yaml`, `values/prod.yaml` — that override only the lines that differ. The hierarchy is: chart defaults < umbrella values < environment values < command-line `--set`.

kustomize

What it is. An alternative to Helm. A tool for layering *patches* over a base set of manifests, without templating.

The mental model that clicks. *kustomize is patches, not templates.* Where Helm asks you to parameterise a single template, kustomize asks you to declare a base set of plain Kubernetes manifests, then a set of overlays that patch specific fields. There are no `{{ ... }}` placeholders; there is a `kustomization.yaml` that says "take the base, then change replicas to 2 in this Deployment".

The one command:

```
kubectl apply -k overlays/prod/
```

Where the framework uses it. It does not. The framework is Helm-first because the umbrella-of-sub-charts pattern is what Helm was designed for. I mention kustomize here because (a) some of your colleagues will prefer it, particularly anyone who has been burned by gnarly Helm templates with seventeen levels of conditional, and (b) you will eventually meet kustomize in the wild — it ships in `kubectl` itself, and most Argo CD setups support both. If you have inherited a kustomize estate, you do not need to rewrite it to use this framework; you can wrap the framework's rendered Helm output as a base and patch from there. Not pretty, but it works.

The engineer skill set, honestly

A data engineer joins your team next Monday. They will be productive in week one if they already know:

- **Python.** Comfortable, not expert. You will write Beam transforms, custom Airflow operators, small CLI utilities. You will read a lot more than you write.
- **SQL.** Properly. Window functions, CTEs, the difference between `LEFT JOIN` and `LEFT JOIN LATERAL`. dbt does not let you cheat on SQL.
- **Basic Apache Beam.** The map / filter / group / combine vocabulary. You do not need to have shipped a streaming pipeline; batch is fine.
- **Basic Airflow.** What a DAG is, what an operator is, the difference between a sensor and a task. You do not need to have built one from scratch.
- **Basic dbt.** Models, sources, tests, materialisations. A week with the dbt docs is enough.
- **Basic `gcloud`.** Setting a project, authenticating, listing resources. The fluent version comes with use.

They will become genuinely productive — owning their own pipeline, on-call for it, deploying it — in month one if they pick up:

- **Terraform.** Variables, modules, state, plan, apply. The framework's infrastructure code is the curriculum.
- **Helm.** Chart anatomy, values hierarchies, template rendering. A weekend with the umbrella chart's `values.yaml` and `helm template` is the right way in.
- **Kubernetes basics.** Pods, Deployments, Services, ConfigMaps, Secrets. The mental model in the previous section is enough to start.
- **GitHub Actions.** Reading the eleven workflows in `.github/workflows/` and understanding what gates what. Chapter 15 is the long version.

These are *nice to have* but no blocker for hiring:

- **Go.** Useful for reading Kubernetes operator code if you ever debug one, but very few data engineers write Go day-to-day. Do not screen for it.
- **Cloud Build internals.** The framework uses GitHub Actions, not Cloud Build. If you have it, lovely; if not, you will not notice.
- **Advanced networking.** Routing, peering, transit gateways. The platform team owns the host project; the data engineer consumes the subnet name. Do not expect every data engineer to draw a VPC diagram from memory.
- **Java.** The Beam Python SDK is sufficient for everything in this book. Java is a slight performance win for Dataflow but a slight maintainability loss; the trade is rarely worth it for a small team.

If you are an engineer reading this *as* the new joiner: do not panic. The first four bullets in the "week one" list are the load-bearing ones. Everything else you can learn on the job, and the framework is structured to teach you as you go.

Linking back to the framework

Each skill above has a specific place in the framework where it stops being abstract:

- **Docker** — when you build a Dataflow Flex Template, the Dockerfile under `templates/<pipeline>/Dockerfile` is your first real Docker file. The dbt-runner Dockerfile under `infrastructure/k8s/dbt-runner/` is the second.

- **Kubernetes** — Chapter 14 walks through `helm install pipeline-system --values values/prod.yaml` and what it puts on your cluster. That is the first time you will need a working `kubectl`.
- **Helm** — `infrastructure/k8s/charts/pipeline-system/` is the umbrella chart. `infrastructure/k8s/charts/pipeline-system/values.yaml` is where you spend most of your Helm time.
- **Terraform** — `infrastructure/terraform/systems/generic/` is your first whole-system Terraform read. The `variables.tf` here is where `enable_composer = false` lives.
- **gcloud** — `scripts/gcp/01_enable_services.sh` through `07_cleanup.sh` are your `gcloud` masterclass; read them in order.
- **Workload Identity** — `scripts/gcp/setup_github_actions.sh` is the canonical example of binding a KSA-equivalent (a GitHub OIDC identity) to a GSA.
- **Composer** — Chapter 9 covers when to use it; the Terraform under `infrastructure/terraform/systems/generic/orchestration/` is what flips it on, gated by `var.deploy_composer`.

If you read those seven files and chapters together, you will have a working mental model of the entire framework's environment surface in an afternoon. That is the goal.

Key takeaways

- Project topology converges on the same shape almost everywhere: one folder per data platform, separate host projects for shared VPC, separate service projects per environment (dev, stg, prod), separate projects for artefacts and for the warehouse. Use the project-factory Terraform module by the third project; `gcloud projects create` is fine for the first two.
- Custom VPC, never `default`. Plan CIDRs once, generously: `/22` for the primary subnet, `/14` for GKE pods, `/22` minimum for the Composer tenant range, `/16` for Private Services Access. Private Google Access plus Cloud NAT is the standard egress shape — workers without public IPs reach Google APIs over the backbone and the public internet through NAT.
- Composer is £300–£450 per month before you schedule a DAG. The framework's default is `enable_composer = false` for a reason. Flip it on only when you can defend the spend.
- The Kubernetes namespace model is simple: leave `default` and `kube-system` alone, deploy the umbrella chart into `pipeline-system`, give pods access to GCP via Workload Identity (KSA bound to GSA — no JSON keys), and keep RBAC to three roles: `view`, `edit`, and a custom `namespace-admin`.
- The four mental models that unlock everything: Docker is an immutable artefact; Kubernetes is declarative state with controllers reconciling; Helm is parameterised templates for Kubernetes; kustomize is patches over a base. If you internalise those four sentences, the tooling stops feeling like a foreign language.

- For hiring: Python, SQL, basic Beam, basic Airflow, basic dbt, basic `gcloud` are the week-one bar. Terraform, Helm, Kubernetes basics, and GitHub Actions are month-one. Go, Cloud Build internals, and advanced networking are nice-to-have, not blockers.

Chapter 21 — Working with AI Coding Agents on Pipeline Work

I have rewritten this chapter four times. Each rewrite happened because something on the agent side changed — a new model, a new IDE integration, a new way of letting an agent run shell commands. By the time you read it I would not be surprised if some of the product names are quaint. So before anything else, let me say what this chapter is and is not.

It is **not** a tour of vendors. It is **not** a benchmark of which model writes the cleanest dbt SQL today. It **is** a working data engineer's account of how to drive AI coding agents on the kind of pipeline work this book has been about — the schema-as-source-of-truth, dbt-shaped, audit-trailed work — and what reliably goes wrong.

I am writing it because pretending the world did not change would be ridiculous. In 2026, most of the teams I work with are using agents for routine pipeline work every day. Skipping the topic in a book on production data pipelines would be like writing a book on web development in 2014 and not mentioning Git. The interesting and slightly embarrassing thing is that the framework I have spent twenty-one chapters describing turns out, almost entirely by accident, to be unusually well-shaped for agent work. We did not design it that way. We designed it for human auditors and tired on-call engineers. It just so happens that an LLM and a tired on-call engineer want roughly the same things: a single source of truth, predictable file layouts, named patterns, and a CI that fails loudly.

This chapter is, therefore, half pragmatic guide and half candid admission that we lucked out.

The four agent shapes you'll meet

You will, in practice, encounter four distinct shapes of agent on pipeline work. They are good at different things, bad at different things, and you should not pick one and use it for everything.

IDE-coupled agents. Cursor, Claude Code's editor integration, GitHub Copilot Workspace, Cody, JetBrains AI. These live inside your editor, see your open files, and can edit them in place. They are excellent for the inner loop — "here is a half-written dbt model, finish it", "rename this variable everywhere", "add a `not_null` test for this column". They have a privileged view of the small piece of the repo you currently care

about, and a hopeless view of everything else. They will, given the chance, happily invent imports from modules that do not exist in this repo's `requirements.txt`. The framework's `EntitySchema` import-and-validate flow rescues them from themselves on the imports they care about most.

CLI / sandbox agents. Claude Code in the terminal, Aider, OpenAI's `codex` CLI, the long tail of TUI agents people are building this year. These run as a shell session you can leave alone for an hour. They are the right tool when the work is a multi-step refactor: "read these five SQL files, generate five dbt models, generate the schema YAML, run `dbt compile`, fix the errors, run `dbt test`, report back." They are bad at anything that needs a human judgement call — "is this the right grain?" — and you have to be religious about reading the diffs they produce. I treat their output exactly like a PR from an enthusiastic graduate: enthusiastic, mostly right, occasionally confidently wrong in a way that would take down production.

Platform-embedded agents. Vertex AI Code Assist, BigQuery's SQL completion, dbt Cloud's "explain this model" assistants. These have the narrowest scope and the deepest integration. Vertex's SQL completion knows your actual schema; it does not hallucinate column names the way a generic chat model does. Use them for what they are: smart autocomplete inside one tool. Do not try to drive an end-to-end refactor through a BigQuery query editor.

Your own agent, built on the model APIs. Anthropic's, OpenAI's, Google's. Some teams I work with have built small in-house agents that run on a schedule, read the audit trail, and file PRs that fix obvious issues — a missing `not_null` test, a missing PII flag on a column matching a known regex. This is the most powerful shape and the most dangerous one.

What all four have in common: they work much better when the codebase has a clear structure they can pattern-match against. The framework's layered conventions — `models/staging/`, `models/fdp/`, `models/marts/`, `EntitySchema` at the root of every entity — give an agent a lot to grip on. A pipeline repo that is a tangle of one-off SQL scripts and per-DAG bespoke Python is a much harder thing to point an agent at.

The single most important prompting principle

If you take one thing from this chapter, take this: **give the agent the schema, not just the SQL**.

Here is a bad prompt:

```
Write a dbt model for customer addresses.
```

Here is a good one:

```
Add a new MAP entity called `customer_addresses` mapped from
the `mainframe_customer_addr` ODP table. The source-of-truth
schema is at `gcp_pipeline_schemas/customer.py` — read it and
treat the field types, PII flags, and required flags as binding.

Generate:
```

```
1. The `EntitySchema` for `customer_addresses` in
   `gcp_pipeline_schemas/customer_addresses.py`.
2. A dbt model at `models/fdp/customer_addresses.sql` following
   the MAP pattern in `models/fdp/portfolio_account_excess.sql`.
   Use `gcp_pipeline_transform.generate_audit_columns()`.
   Use `gcp_pipeline_transform.apply_pii_masking()` for every
   field marked `pii=True` in the schema.
3. A `schema.yml` entry with `unique` and `not_null` on the
   declared primary key.
4. A test fixture at `tests/fixtures/customer_addresses.csv`
   using `gcp_pipeline_tester.RowFactory` to derive ten rows.
```

The first prompt produces something that looks like a dbt model and is structurally wrong in every way that matters: invented column names, no audit columns, no PII masking, no test fixture, materialisation choice picked from the training distribution rather than from your conventions.

The second prompt produces something that, in my experience, lands in the right shape eighty per cent of the time. The agent is not smarter for the second prompt. The second prompt has done the agent's hardest job — finding the right context — for it. The framework's schema discipline pays off here in a way it does not anywhere else: there is exactly one place to point the agent at, and that place is authoritative.

A related point: do not ask the agent to "look at" something without telling it the path. "Look at the customers schema" is ambiguous; "read `gcp_pipeline_schemas/customer.py`" is not. Agents are bad at the kind of fuzzy reference resolution humans take for granted, and they will silently invent rather than ask.

A worked example: five validated joins to a productionised FDP

This is the example a reader specifically asked for, and it is one I have run with three different teams now. The setup is real and the prompts below are the ones I actually used, lightly anonymised.

The setup. An analyst — call her Ravi — has spent two weeks in the staging BigQuery dataset working out how to combine five ODP tables into a single per-customer view. She has five SQL files. Each one is a JOIN she has validated against a known-good control file the source-of-truth team produced. She has signed them off. Her job ends here; yours, as the data engineer, begins.

Your job is to turn those five SQL files into a productionised FDP. That means: incremental where it can be, audited, PII-masked, tested, scheduled, alerted on, and reviewable. The agent is going to do the bulk of the typing. You are going to make the judgement calls.

Prompt 1 — understand the shape

```
Read the five SQL files in `analyst-handoff/ravi/2026-05/`:
```

```
  - join_01_customer_to_address.sql
  - join_02_customer_to_account.sql
  - join_03_account_to_transaction.sql
  - join_04_account_to_decision.sql
  - join_05_customer_to_marketing_consent.sql

Also read the EntitySchema for every table referenced, under
`gcp_pipeline_schemas/`.

Then answer three questions:
  1. What is the lowest-common-denominator grain across the five
     joins? (i.e. one row per customer, per account, per event?)
  2. Should this become one large JOIN model in `models/fdp/`,
     or five smaller MAP models with the joins re-expressed as
     a single mart? Argue for one over the other.
  3. Which fields in the combined output are PII according to
     the source schemas? List them.

Do not write any SQL yet.
```

This is the prompt I run first, every time. Its job is to surface the design choice — JOIN-heavy FDP or MAP-with-mart — before the agent commits a thousand lines of plausible-looking SQL to that choice. Agents are very willing to plough straight into code. They are less willing to argue for an architecture, and you have to make them.

What to verify in the output. Read the three answers as if they were a junior engineer's. The grain answer is usually right; the JOIN-vs-MAP recommendation is usually defensible but not necessarily right; the PII list is the one I check carefully, because agents miss the non-obvious fields. A `marketing_consent_email_hash` is PII even though it has "hash" in the name, because the rest of the row identifies the person. The schema flag will say so. Trust the schema, not the agent's gloss on it.

Prompt 2 — generate the models

Assume the previous prompt settled on five MAP models, with the joins expressed in a single mart view downstream. (This is the choice I would defend nine times in ten for the framework: it keeps the FDP layer simple and pushes the cross-table logic into a place that is cheap to rebuild.)

```
For each of the five ODP tables identified in Prompt 1, generate
a dbt model in `models/fdp/`. Follow the pattern of the existing
MAP model at `models/fdp/portfolio_account_excess.sql`.

Mandatory requirements (failure to meet any of these will fail CI):
  - Materialisation: `incremental` if the source has a reliable
    `_loaded_at` column; `table` otherwise. State which you chose
    and why in a comment at the top of the file.
  - `unique_key`: set to the primary key declared in the
    EntitySchema. Do not invent one.
  - `incremental_strategy='merge'` only if the source supports
```

```
    idempotent re-loads (check by reading the EntitySchema —
    if `idempotent=True`, use merge; otherwise use the default
    append strategy and explain in a comment.
  - Inject `{{ gcp_pipeline_transform.generate_audit_columns() }}`
    in the SELECT.
  - For every column flagged `pii=True` in the schema, emit a
    masked variant using
    `{{ gcp_pipeline_transform.apply_pii_masking(schema=..., field=...) }}`.
  - Reference the staging view, not the raw ODP table.

After generating all five files, run `dbt compile` and report
the output. Do not run `dbt run` yet.
```

What to verify in the output. Four things, in this order. First: does each file open with the comment explaining the materialisation and incremental choice? If not, the agent has skipped a step and will probably have skipped others. Second: is the audit macro present in every file? Agents forget this surprisingly often, and the CI's `audit_columns_present` test will catch it, but you would rather notice now than at 17:50 on a Friday. Third: PII masking — cross-check the masked fields against the schema's `pii=True` flags. Agents sometimes mask a field that is not PII (harmless but ugly) and sometimes miss a field that is (not harmless). Fourth: did `dbt compile` actually pass? If the agent reports "compiled successfully" but you can see in the diff that it referenced a column called `customer_postcode_pc` and you know the schema says `postcode`, the agent has hallucinated and the compile didn't run, or it ran against the wrong target. Don't take the agent's word for it; run `dbt compile` yourself.

Prompt 3 — generate the tests

```
Generate the dbt tests for the five new models in
`models/fdp/schema.yml`. For each model, declare:
  - `not_null` on the unique_key
  - `unique` on the unique_key
  - `not_null` on every column flagged `required=True` in the
    EntitySchema
  - `accepted_values` on every column with `allowed_values`
    declared in the EntitySchema
  - A custom test
    `gcp_pipeline_transform.row_count_matches_source` comparing
    the count in the FDP model to the count in the corresponding
    staging view, allowing the standard 0.1% tolerance for
    in-flight rows.

Also add the standard project-wide tests:
  - `gcp_pipeline_transform.audit_columns_present`
  - `gcp_pipeline_transform.pii_fields_masked`

Run `dbt test --select <new_models>` after generating, and
report the result.
```

What to verify in the output. The reconciliation test is the one that catches the most production-critical bugs, and it is the one agents most often produce slightly wrong — wrong tolerance, wrong direction (FDP can be lower than source under incremental load, but should not be higher), wrong cardinality (compared against the raw ODP rather than the staging view). Read the generated YAML carefully. The `accepted_values` test is one agents tend to over-apply, because the schema sometimes declares a long list of historical values that include retired codes; you may need to trim the allowed list down to the values that are actually current.

Prompt 4 — generate the runbook

```
Generate a runbook at `docs/runbooks/customer_addresses.md`
(and one each for the other four entities) covering:
  - Source system and contact.
  - Expected schedule and SLA in minutes.
  - Expected daily row count range (read from the
    `expected_row_count_range` field in the EntitySchema; if
    not set, state "TODO: confirm with source team").
  - Backfill procedure: which commands to run, in what order,
    and what to check after each.
  - On-call paging conditions: which Alert Manager rule names
    page, and at what severity.
  - Known failure modes and their remediations.

For paging conditions, refer to the alert rules in
`infrastructure/terraform/modules/alerts/rules.tf` — do not
invent rule names.
```

What to verify in the output. This is the prompt where agents are least useful and most confident-sounding. Runbooks have to reflect your actual on-call rota, your team's actual escalation policy, your actual recovery procedures. An agent will produce something that sounds right and is mostly wrong: it will invent reasonable-sounding remediation steps, cite alert rules that do not exist, and confidently state SLAs that nobody agreed to. The right response is to use the agent's output as a structured first draft and rewrite the operational sections by hand. The structure is worth keeping; the content needs a human who has done a 03:00 page.

That is the worked example. Five validated joins in, one FDP layer out, in roughly two hours of human time and forty minutes of agent time. Done badly, the same job is two days of copy-paste, and you would have missed the PII masking on `marketing_consent_email_hash`.

The agent's blind spots, mapped to the framework's guardrails

Working with agents enough, you start to notice the same failures showing up. The pleasant surprise is that for each one, the framework already has the guardrail. Spelling these out explicitly seems to help teams get past the "agents are unreliable,

can we trust them?" debate, because the answer turns out to be "they are unreliable in specific, repeatable ways, and the framework already catches most of them".

Agents hallucinate column names. This is the single most common failure. The model thinks the customer table has a `customer_name` column when in fact the column is `full_name`. The guardrail: the `EntitySchema` import-and-validate step. When the dbt model compiles, the framework's macros pull field names from the schema. A reference to a non-existent column fails `dbt compile` cleanly with a column-resolution error pointing at the line.

Agents forget audit columns. They write `SELECT a, b, c FROM ...` and walk away. The guardrail: the CI test `gcp_pipeline_transform.audit_columns_present` asserts that every FDP row has a non-null `_fdp_run_id`. The test fails; the build fails; the PR cannot merge.

Agents invent dbt config that does not exist. Particularly fun: an agent will sometimes set `materialized='snapshot'` on a model with no `target_schema` block, or set `incremental_strategy='delete+insert'` in a project that has never used that strategy. The guardrail: `dbt compile` rejects unknown configs and incompatible strategies. The discipline is to actually run compile, not to trust the agent that it ran compile.

Agents pick `merge` strategy without thinking about idempotence. They see the word `merge` in your other models and copy it across. If the source isn't idempotent, `merge` produces duplicates with different `_fdp_run_id`s. The guardrail here is partly the framework (the schema's `idempotent` flag is the right place to read from) and partly process: the prompt above explicitly asks the agent to explain why, in a comment. Forcing the agent to articulate the choice catches the copy-paste.

Agents under-think the failure modes. Their runbooks describe the happy path beautifully and the on-call path generically. The guardrail is not the framework, it is you. You rewrite the on-call sections.

Agents over-eagerly add tests. This is the opposite problem: an agent will sometimes add seven redundant tests on the same column because it pattern-matched against an example. The guardrail is review. The PR review is where you trim, not where you check correctness — that is what CI is for.

The general shape: agents fail in ways that look catastrophic in isolation and are routine when caught by an existing test, lint rule, or compile step. The framework's guardrails are the same ones that catch tired-human errors, because tired humans and agents make similar mistakes. They both forget the audit macro.

What to do when the agent's output is wrong

Treat the agent like a junior engineer producing a PR. Specifically:

- **Never copy-paste blindly.** The agent's output goes onto a branch, never straight into `main`. The diff is reviewed. The CI runs.
- **Always run `dbt compile` then `dbt test` before merging.** Not the agent's report of having run them. You run them. If the agent ran them in its sandbox, fine; you run them again locally or in CI.

- **Never let an agent push to `main`.** This sounds obvious. It is not — there is a class of in-house agents that file PRs that auto-merge once CI passes. CI is not a substitute for a human reviewer on infra-changing code. (For the kinds of PRs that change a docstring or fix a typo in a runbook, auto-merge is fine.)
- **Pin the agent's permissions.** If the agent can run shell commands, scope it to a single directory. If it can hit GCP, scope it with a service account that has no production permissions. The most expensive AI incident I have seen in 2026 was a developer's agent running `gcloud dataflow jobs cancel` against the wrong project because it had `--all-projects` configured for convenience.
- **Read the diff.** I cannot stress this enough. The reason `git diff` is in your `pre-commit-verify` skill is that it is the one habit that catches the agent's most expensive mistakes.

Frame all of the above as code review. Nothing about the agent's PR is different from a graduate's PR. The same rigour applies; the same checks apply; the same blast-radius limits apply.

Pattern: the prompt template

The single most useful artefact I give to teams adopting this workflow is a `PROMPTS.md` at the repo root with a reusable template. The template enforces the "give the agent the schema, not just the SQL" discipline by giving every prompt a fixed structure. Here it is.

```
# Prompt template — new entity in `models/fdp/`

## Inputs (fill in for each invocation)

- **Entity name:** `<entity_name>`
- **Source ODP table:** `<odp_table>`
- **Target layer:** `fdp` (this template assumes FDP; copy and
  adapt for marts/analytics)
- **Pattern:** `MAP` | `JOIN`
- **EntitySchema path:** `gcp_pipeline_schemas/<entity>.py`
- **Reference model to follow:** `models/fdp/<existing_model>.sql`
- **Analyst-handoff SQL (if any):** `<path or "none">`

## Required outputs

1. EntitySchema at the path above (if not already present).
2. dbt model at `models/fdp/<entity_name>.sql`.
3. Schema YAML entry in `models/fdp/schema.yml`.
4. Test fixture at `tests/fixtures/<entity_name>.csv`.
5. Runbook stub at `docs/runbooks/<entity_name>.md`.

## Non-negotiables

- Use `{{ gcp_pipeline_transform.generate_audit_columns() }}`.
- Mask every PII field via
  `{{ gcp_pipeline_transform.apply_pii_masking(...) }}`.
- Set `unique_key` to the schema's declared primary key.
- Default to `materialized='incremental'` if the source has a
```

```
  `_loaded_at` column; otherwise `table`. State which and why.
- Default to `incremental_strategy='append'` unless the schema
  has `idempotent=True`, in which case use `merge`.
- Reference the staging view (`stg_<entity_name>`), not the
  raw ODP table.

## Verification (do before reporting back)

- Run `dbt compile`. Paste the output.
- Run `dbt test --select <entity_name>`. Paste the output.
- List, in plain English, every PII field you masked and why.
- List, in plain English, every column you referenced that
  does not appear in the EntitySchema. (Expected answer: none.)

## Out of scope

- Do not modify any existing model.
- Do not modify any orchestration code (`generate_dags.py`,
  `system.yaml`).
- Do not create new macros.
- Do not change PyPI dependencies.
```

That template — pasted at the top of a `PROMPTS.md` in the repo root, with team-specific paths filled in — is the takeaway artefact of this chapter. It does three useful things at once: it forces the prompter to think before they type, it scopes the agent's work narrowly enough that the diff is reviewable, and it standardises the language across the team so that prompts are themselves a reviewable artefact in PR descriptions.

What agents can't do (yet)

Honest list, as of writing:

- **Design the data model.** A new entity's place in the FDP — what grain it sits at, what it joins to, what mart it eventually serves — is a judgement call. Agents will produce a defensible answer; they will not produce the right one without a lot of context.
- **Choose between JOIN and MAP.** Related to the above. Agents will pick whichever pattern appears more often in the codebase they have seen, which is not the same as the one that fits your case. Make the call yourself.
- **Make a cost-vs-speed trade-off.** Whether to run a daily full refresh or a five-minute incremental is a FinOps decision. Agents have no idea what your bill looks like.
- **Write a runbook from scratch.** They will produce a plausible-sounding generic runbook with invented page rules, made-up SLA numbers, and reasonable-but-wrong recovery steps. Use them for structure, not content.
- **Interview a data engineer.** Appendix E is still your job, and I would be alarmed if it stopped being.
- **Argue back.** A good engineer will tell you your design is wrong. An agent will produce whatever you ask for, even when the ask is incoherent. The discipline of arguing back has to come from you, or from a colleague.

I expect at least three of these to age badly within two years. I would happily be wrong about all of them. The book is robust to that: the framework's structure is the part that matters, and the framework's structure is what makes agents useful in the first place.

A note on this chapter aging

If you are reading this in 2028 and laughing at the product names, fair enough. The principles — schema as anchor, narrow scoped prompts, treat the agent's output as a PR, lean on existing guardrails — will outlast the tooling. The bit that will not need replacing is the framework's discipline: an `EntitySchema` is just as useful for grounding an agent in 2028 as in 2026. The agents are tools. The framework is what makes the tools useful.

Key takeaways

- The framework's design — `EntitySchema` as single source of truth, named JOIN/MAP patterns, layered `models/` directories, mandatory audit and PII macros — was built for human auditors and happens to be exactly the shape AI coding agents work best with. That is luck, not foresight, but it is real.
- The single most important prompting discipline is to give the agent the *schema path*, not just the SQL. "Generate a dbt model for `customer_addresses`, reading the schema at `gcp_pipeline_schemas/customer.py`" beats "write a dbt model for customer addresses" by an order of magnitude in reliability.
- Treat every agent output as a junior engineer's PR: review the diff, run `dbt compile` and `dbt test` yourself, never let an agent push to `main`, and scope the agent's GCP permissions tightly. The most expensive AI incidents in 2026 came from agents with overly broad credentials, not from bad prompts.
- Agents reliably fail in specific ways — hallucinated column names, forgotten audit columns, copy-pasted `merge` strategy, plausible-sounding runbooks with made-up alert rules. Every one of these failures is caught by an existing framework guardrail (schema validation, `audit_columns_present` test, `dbt compile`, your own review), provided you actually run the guardrail.
- Drop a `PROMPTS.md` at the repo root with a reusable template. It enforces the schema-first discipline, standardises the team's prompts, and turns prompts themselves into a reviewable artefact in PR descriptions.

Chapter 22 — Getting Started and a Roadmap

Five minutes: install from PyPI

```
pip install gcp-pipeline-framework
python -m gcp_pipeline_framework.reconstruct \
    --dest ~/projects/my-pipeline-project
cd ~/projects/my-pipeline-project
ls
```

You now have docs, infrastructure, scripts, templates, and DAG sources locally. The version installed is the latest from PyPI; pin with `pip install gcp-pipeline-framework==1.0.29` for reproducibility.

An hour: deploy the generic system to a sandbox project

```
gcloud config set project my-sandbox-project
gcloud auth login

./scripts/gcp/01_enable_services.sh
./scripts/gcp/02_create_state_bucket.sh
./scripts/gcp/03_create_infrastructure.sh all

gh secret set GCP_PROJECT_ID --body 'my-sandbox-project'
./scripts/gcp/setup_github_actions.sh

git add . && git commit -m "Initial deploy" && git push origin main

gh run list --workflow=deploy-generic.yml --limit 3
./scripts/gcp/06_test_pipeline.sh generic
```

That is a full deploy of the three-unit `generic` system without Composer. Cost: low double-digit dollars per month.

A day: add your own entity

1. Define an `EntitySchema` for the entity in your codebase.

2. Add four lines to `system.yaml`.
3. Add an FDP model in `bigquery-to-mapped-product/models/fdp/`.
4. Push to a branch; the deploy-generic workflow tests and deploys only what changed.
5. Drop a sample file into the landing bucket; watch the audit trail.

The framework's design means the new entity needs almost no orchestration code. The factory generates the DAG; the schema drives the validator; the macros inject the audit columns.

A roadmap

The framework is at version 1.0.29 — solidly past the cliff of "first useful release" but not yet at "second-system stability". A reasonable next year of work, in priority order:

1. **Automated e2e in CI.** The single highest-leverage change.
2. **A working streaming reference deployment.** Postgres CDC, end-to-end.
3. **A small admin UI.** Quarantine review at minimum.
4. **Cost-budget alerts.** Threshold-based, per-entity.
5. **Canary deploys.** A few hours on a few files before full rollout.
6. **Property-based parser tests.** Hypothesis on HDR/TRL.
7. **A documented deployment-profile abstraction.** Named worker profiles.
8. **An operational handbook.** "If you see error code X, do Y." Distinct from architectural docs.

Closing thoughts

The boring truth about data pipelines is that the hard problem is not "how do I move bytes from A to B". The hard problem is "how do I make the movement of bytes legible, auditable, recoverable, and cheap". `gcp-pipeline-framework` is a serious attempt at the boring truth. It will not be the last attempt; better ones will come, and I would be delighted to read them. But it is, in 2026, a credible answer to the question "how do I run a mainframe-to-BigQuery pipeline I can stake my career on".

If you build something that improves on it, I would love to hear about it. The best version of this book is the one that someone else writes about a framework that makes mine look antique. Until then: thank you for reading, and good luck with the on-call rota.

— *Joseph Aruja*

Key takeaways

- Getting started takes five minutes (`pip install gcp-pipeline-framework` plus `reconstruct.py`), an hour to deploy the generic system to a sandbox, and a day to add your own entity. The on-ramp is gentle by design.
- The roadmap's highest-leverage item is automated e2e in CI — everything else is easier once you know the full round-trip works before every release.

- Version 1.0.29 is past ”first useful release” but not yet at ”second-system stability”. Adopt it knowing the rough edges are known and documented, not hidden.
- The boring truth of data engineering is not moving bytes from A to B — it is making that movement legible, auditable, recoverable, and cheap to operate for years. That is what this framework is a serious attempt at.

Chapter 23 — Spring for Data Pipelines: The Multi-Cloud Roadmap

Why I am renaming a perfectly working framework

The framework has a name now: **Culvert**. A culvert is the engineered pipe that carries water from one side of a road to the other — controlled flow through a designed channel, holding back what should be held back, releasing what should be released, at a known rate. The metaphor is exact. A data pipeline is a culvert: an engineered channel carrying records from a source to a destination, with controlled gates — governance, masking, quality checks — along the way. The name is short, distinctive, and unclaimed on PyPI, which are the three properties that matter for an open-source framework name. The book you are reading was written about the *reference implementation* — the GCP-specific deployment at `github.com/enrichmeai/gcp-pipeline-reference` that the rest of this book documents. **Culvert is the framework that reference grew into.** It lives at `github.com/enrichmeai/culvert`. The remainder of this chapter is about Culvert: what it is today (GCP only), what shape it commits to (cloud-neutral contracts, swappable cloud modules), and how the migration from `gcp-pipeline-*` to `culvert-*` lands across the next six months of work.

The packages on PyPI are called `gcp-pipeline-core`, `gcp-pipeline-beam`, `gcp-pipeline-orchestration`, `gcp-pipeline-transform`, and `gcp-pipeline-tester`. They will, in the next major release, be called `culvert-core`, `culvert-gcp-dataflow`, `culvert-gcp-composer`, `culvert-gcp-dbt`, and `culvert-test`. A handful of new modules — `culvert-quality`, `culvert-governance`, `culvert-observability`, `culvert-finops`, `culvert-orchestration-core` — will appear alongside them. The reserved slots `culvert-aws-*` and `culvert-azure-*` will sit empty in the layout, unbuilt, on purpose.

I want to explain why, because at first glance this looks like exactly the kind of cosmetic renaming that gets engineers fired in their second year. The first time someone proposed I rename my framework I rolled my eyes hard enough to dislodge a contact lens. I had a working framework. The packages were on PyPI. Customers were running pipelines against it in production. What sort of muppet rewrites the package names of a system that already works?

The honest answer is that I sat down a few months ago and did a proper audit of the codebase against the question *how much of this is actually GCP code?* I expected the answer to be "almost all of it" — the framework lives in a directory called `gcp-`

`pipeline-libraries`, every module starts with `gcp_pipeline_`, every docstring opens with "GCP Pipeline Framework — …". When I went through the files one at a time and counted imports, the answer was about fifty-five percent. The other forty-five-or-so percent was already cloud-neutral, in fact if not in name. The data-quality dimensions, the error taxonomy, the audit records, the lineage tracker, the structured logger, the run-ID generator, the alert manager, the OTEL bridge, the schema dataclasses, the HDR/TRL parser, the validators — none of these imported `google.cloud` anything. They lived in a directory called `gcp_pipeline_core` and they were not GCP code. They were generic Python with a GCP prefix glued to the front.

That is in-name-only coupling, and once you notice it you cannot un-notice it. The rename is the smallest move that lets the framework grow into the shape it already half-is. It is not a rewrite. It is closer to admitting what already exists.

The rest of this chapter is about what that admission lets you do.

The Spring precedent, told properly

The cultural reference point I keep returning to — and I named it in the Preface, so this is not a surprise twist — is Spring Framework. The relevant fact about Spring is *not* that it ended up multi-database, multi-runtime, and multi-cloud. The relevant fact is the order in which it got there.

`spring-data-jpa` shipped first. It was the only persistence module for years. It targeted relational databases through a perfectly good Java standard, and people built real applications against it without ever having to wonder whether Spring would one day support a document store. When `spring-data-mongodb` finally appeared, it was written largely by the MongoDB team itself, and the JPA users did not have to learn a single new concept; the contracts that `spring-data` had defined — repositories, queries, conversion services — were honest enough that MongoDB could plug into them without contorting itself or contorting the JPA users' code.

What makes that story work is that `spring-core` was never contaminated with relational assumptions. It hosted *any* persistence model, even when only one existed in the wild. The team made the abstractions cloud-neutral — to drag the metaphor forward by twenty years — and then shipped the implementation they actually needed. The other implementations either followed from a community that wanted them, or were built deliberately and much later. They were not promised; they were enabled.

That is the move here. I make the abstractions cloud-neutral. I ship the GCP implementation, which is the only one I have any business shipping because it is the only cloud I have run this framework against. And I wait. If someone three years from now wants `culvert-aws-redshift` enough to write it, the contracts make that a 2-4 week build per service rather than a rewrite. If nobody ever writes it, the framework is no worse for the rename — the GCP code is the same code, with honest names.

What was already cloud-neutral

The audit's most useful finding was the inventory of files that contain zero references to `google`, `bigquery`, `gcs`, `dataflow`, or `pubsub`. The list is longer than I expected:

The entire `data_quality/` subpackage — `checker.py`, `dimensions.py`, `scoring.py`, `anomaly.py`, `reporting.py`. These operate on `List[Dict[str, Any]]` and have always done so. The error taxonomy in `error_handling/errors.py`, `handler.py`, `context.py`, `models.py` — every exception class, the classifier, the retry policy, the in-memory storage. The audit primitives in `records.py`, `trail.py`, `lineage.py` — pure dataclasses and dict assembly. The whole of `data_deletion/` except for one GCS subclass — the malformation detector, the quarantine manager, the deletion workflow, the recovery bookkeeping. The HDR/TRL parser, modulo one `gs://`-sniffing block I will come back to. The monitoring primitives: `MetricsCollector`, `HealthChecker`, `AlertManager`, the OTEL tracing and context modules. The structured logger. The run-ID generator. The schema dataclasses. The finops cost-metrics model and the label dataclass.

None of this code was wrong. The naming was wrong. These modules have been generic the whole time; calling them GCP because they live under a `gcp_pipeline_*` namespace is a category error I happened to commit when I first laid the repo out, and the audit caught me at it.

What the rename does for this code is honest labelling. `culvert-core` is allowed to import `typing_extensions`, `pydantic`, and `opentelemetry-api`. It is not allowed to import `google.cloud.anything`. There is a single-line CI check — `grep -r "google\.cloud" src/` returning non-zero fails the build — that enforces that boundary cheaply. It is the smallest possible mechanism, and it is the same trick Spring projects have used for two decades to keep their kernels honest.

What is genuinely GCP

The other half of the audit is the inventory of code that is GCP-coupled by design. I want to name these by name because I am not going to pretend they are not what they are.

The BigQuery client wrapper in `clients/bigquery_client.py`. The GCS client wrapper. The Pub/Sub client wrapper. The BigQuery-backed job-control repository — every method on `JobControlRepository` builds parameterised BigQuery SQL and ships it through `bigquery.Client.query()`; the class has 49 BigQuery references across 511 lines and that is correct, because it is doing BigQuery work. The cost tracker in `finops/tracker.py` with its `BQ_COST_PER_TIB = 6.25` constant. The Pub/Sub audit publisher. The Cloud Monitoring alert backend. The GCS error storage. The GCS recovery manager. The GCS file lifecycle and archiver. The Dataplex governance hooks. The entire Beam-on-Dataflow execution package. The entire Composer DAG factory built around `airflow.providers.google.cloud.*`. The dbt macros that emit BigQuery SQL.

All of this stays GCP. All of it gets named honestly: `culvert-gcp-bigquery`, `culvert-gcp-dataflow`, `culvert-gcp-composer`, `culvert-gcp-gcs`, `culvert-gcp-pubsub`, `culvert-gcp-dataplex`, `culvert-gcp-observability`, `culvert-gcp-secrets`, `culvert-gcp-dbt`. These are first-class modules in the framework, not afterthoughts. They are what makes the framework useful to a customer running on GCP today. They get to embrace BigQuery's clustering and partitioning and slot-aware cost predicates without apology, because they are BigQuery modules.

The point is that they are no longer the *whole* framework. They are the GCP family within it.

Why "culvert-" and not "pipeline-"

The naming convention deserves a sentence because I wrestled with it and would rather you not have to.

I started with `pipeline-core`. It is shorter. It is cleaner. It is also wrong, because `pipeline` is one of the most overloaded words in software — CI/CD pipelines, ML pipelines, rendering pipelines, build pipelines, Jenkins pipelines. A package called `pipeline-core` on PyPI would sit next to seven other things claiming the same name, and the cognitive overhead of distinguishing them would never go away.

The `data-` prefix is doing real work. It marks the domain. Compare to how Spring distinguishes `spring-batch` (not `spring-jobs`), `spring-data` (not `spring-persistence`), `spring-cloud` (not `spring-distributed`). The pattern is *spring, then the domain word, then the specifics*. `culvert-core` follows the same shape: framework family, domain, role. The cloud modules become `culvert-gcp-bigquery` — family, cloud, service. When the reserved AWS slot eventually opens, it becomes `culvert-aws-redshift`, and a reader who has never seen the framework can guess what each word does.

This is convention as documentation, and it costs me nothing to get right at rename time. It would cost me a great deal to fix later.

The contracts, briefly

The framework's relationship with the cloud lives in roughly a dozen Python protocols. They are small. They are not the interesting part of the codebase. They are the part that lets every other part be honest about whether it is cloud-specific or not.

The surface looks like this, and I am writing it from memory because if I cannot remember it the surface is too big:

`Source[T]` and `Sink[T]` and `Transform[T, U]` are the foundational primitives — anything that yields, consumes, or maps records. `Pipeline` is a composition of `PipelineStage` objects. `RuntimeContext` is the framework's dependency-injection container: it carries the run ID, the environment, the config, the secrets provider, the observability hook, the lineage emitter, the finops sink, the governance engine, and the lookup table for protocol implementations. Every `Source`, `Sink`, or `Transform` method takes a `RuntimeContext` as its second argument. Then the protocol seams to the cloud: `JobControlRepository` (the pipeline-job ledger), `BlobStore` (object storage), `Warehouse` (tabular query and load), `AuditEventPublisher` (audit emission), `GovernancePolicy` (masking, classification, retention lookup), `LineageEmitter` (OpenLineage-shaped events), `ObservabilityHook` (the metrics-logs-traces seam), `FinOpsSink` (cost-record persistence), `SecretProvider` (secret lookup).

That is the whole surface. Nothing else in the framework talks to a cloud SDK directly. If you find yourself reaching for `from google.cloud import bigquery` in a file that

does not live under `culvert-gcp-bigquery`, you have made a mistake and the CI grep check will tell you so.

A worked example, because the abstract version is hard to argue with and hard to evaluate. Here is the `Warehouse` protocol stripped to its essentials:

```
class Warehouse(Protocol):
    def query(self, sql: str, params=None) -> Iterator[Mapping[str, Any]]: ...
    def execute(self, sql: str, params=None) -> None: ...
    def load_from_uri(self, uri: str, target_table: str,
                      schema: EntitySchema) -> int: ...
    def merge(self, source_table: str, target_table: str,
              keys: list[str]) -> int: ...
    def table_exists(self, fqtn: str) -> bool: ...
```

The GCP implementation lives in `culvert-gcp-bigquery/warehouse.py`:

```
class BigQueryWarehouse:
    def __init__(self, client: bigquery.Client):
        self._client = client

    def query(self, sql, params=None):
        job = self._client.query(sql, job_config=_params_to_config(params))
        for row in job.result():
            yield dict(row)

    def load_from_uri(self, uri, target_table, schema):
        cfg = bigquery.LoadJobConfig(schema=_to_bq_schema(schema),
                                     source_format="CSV")
        return self._client.load_table_from_uri(uri, target_table,
                                                job_config=cfg).result().output_rows
    # ...
```

A hypothetical AWS implementation, which I am not going to write but which I want the reader to see is *possible* without changing anything in core, would live in `culvert-aws-redshift/warehouse.py` and look something like:

```
class RedshiftWarehouse:
    def __init__(self, conn: redshift_connector.Connection):
        self._conn = conn

    def query(self, sql, params=None):
        cur = self._conn.cursor()
        cur.execute(sql, params or {})
        cols = [d[0] for d in cur.description]
        for row in cur:
            yield dict(zip(cols, row))

    def load_from_uri(self, uri, target_table, schema):
        copy_sql = f"COPY {target_table} FROM '{uri}' IAM_ROLE ... FORMAT CSV"
        cur = self._conn.cursor()
        cur.execute(copy_sql)
        return cur.rowcount
    # ...
```

These are different. The first one talks to BigQuery; the second one talks to Redshift. The protocol does not care. The pipeline code that calls `context.get(Warehouse).load_from_uri(...)` does not care. The decision about which `Warehouse` is in the runtime context is made once, at bootstrap, by whichever `culvert-*` cloud package the user has installed. That is the entire trick. Spring did this in 2003. We are not inventing anything.

The decorator surface

The user-facing API is the second piece, and it is the bit that earns the "Spring for data pipelines" framing. Spring's `@Component`, `@Service`, `@Repository`, `@Configuration`, `@Autowired` annotations are not magic — they are the framework's permission to introspect, wire, and instrument the code. The Python equivalents are decorators. The framework's are `@pipeline`, `@source`, `@transform`, `@sink`, `@governed`, `@masked`, and `@quality_check`.

A typical FDP pipeline written against the new surface looks like this:

```
from data_pipeline_core import (
    pipeline, source, transform, sink,
    governed, masked, quality_check, RuntimeContext,
)
from data_pipeline_core.schema import EntitySchema

CUSTOMER = EntitySchema.from_yaml("schemas/customer.yaml")

@pipeline(
    name="customer_fdp",
    schedule="0 2 * * *",
    retry={"attempts": 3, "backoff_seconds": 60},
    finops_tags={"cost_center": "retail", "owner": "data-platform"},
)
class CustomerFDP:

    @source(uri="gs://incoming/customer/*.dat",
            schema=CUSTOMER, format="fixed-width-hdr-trl")
    def read_files(self, context: RuntimeContext): ...

    @transform(input="read_files")
    @quality_check(dimension="completeness", min_score=0.99)
    @governed(retention_days=2555, classification="PII")
    def landing(self, records, context):
        for r in records:
            yield {**r, "ingested_at": context.now()}

    @transform(input="landing")
    @masked(fields=["ssn", "dob"], policy="last4")
    def mask_pii(self, records, context):
        return records

    @sink(target="bigquery://retail.curated.customer", schema=CUSTOMER,
```

```
            write_disposition="WRITE_APPEND")
    def write_curated(self, records, context): ...
```

What each decorator wires up, in plain English. `@pipeline` registers the class with the runtime's component registry, assigns a `run_id` per invocation, attaches the schedule and retry config to the cloud-neutral orchestration model (which `culvert-gcp-composer` later compiles into an Airflow DAG), threads the finops tags through every downstream cost record, and wraps the whole pipeline in an OTEL span. `@source` declares an input — the URI is opaque to the framework; a `BlobStore` from auto-config parses it. `@transform` declares a stage with an explicit input edge. `@quality_check` invokes `culvert-quality` against the named dimension; a failed check routes through the error classifier, which knows (because validation does not retry) not to bother retrying. `@governed` looks up the field's policies via the `GovernancePolicy` protocol — Dataplex when you have it, a static YAML when you do not. `@masked` applies the configured masking strategy before the records leave the stage. `@sink` declares the output; the URI scheme tells the framework whether it is talking to a `Warehouse` or a `BlobStore`.

That same code runs on GCP today. It would run on AWS the day someone shipped `culvert-aws-redshift` and `culvert-aws-s3` — the *only* thing that would change is the URI scheme in the `@source` and `@sink` decorators and the cloud packages in `pyproject.toml`. The pipeline class body, the decorators, the schema, the quality checks, the governance rules, the finops tags — all unchanged.

I have written enough frameworks to know that I am promising something here that frameworks routinely fail to deliver. I will not know whether the abstraction holds until someone other than me builds the AWS adapter. That is the honest answer. The decorators are designed so that if the abstraction is wrong, the AWS adapter will tell us, and we will fix the protocols rather than papering over them.

What you can actually do today

I want to be precise about this, because the gap between "the framework has multi-cloud contracts" and "the framework runs on multi-cloud" is the gap a lot of vendors smudge over and a lot of customers get burned by.

Today, on GCP, the framework runs production pipelines and has done for a year. After the rename ships, today on GCP, the framework will run production pipelines and have done for a year, and your `from gcp_pipeline_core...` imports will keep working because the old packages become deprecation shims that re-export the new ones with a `DeprecationWarning`. You will see warnings in your test logs telling you to update imports. You will update them at your own pace over the next release cycle. Your DAGs will not change. Your Terraform will not change. Your cost will not change.

Today, on AWS, the framework does not run. There is no `culvert-aws-*` package. There is no Redshift `Warehouse` implementation, no S3 `BlobStore`, no DynamoDB job-control repository, no MWAA orchestrator. Calling the rename "multi-cloud" without those modules would be misleading, and the README and this chapter say so plainly. What the rename buys an AWS-curious reader is the *shape*: the contracts exist, the

layout reserves the package names, and the contract-test harness (more on that in the next section) will validate any adapter that someone eventually writes against the same behavioural specifications the GCP adapters pass today.

That is the honest delta. If you are running GCP pipelines, the rename is a metadata change. If you are running AWS pipelines and you wanted the framework, you cannot have it yet — but you can see how you would build it.

A year-ahead direction, told as a sequence

The migration breaks into six stages, and I want to walk through them as a story rather than a checklist because the order matters and the dependencies matter.

Stage 0 is preparation. I extract the Python `Protocol` definitions for `Source`, `Sink`, `Transform`, `Pipeline`, `RuntimeContext`, `JobControlRepository`, `BlobStore`, `Warehouse`, `AuditEventPublisher`, `GovernancePolicy`, `LineageEmitter`, `ObservabilityHook`, `FinOpsSink`, and `SecretProvider` into a fresh `culvert-core` distribution that sits *next to* the existing `gcp_pipeline_core` without replacing it. The existing classes are not yet refactored to implement the protocols; the protocols *describe* the existing classes' public methods, which is why this step is days rather than weeks. Nobody downstream is touched. The deliverable is an internal-PyPI release of `culvert-core==0.1.0` and a design review of the protocol shapes against the existing classes.

Stage 1 is the rename itself. Every `gcp-pipeline-*` distribution gets a sibling `culvert-*` that contains the real code. The old distributions become deprecation shims — `from gcp_pipeline_core.audit import AuditTrail` resolves to `data_pipeline_core.audit.trail.AuditTrail` and emits a `DeprecationWarning` with a one-line migration hint. CI pipelines and deployment workflows keep running because they reference Python package names through `requirements.txt`-style files and the shims keep the old names resolvable. The audit counted about 150 cross-package imports across libraries, another 80 in the reference deployments, and 30 in the test suites; almost all of these are mechanical with `sed`. Effort: roughly a week.

Stage 2 is the split. The GCP-coupled files identified in the audit move out of `culvert-core` into `culvert-gcp-*` modules. `BasePipeline` no longer imports `BigQueryClient` directly; it accepts a `Warehouse` typed dependency. `dag_factory` accepts an `Orchestrator` typed dependency. The shim re-exports stay in place. This is real refactoring, not search-and-replace, because the `dag_factory` is a load-bearing piece and the BigQuery-backed `JobControlRepository` is referenced in a dozen places. Effort: two to three weeks. Risk: medium. The contract-test harness from stage 4 is the safety net that makes this defensible.

Stage 3 is the new user-facing surface. The auto-configuration registry, the decorators, the bootstrap routine, the first starter scaffold (`culvert-starter-gcp-fdp`). This is the work that turns the framework from "library you import" into "framework you write against". It is additive — nothing breaks — but it is the bulk of the design work, because the decorators need to work cleanly with both Dataflow and Composer execution and need to thread the runtime context correctly through both. Effort: three to four weeks. The deliverable is a working starter that a user can `copier copy` and deploy to GCP in an afternoon.

Stage 4 is contract testing. The `culvert-test/contracts/` subpackage gets a test suite per protocol. Each test exercises the documented behaviour of the protocol without referencing any concrete implementation — "a `JobControlRepository.create_job` followed by `get_job` returns the same job", "a `BlobStore.put` followed by `get` returns the same bytes", "a `Warehouse.load_from_uri` of a known CSV writes the expected row count". The GCP adapters run those tests and pass. This is what makes the abstraction *real*: it forces the protocol authors to spell out the contract in executable form, and it catches abstraction leaks before any non-GCP implementation ever tries to comply. Effort: one to two weeks.

Stage 5, if it happens, is the first non-GCP module. I would recommend `culvert-aws-s3`, because S3 is the simplest of the AWS services to wrap and `BlobStore` is the simplest of the contracts. If it takes a week, the design works. If it takes a month, the design needs another iteration. We are not committing to this. We are reserving the option, and the layout makes the option cheap to exercise — which is a very different thing from promising it.

Across the whole sequence the runtime behaviour does not change. The framework's user-facing API gains the decorators in stage 3 and gains nothing else. Existing tests continue to pass against existing code. Deployments continue to work because the shim distributions keep the old names resolvable for at least one major version.

What I am explicitly not doing

I want to enumerate the non-goals because the easiest mistake I could make here is the rhetorical drift from "the contracts allow multi-cloud" to "we are a multi-cloud framework". I have watched other vendors make exactly that drift and I have watched their customers pay for it.

I am not shipping AWS or Azure modules. The slots in the layout are reserved names; they are not commitments. If your procurement team is asking whether you can run this framework on AWS in 2026, the answer is no.

I am not creating a lowest-common-denominator abstraction. The `Warehouse` protocol covers the operations every serious warehouse supports and stops there. BigQuery's clustering, partitioning, materialised views, BI Engine, and slot-aware cost predicates are not in the protocol — they live in a `BigQueryExtensions` interface in `culvert-gcp-bigquery` that users call directly when they need BigQuery-specific behaviour. I will not water down BigQuery to fit Redshift. If you want BigQuery semantics, you call BigQuery extensions explicitly.

I am not pretending the framework runs on three clouds when it runs on one. The README and the marketing copy will say "GCP is the supported cloud; the framework is structured so that other adapters are possible but not promised". That is a defensible sentence. "We support AWS" with no shipped AWS module is not.

I am not making `culvert-core` depend on any Google Cloud client library. The CI grep check enforces this on every commit. It is the cheapest possible mechanism, and it has caught analogous mistakes in Spring projects for two decades.

I am not rewriting the GCP code that works. The migration is a rename, a split, and a contract extraction. The BigQuery SQL in `JobControlRepository` does not change. The Beam pipelines do not change. The Composer DAG factory does not change. The audit's verdict — that the GCP-specific implementations are coherent and well-named — is the load-bearing precondition for this being a week-scale effort rather than a quarter-scale rewrite. I am not going to invalidate that precondition by tinkering with working code on the way through.

An invitation

This framework gets to be "Spring for data pipelines" in the full sense — not the metaphor, the *thing* — only if someone other than me writes `culvert-aws-redshift` or `culvert-azure-synapse`. I cannot do it. I do not run AWS pipelines in anger. I have never had a production Azure data workload. Anything I wrote against those clouds would be a tourist's adapter, validated against nothing more serious than the AWS free tier, and it would let the abstractions drift in directions that fit nobody's real workload.

What I can do is make the contracts honest, ship the GCP implementations that pass the contract tests, document the layout so the AWS slot has a clear shape, and write this chapter. The rest is up to whoever, three years from now, reads the audit and the redesign documents in `docs/framework-evolution/` and decides the gap is worth closing for their own organisation.

If you are reading this and you run AWS data pipelines and you wish there were a framework with the audit-trail story, the cost-tracking story, the contract-testing story, the governance story, and the observability story that this one has on GCP — the shape is here. The contracts are designed for you. The `culvert-aws-*` slot in the layout is yours. Write the S3 `BlobStore` first because it is the cheapest to validate; ship it under the framework's naming convention; run the contract tests in `culvert-test/contracts/test_blob_store_contract.py`; open a pull request. I will review it carefully, the existing GCP users will not notice anything has changed in their code, and the framework will quietly take a real step toward what it has been pretending to be all along.

That is the Spring move. Small core. Honest abstractions. One reference implementation. Wait for the community to build the rest. It worked in 2003. It can work in 2026.

The rename is the first step. The rest is patient construction, and — I have to keep reminding myself of this — patience is not a deliverable. It is a posture.

Where Culvert lives

The project lives at `github.com/enrichmeai/culvert`. The cloud-neutral core ships as `culvert-core` on PyPI; the GCP modules ship as `culvert-gcp-bigquery`, `culvert-gcp-dataflow`, `culvert-gcp-composer`, `culvert-gcp-dataplex`, `culvert-gcp-gcs`, and `culvert-gcp-dbt`. If you are reading this and you run AWS or Azure pipelines, the empty `culvert-aws-*` and `culvert-azure-*` slots in the layout are an open invitation

— the contracts are already designed for you; the only thing missing is the adapter, and the framework's naming convention has reserved the shelf space.

Key takeaways

- The rename from `gcp-pipeline-*` to `culvert-*` is admission, not invention. An audit found roughly 45-55% of the codebase was already cloud-neutral in fact; the rename surfaces what was already true and isolates the genuinely GCP-coupled bits into honestly named modules.
- Spring did not ship multi-database in version one. It shipped `spring-data-jpa` first, kept `spring-core` honest about persistence, and let the MongoDB module follow years later. The roadmap here mirrors that: cloud-neutral contracts now, GCP reference implementation now, AWS and Azure slots reserved but unbuilt.
- The contract surface is roughly twelve protocols (`Source`, `Sink`, `Transform`, `Pipeline`, `RuntimeContext`, `JobControlRepository`, `BlobStore`, `Warehouse`, `AuditEventPublisher`, `GovernancePolicy`, `LineageEmitter`, `ObservabilityHook`, `FinOpsSink`, `SecretProvider`). Anything that needs to be cloud-specific touches one of them; anything that does not, does not.
- Today's GCP users see a deprecation shim that keeps `gcp-pipeline-*` imports working for one release cycle. Their DAGs, Terraform, and cost profile do not change. The new decorator surface (`@pipeline`, `@source`, `@transform`, `@sink`, `@governed`, `@masked`, `@quality_check`) is additive.
- The non-goals are as load-bearing as the goals. No AWS or Azure modules from the original team. No lowest-common-denominator warehouse abstraction. No marketing claim of multi-cloud support before the modules exist. A CI grep check fails the build if a `google.cloud` import leaks into core.
- The framework becomes "Spring for data pipelines" in earnest only when someone else writes the next adapter. The contracts make that possible; they do not make it inevitable. If you run AWS data pipelines and you want the audit, finops, contract-testing, and governance story this framework has on GCP, the AWS slot in the layout is yours to fill.

Appendix A — Library Reference

This appendix is a compact reference to the public surface of each framework library. It is intentionally telegraphic; consult the library README for the full API.

A.1 `gcp-pipeline-core`

```
gcp_pipeline_core/
├── audit/
│   ├── AuditTrail            # buffered audit-event writer
│   ├── ReconciliationEngine  # joins envelope, valid, invalid, BQ counts
│   └── DuplicateDetector     # hash-based duplicate row detection
├── monitoring/
│   ├── MetricsCollector      # counters/gauges/histograms/timers
│   ├── MigrationMetrics      # opinionated metric set for migrations
│   ├── ObservabilityManager  # composition root
│   ├── HealthChecker         # error rate, queue depth, memory, latency
│   ├── AlertManager          # multi-backend alert dispatch
│   └── OTELConfig           # OpenTelemetry setup with graceful degradation
├── finops/
│   ├── BigQueryCostTracker
│   ├── CloudStorageCostTracker
│   └── PubSubCostTracker
├── error_handling/
│   ├── ErrorHandler
│   ├── ErrorClassifier       # validation / integration / resource
│   └── RetryPolicy           # exponential backoff with jitter
├── job_control/
│   ├── JobControlRepository  # state for pipeline_runs
│   └── PipelineJob           # pydantic schema
├── clients/
│   ├── GCSClient
│   ├── BigQueryClient
│   └── PubSubClient
├── data_quality/
│   ├── DataQualityChecker
│   ├── ScoreCalculator       # A-F grade rollup
│   └── AnomalyDetector       # rolling baseline, 3-sigma alert
```

```
├── data_deletion/
│   ├── SafeDataDeletion      # REVIEW → HOLD → DELETE → ARCHIVE
│   └── QuarantineManager
├── utilities/
│   ├── configure_structured_logging
│   └── generate_run_id
└── schema.py
    ├── EntitySchema
    └── SchemaField
```

A.2 `gcp-pipeline-beam`

```
gcp_pipeline_beam/
├── pipelines/
│   ├── base/
│   │   ├── BasePipeline      # abstract pipeline with audit/FinOps wired
│   │   └── PipelineConfig
│   └── beam/
│       ├── BeamPipelineBuilder
│       └── transforms/
│           ├── RobustCsvParseDoFn
│           └── SchemaValidateRecordDoFn
├── file_management/
│   ├── HDRTRLParser
│   ├── SplitFileHandler
│   └── FileArchiver
└── validators/
    ├── SchemaValidator
    ├── SSNValidator
    ├── DateValidator
    ├── NumericValidator
    ├── RegexValidator
    └── RequiredValidator
```

A.3 `gcp-pipeline-orchestration`

```
gcp_pipeline_orchestration/
├── sensors/
│   └── BasePubSubPullSensor      # idempotent Pub/Sub pull sensor
├── operators/
│   ├── DataflowFlexTemplateOperator
│   └── DbtRunOperator
├── factories/
│   └── DagFactory                # config-driven DAG generation
├── dependency.py
│   └── EntityDependencyChecker   # JOIN preconditions
├── callbacks/
│   ├── ErrorCallbacks
```

```
│   └── AlertCallbacks
└── routing/
    └── DeadletterRouter
```

A.4 `gcp-pipeline-transform`

```
gcp_pipeline_transform/
├── macros/
│   ├── generate_audit_columns.sql
│   ├── apply_pii_masking.sql
│   ├── data_quality_check.sql
│   └── generate_schema_name.sql
└── helpers/
    └── schema_loader.py          # reads EntitySchema for masking lookup
```

A.5 `gcp-pipeline-tester`

```
gcp_pipeline_tester/
├── PipelineTestCase              # base test class
├── fakes/
│   ├── FakeBigQueryClient
│   ├── FakeGCSClient
│   └── FakePubSubClient
├── factories/
│   └── RowFactory                # schema-driven row generation
├── snapshots/
│   └── PCollectionSnapshot
└── time/
    └── TimeTraveller             # freezegun wrapper
```

A.6 `gcp-pipeline-framework`

The umbrella package. No public API of its own beyond `gcp_pipeline_framework.reconstruct`. Installs all five sibling packages plus the reference deployments as embedded assets.

Appendix B — Directory Map of the Reference Repository

```
gcp-pipeline-reference/
├── README.md
├── VERSION
├── pyproject.toml
├── reconstruct.py
├── qodana.yaml
│
├── gcp-pipeline-libraries/
│   ├── gcp-pipeline-core/
│   ├── gcp-pipeline-beam/
│   ├── gcp-pipeline-orchestration/
│   ├── gcp-pipeline-transform/
│   ├── gcp-pipeline-tester/
│   └── gcp-pipeline-framework/        # umbrella
│
├── deployments/
│   ├── original-data-to-bigqueryload/      # ingestion (Beam)
│   ├── bigquery-to-mapped-product/         # FDP transform (dbt)
│   ├── data-pipeline-orchestrator/         # 5 DAGs (Airflow)
│   ├── fdp-to-consumable-product/          # CDP transform (dbt)
│   ├── mainframe-segment-transform/        # FDP → mainframe (Beam)
│   ├── spanner-to-bigquery-load/           # federated (dbt)
│   ├── postgres-cdc-streaming/             # streaming reference (Beam)
│   └── fdp-trigger/                        # downstream notification
│
├── infrastructure/
│   └── terraform/
│       ├── main.tf
│       ├── security.tf
│       ├── dataflow.tf
│       └── systems/
│           └── generic/
│               ├── ingestion/
│               ├── transformation/
```

```
│               └── orchestration/
│
├── .github/workflows/
│   ├── test.yml
│   ├── publish-libraries.yml
│   ├── publish-deployments.yml
│   ├── deploy-generic.yml
│   ├── deploy-orchestration.yml
│   ├── qodana_code_quality.yml
│   ├── ci-automation.yml
│   └── release.yml
│
├── scripts/
│   ├── gcp/
│   │   ├── 01_enable_services.sh
│   │   ├── 02_create_state_bucket.sh
│   │   ├── 03_create_infrastructure.sh
│   │   ├── 05_verify_setup.sh
│   │   ├── 06_test_pipeline.sh
│   │   ├── 07_cleanup.sh
│   │   ├── e2e_pipeline_test.sh
│   │   └── setup_github_actions.sh
│   ├── automation/
│   │   ├── deploy_and_test_e2e.sh
│   │   └── run_all_checks.sh
│   ├── run_all_library_tests.sh
│   ├── publish_libraries.sh
│   └── scaffold_orchestration.sh
│
├── docs/                                   # 35+ guides
├── templates/                              # DAG, dbt, dockerfile templates
└── test_data/                              # CSV fixtures with HDR/TRL
```

Appendix C — A Cost Model for the Reference Implementation

A small, honest model for what a deployed `generic` system costs in a moderate-volume prod environment (4 entities, daily extracts, ~5 million rows per entity). All numbers are approximate, on-demand priced, in US dollars per month.

Component	Notes	Cost
Cloud Storage (landing, archive, error)	200 GB Standard, 500 GB Coldline	$15
Pub/Sub	~10k messages/day	$1
Dataflow (ingestion)	4 entities × daily × 30 min × n1-standard-2	$90
BigQuery storage (ODP+FDP+marts)	~1 TB Active + 1 TB Long-term	$25
BigQuery compute (queries)	~5 TB/month scanned	$25
Cloud Composer 2 (optional)	smallest config, 24/7	$300
Cloud Logging	structured logs, default retention	$20
Cloud Monitoring	custom metrics	$10
Total without Composer		**~$185**
Total with Composer		**~$485**

The Composer line dominates. This is why the framework's default does not provision it. For a team that needs orchestration but cannot justify $300/month, the recommended substitute is Cloud Functions for the trigger plus Cloud Run Jobs for scheduled tasks, which lands in the $20–$40 range.

Appendix D — Glossary

- **CDP** — Consumable Data Product. Narrow, contracted views derived from FDP for downstream consumers.
- **DLQ** — Dead-Letter Queue. Pub/Sub destination for messages that fail to deliver.
- **FDP** — Foundation Data Product. The clean, business-shaped layer of BigQuery built from ODP.
- **Flex Template** — A Dataflow packaging format where the pipeline is a Docker image launched with parameters.
- **HDR/TRL** — Header/Trailer envelope on a mainframe extract file.
- **JOIN pattern** — A transformation that combines multiple ODP sources into one FDP table.
- **MAP pattern** — A transformation that maps one ODP source to one FDP table.
- **ODP** — Original Data Product. The untransformed BigQuery layer that mirrors mainframe extracts.
- **OTEL** — OpenTelemetry. Vendor-neutral observability framework.
- **PII** — Personally Identifiable Information.
- **Reconciliation** — The check that envelope counts, ingested counts, and BigQuery row counts agree.
- **Run ID** — The unique identifier for a single pipeline execution; threaded through every artefact.
- **System** — A logical grouping of entities sharing infrastructure. The reference implementation has one: `generic`.
- **Three-unit deployment model** — Ingestion, transformation, and orchestration as independently versioned, deployed, owned units.
- **Unit** — One of the three deployment units within a system.
- **WIF** — Workload Identity Federation. GCP's keyless authentication pattern for external systems (e.g. GitHub).

About the Author

Joseph Aruja is a Senior Lead Engineer based in Leeds, UK, with twenty-five years of hands-on experience building production systems for banks, government departments, retailers, transport authorities, automotive OEMs, healthcare bodies, and travel platforms.

He is a member of the JSR 255 (JMX) specification group within the Java Community Process, holds an MSc in Information Systems and the BCS Practitioner Certificate in Enterprise & Solution Architecture. He has worked across the full lifecycle of distributed-systems engineering — from architecture and solution design through to mentoring teams and writing production code himself, often on the same project.

His career has alternated deliberately between **regulated industries** and **scale-driven engineering**:

- **National-scale public services.** Technical lead for the NHS Spine Release 7A, contributing to ETP (Electronic Transmission of Prescriptions), DSA (Demographic Spine Application), QMAS, and XTS — services that, between them, underpin a substantial share of UK primary-care infrastructure. Designed the SAML-based single-sign-on framework reused across every Spine module, and built the proof-of-concept that replaced the Common Services Framework's custom messaging stack with Apache Camel.
- **Financial services.** Microservices for HSBC, First Direct, and M&S Bank across loans, current accounts, credit cards, and savings (12 live services under FCA compliance, reactive microservices on Spring Boot + RxJava + Kotlin). Currently Senior Lead Engineer on a financial-services mainframe-to-cloud migration. Earlier work on pricing automation for Allstate Insurance (Kafka-orchestrated rate-to-market platform).
- **Government compliance.** Home Office Asylum Services (microservices architecture for case-working), UK Border Agency Employment Checking Service (application security and integration), DWP Universal Credit Evidence subsystem, GOV.UK Home Office Framework (Audit Logging and Product Catalogue migration from Postgres to Redis).
- **Mainframe-to-modern integration.** This book's lineage starts here. Wm Morrison Supermarkets' Evolve programme (2009–2010): subject-matter expert for integrating Oracle Retail Price Management and Oracle Retail Merchandising with the legacy mainframe via the Oracle Retail Integration Bus, Oracle SOA

Suite, BPEL, AIA reference architecture, Oracle AQ, and custom HP OpenView monitoring.

- **High-volume consumer platforms.** Booking.com (migrating the on-prem booking engine to AWS EKS, building a scalable reservation-number-provider microservice, swapping Hystrix for Resilience4j), Caesars Digital / William Hill US (EPOS terminal estate, PCI-DSS-compliant authentication, AWS EKS).
- **Connected vehicles.** Jaguar Land Rover, twice. First on the VCS team (migrating event and historic-data frameworks to Java 17 / Spring Boot 3 on GCP); now on the Subscription Control Platform (re-architecting, building internal tooling, setting engineering standards).
- **Smart-city transport.** Smart Ticketing on Greater Manchester Metrolink for Transport for Greater Manchester (with Worldline) — ITSO smart cards and contactless EMV under PCI-DSS, OAuth2, multi-tenant Liferay portal, Spring Webflow, Spring Data + QueryDSL. Public-facing at getmethere.com.

He works fluently across the JVM (Java 8 through 21, Spring Boot, Spring Cloud, Dropwizard, Kotlin) and the Python data ecosystem (Python 3.7+, Apache Beam, dbt, Airflow). The framework this book describes — `gcp-pipeline-framework` — is the consolidation of patterns he has been writing variations of since 2009, when *data pipeline* was still called *integration* and the platform of choice was Oracle SOA Suite.

He writes about data engineering, GCP, and mainframe modernisation on Medium.

Contact: joseph.a.aruja@gmail.com **LinkedIn:** https://www.linkedin.com/in/josepharuja/

Index

Appendix E — Interviewing for the Work

A framework is only as durable as the next engineer who joins the team. This appendix is the interview format I run when hiring senior data engineers onto a pipeline platform like the one in this book — internal moves and external hires alike. It is not a question bank to be read out verbatim. It is a structure designed to surface, in one hour, whether the person sitting opposite you can actually do the work the rest of the book describes.

I have settled on **six primary questions over sixty minutes, with a three-person panel and one candidate at a time**. The panel I prefer is the engineering lead (you, if you have built or operate the platform), the product owner who lives with the data, and a peer engineer from the team the candidate would join. Each person leads two questions; everyone takes notes; nobody dominates. Two candidates back-to-back in a morning is a comfortable cadence and lets the panel calibrate against itself while the conversations are still fresh.

Why six questions in sixty minutes

Most engineering interviews fail in one of two directions. They either drown the candidate in trivia (name three Beam runners; what does `idempotency_key` do in `BigQueryIO.write`) or they ask one enormous question and run out of time before the answer goes anywhere interesting. Six questions is the sweet spot: enough breadth to cover ingestion, transformation, operations, security, and process; few enough that each conversation can actually breathe. Plan eight to ten minutes per question, leaving four minutes at the start for context and four at the end for the candidate's own questions.

The format below assumes a candidate who has worked on production data pipelines somewhere — not necessarily this stack, but something close enough that the vocabulary lands. If you are interviewing for a junior role, halve the depth and double the patience; the structure is wrong but the topics still cover the right ground.

Suggested timing

Block	Minutes	Lead
Welcome, context, role	4	Lead
Q1 — Ingestion design	9	Lead
Q2 — Remediation flow	8	Product owner
Q3 — Duplication: prevent and recover	9	Peer engineer
Q4 — dbt auth across environments	7	Lead
Q5 — Two-part data-quality scenario	11	Product owner
Q6 — CI/CD for data systems	8	Peer engineer
Candidate's questions	4	All

If a question runs long because the answer is genuinely good, take time off Q6 — not off the candidate's own questions. That last block is where people decide whether to accept your offer.

The six questions

The questions below are framed as prompts, what to listen for, where to push, and what counts as a red flag. The "what to listen for" sections describe *signals*, not vocabulary — score what the candidate understands, not which exact words they use. Two candidates can describe the same correct architecture in different terms; both should pass.

Q1 — Design an ODP-to-FDP pipeline from a fixed-width file

> *Lead asks.* "You are landing a daily fixed-width extract — say, 200 GB, twelve million records, a header and trailer with control counts — into a GCS bucket. The business wants a clean BigQuery table downstream. Walk me through the architecture, from the moment the file lands to the point an analyst can query a curated view."

Listening for. A clean separation between the *envelope* (header/trailer record-count reconciliation) and the *payload* (per-record parsing and validation). A landing-then-promote pattern, not direct-to-BigQuery writes. A bronze/silver/gold or ODP/FDP/CDP layering with a defensible reason for each layer. Schema-as-config rather than hand-rolled column lists. Reconciliation as a first-class step, not a footnote. A coherent story about where the work runs: Dataflow Flex Template for parsing because it scales, dbt-on-Composer (or equivalent) for the transformation, and a clear answer to "why those two and not one or the other".

Push on. What happens to records that fail validation — quarantine, dead-letter table, or hard fail the run? How is the run identified end to end, and how does that ID show up in the BigQuery rows themselves? Where does the schema live — code, a SQL migration, a config file, a contract document? How would the design change if the file arrives in five parts with `.ok` sentinels instead of one?

The no-golden-path follow-up. Once the main answer lands, drop this in: *"What if there's no precedent for this — you are the first team to build a pipeline from this source, the source team has not produced a schema for you, and nobody internally*

has done one of these before. Where do you actually start?" This is the senior signal you are testing for. You want to hear: ask the source team for a sample file and run statistics; profile it; propose a candidate schema; circulate it for review; build a small, throwaway pipeline against one day's data; only then commit to a long-term design. The opposite of "I'd look at the existing pipeline and copy it".

Red flags. Loading the raw file straight into the final BigQuery table. Treating validation as something to add later. No mention of reconciliation. Assuming the schema is fixed and known. Reaching immediately for Composer without considering whether the orchestration needs to be that heavy.

Q2 — Walk a remediation cycle

> *Product owner asks.* "Yesterday's run finished. This morning we discovered three thousand rows in the foundation table have the wrong account-type code — a downstream business team noticed before we did. The data has already flowed into a consumable view. What do you do, in what order, with whom, on what timeline?"

Listening for. Calm, ordered thinking. Stakeholder communication first — "I tell the consumer-facing team we have a known issue and freeze the downstream view" — before any technical remediation. Then a triage step: how many rows, which entity, when did it start, is the source data wrong or did the transformation lie? Then a reversible fix: rerun the affected partition rather than patch values by hand. Then a verification step before unfreezing. Finally a post-incident note that ends up in a runbook, not a Slack thread.

Push on. Who owns the call about whether to roll back? What is the SLA promised to the downstream consumers, and does this incident breach it? How would the team know if the bug had been in production for six months instead of one day — and is that a different incident class? What guardrail would prevent the next instance?

Red flags. Reaching for `UPDATE` statements as the first option. No mention of the consumer-facing team until the technical fix is done. No clear distinction between "the data was wrong on arrival" and "we processed correct data incorrectly".

Q3 — Prevent and recover from duplication in a dbt foundation model

> *Peer engineer asks.* "You own an FDP table — a foundation customer model — built with dbt-bigquery as an incremental model. Two-part question. First: how do you design it so duplicates cannot survive a partial rerun? Second: imagine duplicates have leaked in anyway — twenty thousand rows over a week. How do you recover, in production, without taking the table offline?"

Listening for, part one. `unique_key` declared on the incremental model, with `incremental_strategy='merge'`. An explicit understanding of *why* merge is right here and append is wrong. Awareness that `unique_key` is a model-level setting, not a column-level one, and that getting the natural key wrong is the most common cause of duplication in dbt. A passing reference to dbt tests — `unique`, `not_null` — as a CI

guardrail. Optionally: window-function dedupe (`row_number() over (partition by key order by updated_at desc)`) inside the model itself as a belt-and-braces layer.

Listening for, part two. A reversible recovery path. Build a candidate clean table in a separate dataset, validate row counts and checksums against expected, then atomically swap (rename, or `create or replace table` from select). Keep the original table around until the swap has been verified for at least a day. *Never* delete from a foundation table directly: the cure causes the next incident. A note that the same recovery script should be parameterised by date range so the next time this happens it is a five-minute job, not a four-hour scramble.

Push on. What does the table look like during the swap — is there a window where downstream queries see an empty or partial result? How does the audit trail know that twenty thousand rows were retired rather than processed? Would you do this differently if the table were a Type 2 SCD rather than a current-state model?

Red flags. `DELETE FROM` as the recovery plan. No backup of the dirty table. No verification step before the swap. Confusion between `unique_key` and a BigQuery primary-key constraint (BigQuery does not enforce primary keys — `unique_key` is a dbt-level convention).

Q4 — dbt authentication across environments

> *Lead asks.* "Your dbt project runs in three environments — a developer's laptop, a shared dev project in CI, and production. Authentication is different in each. How do you actually wire it up so the same dbt code works everywhere, with no service-account JSON keys downloaded to anyone's machine?"

Listening for. `profiles.yml` with three targets — `dev_local`, `dev_ci`, `prod` — selected by an environment variable. On the laptop, **user OAuth** via `gcloud auth application-default login`; in CI, the GitHub Actions runner authenticates via **Workload Identity Federation**; in both non-laptop cases, the configured target uses `impersonate_service_account` to assume a dbt-specific service account, with the human or runner identity holding `roles/iam.serviceAccountTokenCreator` on that SA. The production SA has tightly scoped BigQuery permissions and nothing else. A clear statement that nobody — *nobody* — downloads a JSON key.

Push on. How does a new developer get set up on day one? What permissions do they need in the dev project — exactly? How is the production SA permissioned compared to the dev SA, and why is the difference what it is? How would the team know if someone tried to bypass impersonation and authenticated with a downloaded key instead?

Red flags. "We put the JSON key in a `.env` file." "We share one developer SA across the team." Treating impersonation as something dbt does opaquely rather than as an explicit `profiles.yml` setting. Not knowing the difference between `roles/iam.serviceAccountUser` and `roles/iam.serviceAccountTokenCreator`.

Q5 — Validate a long-lived consumable data product

> *Product owner asks, then pauses for one part at a time.*

> **Part A.** "Imagine we have built a bespoke consumable data product for a business team — let's say a unified customer view aggregated from four foundation tables. It has been live in production for six months. Tens of millions of rows, growing daily. How would you validate the *quality* of the data in that product, on an ongoing basis, while it is in production?"
>
> **Part B.** "Now a separate but related question. The product owner has signed off the field mapping — every column in the CDP came from a specific source field somewhere upstream. How would you validate, in production, that the *mapping* is correct — that each field in the CDP genuinely is the right field from the source, not just a plausible-looking value?"

Listening for, Part A. Continuous data-quality checks rather than one-shot validation. Row-count reconciliation against expected upstream volumes, day over day. Schema drift detection: nullability, type, cardinality of categorical fields. Distribution checks: anomaly detection on key business metrics (mean, p50, p95, count of distinct customers per day) versus a rolling baseline. Referential checks against the upstream foundation tables. A dashboard the product owner can actually look at, not just a CI test that fires once a release. A clear treatment of *test data versus production data* — the senior answer recognises that quality in a six-month-old production product cannot be assessed from synthetic fixtures. **You are listening for the phrase "live data", or its equivalent.** A junior engineer reaches for unit tests; a senior engineer reaches for sampled production data with PII handled carefully.

Listening for, Part B. This is the harder half, and the one that separates strong candidates. Mapping validation is not the same as quality validation — it asks whether the *plumbing* is right. The senior answer is: take a statistically meaningful sample of records from the CDP, trace each mapped field back to its claimed source, and *compare the live value in the CDP against the live value in the source table for the same record*. Anything else — schema docs, contract files, code review — only proves the mapping was *declared* correctly, not that the data actually flows through it correctly. Watch for the candidate to volunteer this phrasing — *use live data, compare against the source* — without prompting. That is the senior signal worth hiring on.

Push on. How big is "a statistically meaningful sample" here, and why? What do you do when the source field has been transformed (formatting, currency conversion, code lookup) before it lands in the CDP — does the comparison need to invert the transformation? How would you keep this check running on a schedule rather than as a one-off audit?

Red flags. "We have a contract file; that documents the mapping." "We write unit tests with synthetic data." Confusing data quality (is the value plausible?) with mapping correctness (is the value actually from the right source?). Not naming live production data as the source of truth.

Q6 — CI/CD for a data system

> *Peer engineer asks.* "Sketch the CI/CD pipeline for the system we have been talking about — without referring to any specific tool stack unless I ask. What stages does it have, what does each stage gate, and what is the deploy unit?"

Listening for. A clean stage progression: format and lint, unit tests, integration tests against an emulator or short-lived test project, build the deploy artefact (Docker image, dbt manifest, library wheel), deploy to a non-production environment, run a smoke test there, then promote to production behind a manual gate or canary. A clear notion of *what gets versioned* — semantic version on libraries, immutable image tags on services, environment-specific configuration kept out of the artefact itself. A separation between *publishing* (a library reaches an artefact registry) and *deploying* (a running system picks up a new version). Authentication via Workload Identity Federation, not stored keys.

Push on. What makes you choose trunk-based development versus GitFlow for this kind of system, and why? Where does the rollback live — is it a redeploy of the previous tag, a feature flag, a schema migration that has been written reversibly? How does the pipeline behave differently for a documentation change versus a library change versus an infrastructure change — are all three really running the same workflow, or do you path-filter them? What stops someone pushing directly to main and bypassing the gates?

Red flags. "We deploy on every commit to main." (No, you do not — or you do, and you have an incident every Friday.) No distinction between publishing and deploying. No mention of how secrets reach the runner. Treating CI/CD as a single workflow file rather than a layered set of gates.

Calibration notes

A few things worth saying explicitly before the panel runs the morning.

Score concepts, not vocabulary. A candidate who says "we'd use a load-then-promote pattern" and a candidate who says "we'd stage the file in raw, then transform into a curated table" have given the same answer. Mark both up. The interview is not a spelling test.

Numbers are ballpark, not exam-question. If you ask how big a Dataflow worker pool needs to be for 200 GB and the candidate says "twenty workers for two hours, maybe ten if I tune shuffle", that is a fine answer even if the truth on your specific platform is fifteen for an hour. They are showing they have a model; the model can be calibrated on day one.

Time-box ruthlessly. If a question is going badly at the seven-minute mark, move on with a soft reset ("let me take us to the next one"). The interview's job is to surface enough signal for a hire/no-hire call, not to be exhaustively complete.

One panel, one bar. Before the candidates arrive, the three interviewers agree what a passing answer looks like for each question. After both candidates have been through, the three of you score in private, then compare. The most common failure mode of unstructured panels is interviewers drifting to different bars without realising; pre-agreed rubrics prevent that.

Read for the senior signal in Q5 explicitly. That is the question this format is really designed around. Q1 to Q4 and Q6 establish that the candidate can do the work; Q5 establishes whether they can do it *with judgement*. If a candidate answers

Part B without ever volunteering "use live data", they are not a no-hire — but they need the follow-up, and the score reflects that they needed it.

What good looks like

A strong candidate, summarised: walks the ODP/FDP design without reaching for buzzwords; treats validation and reconciliation as built-in stages, not bolt-ons; defaults to reversible operations on production data; names `impersonate_service_account` without prompting; volunteers live-data comparison for mapping checks; distinguishes publish from deploy; pushes back on at least one of the panel's assumptions politely.

A weak candidate gives the right answers in vocabulary but cannot explain *why* underneath. Probe with "what would go wrong if we did the opposite" — the gap shows up immediately.

Key takeaways

- Six questions, sixty minutes, three-person panel — enough breadth to cover ingestion, transformation, operations, security, data quality and process without rushing any single conversation.
- Q5 (the two-part long-lived-CDP scenario) is the senior-judgement question. The signal to listen for is the candidate volunteering *live production data* as the source of truth for both quality and mapping checks.
- Score the concept, not the vocabulary. Two strong candidates can describe the same correct architecture in entirely different words.
- Calibrate before the candidates arrive: pre-agree the bar, pre-agree the rubric, pre-agree who leads what. Unstructured panels drift apart; structured panels converge.
- Reserve the last four minutes for the candidate's own questions, even when running over. Their questions are where they decide whether to accept your offer.

Appendix F — Further Reading

Every framework rests on the shoulders of prior work. The patterns in this book didn't emerge from a vacuum: they were assembled from official documentation, a handful of essential books, and years of reading other people's post-incident write-ups at two in the morning. This appendix collects the resources I keep returning to — the ones that changed how I think, not just what I type. It's opinionated by design: where two resources cover the same ground, I've kept the one I'd reach for first.

Google Cloud documentation that's actually useful

Most Cloud documentation is written for people who have never touched a computer. These are the exceptions.

The **Cloud Storage best practices guide** (`cloud.google.com/storage/docs/best-practices`) covers object naming for throughput, lifecycle rules, and retention policies — read it before you design your landing-zone layout, not after. The **BigQuery cost-control guide** (`cloud.google.com/bigquery/docs/best-practices-costs`) is the single most useful page in the entire BigQuery docs; the slot vs. on-demand decision alone is worth the ten minutes. The **Dataflow Flex Templates documentation** (`cloud.google.com/dataflow/docs/guides/templates/using-flex-templates`) explains the Docker-image packaging model clearly, including the metadata JSON schema that trips everyone up on first contact. The **Cloud Composer 2 architecture overview** (`cloud.google.com/composer/docs/composer-2/composer-versioning-overview`) is worth reading even if you decide not to use Composer — it explains the GKE-backed model and why the cost profile is what it is. The **Workload Identity Federation guide** (`cloud.google.com/iam/docs/workload-identity-federation`) is the authoritative reference for keyless authentication from GitHub Actions; don't let anyone on your team reach for a downloaded service-account key instead. Finally, the **IAM least-privilege patterns** section of the IAM best practices guide (`cloud.google.com/iam/docs/using-iam-securely`) is the page to send to colleagues who are still assigning `roles/editor` because it's easier.

Apache Beam canon

The **Apache Beam programming guide** (beam.apache.org/documentation/programming-guide) is the starting point — it covers PCollections, transforms, windowing, and triggers more clearly than any third-party tutorial. Read it in order, at least once. The **streaming guide** (beam.apache.org/documentation/sdks/python-streaming) fills in

what the programming guide elides about unbounded sources and watermarks; if you're doing anything with Pub/Sub you need both. The **I/O connectors catalogue** (beam.apache.org/documentation/io/built-in) saves you from reinventing connectors that already exist — check it before writing a custom source. Most important of all: *Streaming Systems* by Tyler Akidau, Slava Chernyak, and Reuven Lax (O'Reilly). Akidau led the Dataflow and Beam teams at Google and the book is the intellectual foundation for everything modern stream-processing is built on. If you're going to read one book about data pipelines that isn't this one, make it that one.

dbt and analytics engineering

The **dbt Fundamentals course** (courses.getdbt.com) is free, takes a day, and is the fastest way to go from "I know SQL" to "I understand why dbt exists". Don't skip it on the grounds that you've already read the docs — the course gives you the mental model; the docs give you the API. The **dbt-bigquery adapter documentation** (docs.getdbt.com/docs/core/connect-data-platform/bigquery-setup) covers BigQuery-specific configuration: partitioning, clustering, job labels, and the service-account vs. OAuth credential decision. The **dbt Labs engineering blog** (getdbt.com/blog) publishes posts that are genuinely technical rather than marketing copy; the articles on incremental models and on testing strategy are particularly worth bookmarking. Coalesce — dbt Labs' annual conference — puts its talks on YouTube; the sessions from the past two years on multi-project mesh architecture and semantic layer are the clearest thinking available on where analytics engineering is heading. Tristan Handy's writing on the modern data stack (searchable on his Substack and older posts on getdbt.com) is worth reading for the framing — even where you disagree, it sharpens your own position.

Apache Airflow

The **official Airflow documentation** (airflow.apache.org/docs) is dense but complete. The DAGs-as-code model, the executor options, and the XCom semantics are all explained there; the problem is knowing which sections matter. Marc Lamberti's **Complete Apache Airflow Training** (available via Udemy) is the fastest shortcut to productive with Airflow in a week — his examples are realistic and his explanations of the scheduler internals are the clearest I've found outside of reading the source code. Astronomer's blog on the **dbt-on-Airflow pattern** (astronomer.io/blog) covers the `BashOperator` vs. `DbtRunOperator` trade-off and the task-level retry semantics that catch people out when dbt models fail halfway through a DAG run. Finally, Maxime Beauchemin's 2016 article *The Rise of the Data Engineer* (searchable on Medium) is dated on tooling but still correct on the discipline — it's the text that named the role, and reading it explains why Airflow was designed the way it was.

Mainframe modernisation

IBM's **Mainframe Modernization Reference Architecture** documentation is the canonical starting point for understanding what you're moving *from* — the LPAR model, the storage hierarchy, the batch-job scheduling assumptions. The **FINOS**

Open Mainframe Project (openmainframeproject.org) hosts working groups and open-source tools aimed at exactly this migration space; the COBOL-to-cloud working group materials are useful even if you're not touching COBOL directly. For BigQuery migration patterns specifically, Mike Owens' writing on DB2-to-BigQuery migration (published across the Google Cloud blog and Medium) covers the schema-mapping decisions that cause the most pain: packed decimal fields, zone decimal, and null representation. Phil Wainewright's articles on EBCDIC handling — and specifically the character-set conversion edge cases that bite EBCDIC-to-UTF-8 migrations — are the most practical treatment I've found of a subject most documentation glosses over. If you're parsing mainframe copybooks programmatically, the *com.legstar* library documentation (legstar.com) is the reference you need; it's not glamorous, but it's the most complete treatment of COBOL record-layout parsing available in the open-source ecosystem.

FinOps for data platforms

The **FinOps Foundation framework** (finops.org/framework) establishes the vocabulary and the crawl-walk-run maturity model that makes it possible to have a coherent conversation with finance teams about cloud spend. Start here if your organisation doesn't already have shared language. Google's **BigQuery cost-management whitepaper** (downloadable from cloud.google.com) goes deeper than the best-practices page listed above, covering reservation commitments, flex slots, and the capacity vs. on-demand modelling spreadsheet that engineering managers actually need. *Cloud FinOps* by J.R. Storment and Mike Fuller (O'Reilly, second edition) is the book that turned cost management from a niche concern into a discipline; the chapters on showback, chargeback, and anomaly detection are directly applicable to any team running data workloads at scale. The GCP documentation on **budget alerts and label-based cost attribution** (`cloud.google.com/billing/docs/how-to/budgets`) closes the loop between the theory and the implementation — labels on Dataflow jobs, Composer environments, and BigQuery reservations are how you turn the FinOps framework into actual line items on an invoice.

Observability

Observability Engineering by Charity Majors, Liz Fong-Jones, and George Miranda (O'Reilly) is the book that finally articulated the difference between monitoring and observability in terms that change how you instrument code, not just how you think about it. Read Chapters 1 through 4 before you write your next structured log line. The **OpenTelemetry specification** (opentelemetry.io/docs/specs) is the reference for the semantic conventions that make traces and metrics portable across backends — knowing the spec is what lets you instrument a Beam pipeline in a way that works with both Google Cloud Trace and a future Grafana setup. The **Google SRE Books** — *Site Reliability Engineering* and *The Site Reliability Workbook* — are free online at sre.google and remain the most coherent treatment of reliability as an engineering discipline rather than an operations concern. *Distributed Tracing in Practice* by Austin Parker, Daniel Spoonhower, Jonathan Mace, Ben Sigelman, and Rebecca Isaacs (O'Reilly) bridges the gap between the OTel specification and the practical realities

of instrumenting a multi-service pipeline — the chapter on sampling strategy is particularly useful.

Software architecture and engineering culture

Building Evolutionary Architectures by Neal Ford, Rebecca Parsons, and Patrick Kua (O'Reilly) introduced the concept of fitness functions as architectural guardrails; it's the most useful framing for why the framework's structured tests exist as they do. *Accelerate* by Nicole Forsgren, Jez Humble, and Gene Kim (IT Revolution) provides the empirical basis for the deployment pipeline decisions in this book — if you need to justify trunk-based development and small, frequent releases to a sceptical stakeholder, the DORA metrics research it contains is your evidence. *A Philosophy of Software Design* by John Ousterhout (Yaknyam Press) is the book I wish I'd had in 2001; its treatment of deep vs. shallow modules is the clearest articulation I've read of why the framework's three-unit decomposition lands where it does. *Domain-Driven Design Distilled* by Vaughn Vernon (Addison-Wesley) condenses the bounded-context thinking that informs how the reference implementation draws its system and entity boundaries.

Communities worth lurking in

r/dataengineering on Reddit is noisy but has a high signal floor — the weekly "show and tell" threads aside, the technical questions surface real production problems. The **Apache Beam Slack** (the invite link is on beam.apache.org) has active channels where Beam committers answer questions; it's where I'd go before filing a JIRA. The **dbt Slack** (getdbt.com/community) is large and well-moderated; the `#troubleshooting` and `#best-practices` channels are genuinely useful. The **Apache Airflow Slack** mirrors that pattern. **Locally Optimistic** (locallyoptimistic.com) is a newsletter and Slack community aimed specifically at data practitioners inside companies rather than vendors selling to them — the perspective is refreshingly honest. **Data Engineering Weekly** (dataengineeringweekly.com) is a curated newsletter with a consistent editorial eye; Ananth Packkildurai's selection leans towards the architectural and the practical rather than the promotional. As for **Towards Data Science** on Medium: read it, but filter. The quality variance is extreme — treat it as a discovery mechanism, not a source of truth, and always verify anything operationally important against official documentation.

Colophon

This book was written in Markdown and rendered to PDF using Pandoc with the XeLaTeX engine. The body type is Georgia and code is set in Menlo. The cover was assembled with care; the copy was edited at the kitchen table.

The codebase described in this book is `gcp-pipeline-framework` 1.0.29. It lives, at the time of writing, on the public PyPI index and in the `gcp-pipeline-reference` repository.

Errata, suggestions, and improvements are welcome.